

Design and Implementation of a Real-Time Impedance Controller for a 7-DOF Collaborative Robot

*Entwurf und Implementierung eines Echtzeit-Impedanzreglers für einen
7-achsigen kollaborativen Roboter*

Master Thesis

submitted for the degree of Master of Science (M.Sc.)

submitted at

Hochschule München

Faculty of Mechanical, Automotive and Aeronautical Engineering

submitted by

Heykel Khadhraoui

Study programme: Computational Engineering (M.Sc.)

Matriculation number: 41192924

Main supervisor: Prof. Norbert Nitzsche

Second supervisor: Prof. Ruth Otto

Issue date: 1 May 2026

Submission date: 1 November 2026

Declaration of Authorship / Verfassererklärung

I hereby declare that this master's thesis entitled

*“Design and Implementation of a Real-Time Impedance Controller for a 7-DOF
Collaborative Robot”*

was written independently and without assistance from third parties. All sources, references, software tools, and other resources used have been cited accordingly. This work has not previously been submitted as an examination requirement in this or any other form.

Hiermit erkläre ich, dass ich die vorliegende Masterarbeit selbstständig und ohne fremde Hilfe angefertigt habe. Alle verwendeten Quellen, Hilfsmittel, Softwarewerkzeuge und sonstigen Ressourcen wurden entsprechend angegeben. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde in gleicher oder ähnlicher Form vorgelegt.

Munich, _____

Date

Heykel Khadhraoui

Signature

Colour-Coded Thesis Review

This copy is an editorial and evidential review of the thesis. It carries two kinds of mark. The first four colours appear as boxes and assess the section or subsection that follows the corresponding heading. They do not alter the thesis text, equations, tables, figures, or references.

GREEN The argument and presentation are sufficient for the present thesis. Minor polishing may still be possible.

YELLOW The content is relevant, but the explanation is dense, repetitive, weakly supported, or harder to follow than necessary.

RED The passage retains a generic, templated, or textbook-restatement pattern that can read as machine-produced. This is a stylistic assessment, not a claim about authorship or an AI-detector result.

GREY A reasoning step, evidence link, validation, or required measurement is absent, uncertain, or inconsistent elsewhere.

Green means sufficient, not perfect. A grey assessment does not make the reported data invalid; it identifies where the current evidence does not fully support the surrounding interpretation. Red is reserved for prose structure rather than technical correctness. Where a section contains both sound work and one important weakness, the more serious colour is used and the box names the exact concern.

The remaining two colours record the state of the text rather than an assessment of it, and are therefore applied to the words themselves.

PURPLE The passage has been read, checked and settled. Its wording is final and is not edited again.

BLUE Either new thesis text in a passage not yet settled, or a comment raised against

a settled passage.

Purple is not a judgement that the passage is strong. It records that the passage has been controlled and found correct, and that it already satisfies what the thesis requires of it. Nothing inside it is reworded, tightened, extended, or restructured afterwards. Rewording, sharpening, and stylistic improvement are not grounds for changing a settled passage, and a revised section that carries no blue at all is the normal case rather than an oversight.

Where a settled passage appears to need something, the text is left as it stands and the concern is raised beside it instead, as

The clearance gate holds the pose the tool reached and freezes the descend clock, until a bare Enter releases it. **comment:** “the frozen clock also governs the grind gate, which may be worth stating here”

A comment is a remark about the text, not a part of it. It appears only in this annotated copy and never in the submitted thesis, so a settled passage reads identically in both. It is raised only where the context or the reasoning is genuinely incomplete without the addition, not for anything that would merely read better.

Blue is used directly on the words themselves only in passages that have not yet been settled, where it marks new text as a whole sentence or a whole word and ends in a raised +. The two tints differ in hue so that the boundary survives greyscale printing.

The clean submission files remain `Thesis.pdf` and `Professor_Draft.pdf`. This annotated copy is generated separately as `Review_Draft.pdf`.

Abstract

GREEN — sufficient. The abstract states the problem, implemented mechanism, principal measured ordering, null-space result, and scope limits without overloading the reader with the full parameter matrix.

A small angular deviation causes one edge of a tool to contact a surface first. Under rigid position control the robot resists the rotation that would make the surfaces parallel, so contact forces and moments develop instead.

A real-time Cartesian impedance controller with phase-dependent stiffness and damping was implemented for a seven-degree-of-freedom (DOF) Franka Emika Panda. During contact set-up, gains defined at a virtual centre of compliance were transformed to the tool centre point; the resulting off-diagonal terms coupled translation and rotation, so the commanded normal press generated a corrective moment.

Commanded tilt, directional stiffness, and compliance-centre position were varied experimentally. Within the investigated range, the in-plane position of the compliance centre produced the largest change in alignment. A lever applied in the direction that assisted the corrective rotation gave approximately 6° of improvement in both tangent-axis tests, and the required direction was opposite for t_1 and t_2 . Increased rotational stiffness reduced the alignment response, with axis-dependent differences between the softer settings.

The lever-induced moment keeps a fixed direction, whereas the rotational spring follows the measured error. A tangential lever therefore assists one tilt sign and opposes the other. Almost no improvement remained when it opposed the required rotation, so its direction must be selected from the measured tilt.

Separate free-space hold trials showed that projected null-space damping reduced redundant joint motion. A stronger singular-value conditioning torque increased joint motion without producing a larger final conditioning improvement.

The results apply to one robot, tool mounting, and surface. Every commanded tilt had the same sign, so the lever conclusions are limited to that sign. The end-effector-based metric may also underestimate the physical rotation of the grinding face.

Kurzfassung

GREEN — sufficient. The German summary remains aligned with the English abstract in structure, certainty, findings, and limitations.

Eine kleine Winkelabweichung führt dazu, dass eine Werkzeugkante die Oberfläche zuerst berührt. Bei starrer Positionsregelung behindert der Roboter die ausrichtende Drehung, sodass Kontaktkräfte und -momente entstehen.

Für einen 7-achsigen Franka Emika Panda wurde ein phasenabhängiger kartesischer Echtzeit-Impedanzregler implementiert. Beim Kontaktaufbau wurden Steifigkeit und Dämpfung vom virtuellen Nachgiebigkeitszentrum zum Werkzeugmittelpunkt transformiert. Die resultierende Kopplung wandelte den kommandierten Normaldruck in ein korrigierendes Moment um.

Vorgegebene Verkipfung, richtungsabhängige Steifigkeit und Lage des Nachgiebigkeitszentrums wurden experimentell variiert. Im untersuchten Bereich hatte dessen Lage in der Ebene den größten Einfluss. Ein unterstützender Hebel verbesserte die Ausrichtung in beiden Tangentenachsen-Versuchen um etwa 6° ; die erforderliche Richtung war für t_1 und t_2 entgegengesetzt. Eine höhere Rotationssteifigkeit verringerte die Ausrichtungsreaktion; Unterschiede zwischen den weichen Einstellungen waren achsenabhängig.

Das Hebelmoment behält eine feste Richtung, während die Rotationsfeder dem gemessenen Fehler folgt. Ein tangentialer Hebel unterstützt daher ein Vorzeichen der Verkipfung und wirkt dem anderen entgegen. In der entgegenwirkenden Lage blieb nahezu keine Verbesserung, weshalb die Hebelrichtung aus der gemessenen Verkipfung gewählt werden muss.

In getrennten Halteversuchen im freien Raum verringerte projizierte Nullraumdämpfung die redundante Gelenkbewegung. Ein stärkeres Konditionierungsdrehmoment erhöhte die Bewegung ohne weitere Konditionierungsverbesserung.

Die Aussagen gelten nur für die untersuchte Anordnung und ein Verkippungsvorzeichen;
die EE-Kennzahl kann die Drehung der Schleiffläche unterschätzen.

Contents

Declaration of Authorship / Verfassererklärung	II
Colour-Coded Thesis Review	III
Abstract	V
Kurzfassung	VII
List of Figures	XV
List of Tables	XVI
List of Abbreviations	XVIII
List of Symbols	XIX
1 Introduction	1
1.1 Motivation	1
1.2 Related Work	2
1.3 Problem Statement	3
1.4 Scope and Contributions	4
1.5 Thesis Structure	6
2 Theoretical Background	7
2.1 Reference Frames	7

2.2	Rigid-Body Transformations	8
2.2.1	Rotation Matrices	8
2.2.2	Homogeneous Transformations	9
2.2.3	End-Effector Pose	10
2.2.4	Cartesian Pose Error	10
2.3	Kinematics of a 7-DOF Manipulator	12
2.3.1	Forward Kinematics	12
2.3.2	Differential Kinematics	13
2.3.3	Geometric Jacobian	14
2.4	Cartesian Impedance Control	15
2.4.1	Impedance Wrench	15
2.4.2	Joint-Torque Control Law	16
2.4.3	Coriolis and Gravity Compensation in libfranka	17
2.5	Surface Task Frame	17
2.6	Stiffness and Damping Representation	18
2.7	Centre of Compliance	20
2.8	Null-Space Torque	23
2.8.1	SVD-Based Null-Space Characterisation	23
2.8.2	SVD-Based Pseudoinverse	26
2.8.3	Null-Space Projectors	26
2.8.4	Null-Space Damping Torque	28
2.8.5	Singular-Value Conditioning Torque	28
2.8.6	Complete Null-Space Torque	30
2.9	Complete Joint-Torque Command	30
3	Controller Design and Implementation	32
3.1	Software Architecture and Program Execution	33
3.2	Real-Time Control Loop	35

3.3	Implemented Cartesian Impedance Controller	38
3.4	SVD-Based Null-Space Control	41
3.5	Surface and Tool Geometry	45
3.5.1	Surface Task Frame	45
3.5.2	Surface-Relative Tool Orientation	47
3.5.3	Rectangular Grinding Face and Clearance	49
3.6	Phase-Dependent Gains and Inertia-Scaled Damping	52
3.6.1	Phase-Dependent Gain Construction	52
3.6.2	Calculation of Inertia-Scaled Damping	55
3.7	Virtual Centre of Compliance	58
3.8	Control Modes and Phase Sequence	61
3.8.1	Tool Orientation	62
3.8.2	Surface Approach	63
3.8.3	Set-Up	64
3.8.4	Grinding	66
3.8.5	Pose Hold and Manual Guidance	67
3.8.6	Run Termination	68
3.9	Robot Preparation and Safety Handling	68
3.9.1	Robot Connection and Collision Configuration	69
3.9.2	Initial Joint Posture	70
3.9.3	Gripper Verification	71
3.9.4	Tool Transfer	72
3.9.5	Exception Handling	73
3.10	Data Recording and Run Evaluation	73
3.10.1	Chronological Reconstruction and CSV Output	77
3.10.2	Set-Up Evaluation	79
4	Experimental Methodology	81

4.1	Experimental Investigation	81
4.2	Robot and Software Configuration	84
4.2.1	Experimental System and Reproducibility	85
4.2.2	Configured and Physical Plane	86
4.3	Contact Geometry	87
4.4	Gain Selection Methodology	89
4.4.1	Quasi-Static Stiffness Sizing and Inertia-Scaled Damping	90
4.4.2	Compliance-Centre Gain Design	94
4.4.3	Gain Construction and Implementation Sequence	97
4.4.4	Experimental Gain Variation Strategy	97
4.4.5	Null-Space Settings	99
4.4.6	Internally Commanded Null-Space Disturbance Study	100
4.5	Experimental Procedure	101
4.6	Evaluation Metrics	103
4.6.1	Metric Use by Experimental Case	104
4.7	Safety Limits	105
5	Results and Discussion	107
5.1	Repeatability and Settling	107
5.2	EE-Inferred Alignment Against the Physical Plane	108
5.3	Case A: Commanded Tilt	108
5.4	Case B: Axis-Specific Rotational Stiffness	110
5.5	Case C: Cross-Axis Translational Stiffness	111
5.6	Case D: Virtual Centre of Compliance	112
5.7	Case E: Tilt About the Tool Axes	114
5.8	Internally Commanded Null-Space Response	115
5.9	Discussion	116
5.9.1	Relative Influence of the Investigated Parameters	116

5.9.2	Geometric Interpretation of the Lever	117
5.9.3	Placement of the Virtual Centre of Compliance	118
5.9.4	Practical Implications for Controller Tuning	119
5.10	Limitations	120
5.10.1	Experimental Generalisability	120
5.10.2	Calibration and Measurement Uncertainty	120
5.10.3	Tool-Mount Compliance	120
5.10.4	Null-Space Validation and Timing Scope	121
6	Conclusion and Future Work	122
6.1	Limitations	123
6.1.1	Experimental Generalisability	123
6.1.2	Calibration and Measurement Uncertainty	124
6.1.3	Tool-Mount Compliance	124
6.1.4	Null-Space Validation	124
6.2	Future Work	124
	Bibliography	128
A	Controller Source Listings	129
A.1	Surface Task Frame and Spatial Gains	129
A.2	Tool Orientation and Rotation Error	130
A.3	Geometric Clearance and Phase Targets	130
A.4	Centre of Compliance and the Offset Adjoint	131
A.5	Task-Inertia Damping	132
A.6	Null-Space Torque	133
A.7	Core Control Callback	133
B	Recorded Experimental Data	135

B.1	Control Phases	135
B.2	Signals Used in the Evaluation	135
B.3	Evaluation Quantities	137
C	Controller Parameters	138
C.1	Surface and Tool Geometry	138
C.2	Phase Sequence and Impedance	139
C.3	Inertia-Scaled Damping and Centre of Compliance	141
C.4	Null-Space and Operating Parameters	142
D	Previous Contact-Alignment Campaign	143
D.1	Test Matrix	143
D.2	Settling and Repeatability	144
D.3	Paired Stiffness Results	145
D.4	Compliance-Centre Results	146
D.5	Interpretation Limits	148

List of Figures

4.1	Experimental set-up and surface-frame contact geometry.	86
4.2	Compliance-centre geometry during set-up.	96
5.1	Case-A alignment response by measured initial angle.	109
5.2	Case-B response to tilted-axis rotational stiffness.	110
5.3	Case-C response to cross-axis translational stiffness.	111
5.4	Case-D alignment response by compliance-centre lever.	112
5.5	Case-E response to surface-tangent and tool-axis tilt.	115
5.6	Null-space response under the internally commanded point-force equivalent.	116
D.1	Previous set-up settling comparison.	145
D.2	Previous paired tangent-axis stiffness sweep.	146
D.3	Previous response by compliance-centre coordinate.	147
D.4	Previous fitted in-plane response.	148

List of Tables

3.1	Software modules of the surface-grinding controller.	33
3.2	Coordinate frames used for the phase-dependent Cartesian gains.	53
3.3	Signal groups stored during the real-time control loop.	75
4.1	Completed experimental cases.	83
4.2	Experimental system.	85
4.3	Phase-specific stiffness and design points.	93
4.4	Phase-specific inertia-scaled-damping policy.	94
4.5	Null-space settings in the two campaigns.	99
4.6	Metric use by experimental case.	105
5.1	Case-A alignment response (mean $\pm$ standard deviation, $n = 3$).	109
5.2	Case-B alignment improvement by tilted-axis stiffness (deg).	110
5.3	Case-C alignment improvement by cross-axis stiffness (deg).	111
5.4	Case-D alignment improvement by perpendicular lever (deg).	112
5.5	Case-E improvement by tilt axis.	114
B.1	Recorded signal groups used in the evaluation.	136
C.1	Surface and tool parameters.	139
C.2	Phase and mode gains.	140
C.3	Phase-transition and trajectory parameters.	141
C.4	Null-space and operating parameters.	142

D.1	Previous contact-alignment cases.	144
D.2	Previous residual changes after 5 s.	145
D.3	Previous nominal repeatability ($n = 5$).	145
D.4	Previous paired-stiffness results ($n = 5$).	146

List of Abbreviations

GREEN — **sufficient.** The abbreviations are separated from the symbol list and their printed forms are controlled centrally.

- CSV Comma-Separated Values
- DOF Degree of Freedom
- FCI Franka Control Interface
- SVD Singular Value Decomposition

List of Symbols

GREEN — sufficient. The notation is extensive but well organised, and the unit column makes the mixed translational and rotational quantities traceable.

Frames and Kinematics

$\{0\}$	Robot base frame	[-]
$\{i\}$	Joint frame i	[-]
$\{EE\}$	End-effector (tool) frame	[-]
n	Number of joints ($n = 7$)	[-]
$q, \dot{q}$	Joint position / velocity vector	[rad], [rad/s]
q_0	Home or reference joint configuration	[rad]
x	Stacked Cartesian end-effector state, $x = f(q)$	[m], [rad]
$f(\cdot)$	Forward-kinematics map	[-]
x, y, z	Scalar Cartesian coordinate directions or components; not the stacked state x	[m]
p	Position / translation vector	[m]
R	Rotation matrix	[-]
R_x, R_y, R_z	Elementary rotations about x, y, z	[-]
α, β, γ	Generic rotations about the x, y, z axes	[rad]

T	Homogeneous transformation matrix	[-]
$J(q)$	Geometric Jacobian; linear rows above angular rows	[m/rad], [-]
z_{i-1}	Rotation axis of joint i , expressed in the base frame	[-]
p_{i-1}	Point on the axis of joint i , expressed in the base frame	[m]
x_{EE}, y_{EE}, z_{EE}	End-effector frame axes (base frame)	[-]

Pose and Error

p_{EE}	End-effector position	[m]
R_{EE}, T_{EE}	End-effector orientation / transformation	[-]
p_d, R_d, T_d	Desired position / orientation / transformation	[m], [-]
T_{err}	Relative (error) transformation	[-]
ΔR	Current-to-desired relative rotation,	[-]
	$R_{EE}^\top R_d$	
ϕ	Orientation-error angle	[rad]
$u, [u]_\times$	Current-EE-frame orientation-error axis / its skew matrix	[-]
e_p	Translational (position) error	[m]
e_R	Base-frame rotational error used by the command	[rad]
$e_{R,EE}$	Rotational error expressed in the current EE frame	[rad]
$e_{p,c}, e_c$	Compliance-centre position error / stacked error	[m], [rad]
e	Stacked Cartesian error	[m], [rad]
$\dot{x}$	Cartesian velocity (twist)	[m/s], [rad/s]
$\dot{p}_{EE}, \dot{p}_d, \omega_{EE}$	Measured linear end-effector velocity, desired linear velocity, and angular end-effector velocity	[m/s], [m/s], [rad/s]

Cartesian Impedance Gains

K_p, D_p	Generic translational stiffness / damping block; a frame subscript is added when needed	$[\text{N/m}], [\text{N s/m}]$
K_R, D_R	Generic rotational stiffness / damping block; a frame subscript is added when needed	$[\text{N m/rad}], [\text{N m s/rad}]$
R_{task}, T_R	Task-frame orientation and corresponding block rotation	$[-]$
$K_{p,\text{task}}, D_{p,\text{task}}$	Local translational stiffness / damping in the chosen task frame	$[\text{N/m}], [\text{N s/m}]$
$K_{R,\text{task}}, D_{R,\text{task}}$	Local rotational stiffness / damping in the chosen task frame	$[\text{N m/rad}], [\text{N m s/rad}]$
$K_{p,0}, D_{p,0}$	Translational stiffness / damping used in the base-frame command	$[\text{N/m}], [\text{N s/m}]$
$K_{R,0}, D_{R,0}$	Rotational stiffness / damping used in the base-frame command	$[\text{N m/rad}], [\text{N m s/rad}]$
K_0, D_0	Full 6×6 stiffness / damping used in the base-frame command	$K: [\text{N/m}], [\text{N m/rad}];$ $D: [\text{N s/m}], [\text{N m s/rad}]$

Dynamics and Torques

F	Cartesian wrench	[N], [N m]
f, m	Cartesian force / moment	[N], [N m]
K, D	Full Cartesian stiffness / damping matrices	K : [N/m], [N m/rad]; D : [N s/m], [N m s/rad]
Λ	Operational-space (Cartesian) inertia matrix	[kg], [kg m], [kg m ²]
$M(q)$	Joint-space mass matrix	[kg m ²]
M_i	Directional translational or rotational inertia	[kg] or [kg m ²]
ζ	Directional inertia-scaled-damping ratio used as a tuning heuristic	[-]
τ	Joint torque vector	[N m]
τ_c, τ_g	Coriolis / gravity compensation torque	[N m]
$\tau_{\text{cmd}}, \tau_{\text{task}}$	Commanded / task torque	[N m]

Surface and Contact Geometry

n_s, n_{phys}	Configured surface normal / independently calibrated physical-plane normal	[-]
t_1, t_2	Surface tangent directions (unit)	[-]
$a_s, \tilde{t}_1$	Auxiliary surface direction / projected tangent before normalization	[-]
R_{surface}	Surface task frame $[t_1 \ t_2 \ n_s]$ in the base frame	[-]
a_{EE}, a_T, a_d	Configured EE-frame tool axis / current base-frame tool axis / signed target axis	[-]
s_a	Tool-axis target sign, with $a_d = s_a n_s$	[-]
K_{t_1}, K_{t_2}, K_n	Translational stiffness along t_1, t_2 , and n_s	[N/m]
$K_{r,t_1}, K_{r,t_2}, K_{r,n}$	Rotational stiffness entries in surface-frame column order	[N m/rad]
$h_C, h_{\text{lead}}, \ell_{\text{lead}}$	Averaged-feature height / exact leading-corner height / leading descent projection	[m]
p_s	Point on the configured surface	[m]
p_e, a_e	Tool-edge point / edge direction	[m], [-]
$p_{\Pi, \text{clr}}$	Selected tool feature projected onto the plane at clearance	[m]
$c_{C, \text{EE}}, R_{\text{clr}}$	Selected-feature offset in the EE frame / orientation selected at clearance; both are held constant during set-up	[m], [-]
p_c	Centre of compliance, held constant from first contact	[m]

r_c	Pole-to-TCP lever, $p_{\text{TCP}} - p_c$	[m]
h	Selected normal offset of the centre of compliance	[m]
p_C	Physical contact point	[m]
r_C	TCP-to-contact lever, $p_C - p_{\text{TCP}}$	[m]
f_C	Environment-on-tool contact force used in the geometric sketch	[N]

Compliance and Alignment

$K_{\text{task}}, D_{\text{task}}$	Full local Cartesian stiffness / damping in the chosen task frame	K : [N/m], [N m/rad]; D : [N s/m], [N m s/rad]
$K_{pR,\text{task}}, K_{Rp,\text{task}}$	Task-frame translation–rotation coupling sub-blocks	[N/rad], [N m/m]
$\text{Ad}(r_c), \text{Ad}_g$	Pole-to-TCP point-shift adjoint / alternative notation for that offset	[-]
K_c, D_c	Block-diagonal stiffness / damping at the centre of compliance	K : [N/m], [N m/rad]; D : [N s/m], [N m s/rad]
$K_{\text{TCP}}, D_{\text{TCP}}$	Centre-of-compliance gains mapped by congruence to the TCP	K : [N/m], [N m/rad]; D : [N s/m], [N m s/rad]
K_{coupled}	Coupled full stiffness about the selected centre	[N/m], [N/rad], [N m/m], [N m/rad]
$e_{p,\text{task}}, e_{R,\text{task}}, e_{\text{task}}, \dot{e}_{\text{task}}$	Task-frame translational / rotational / stacked error and error velocity	[m], [rad], [m/s], [rad/s]
$\theta_{\text{align}}, \Delta\theta_{\text{align}}$	EE-inferred tool-axis alignment angle / before–after improvement	[rad]
$\Delta\theta_C$	Finite before–after rotation vector during contact	[rad]

Quasi-Static Stiffness and Damping Design

$k_{p,i}, k_{R,i}$	Principal translational / rotational stiffness	[N/m], [N m/rad]
$k_{p,\text{eff}}(u)$	Effective translational stiffness along unit direction u	[N/m]
d_i	Directional damping coefficient	[N s/m] or [N m s/rad]

Null-Space and Singular Value Decomposition

J_ρ^+	Truncated-SVD Moore–Penrose pseudoinverse	[rad/m], [-]
N_q, N_τ	Joint-velocity / torque null-space projector	[-]
ρ, ε_ρ	Relative SVD tolerance / resulting singular-value cutoff	[-], [m/rad]
$\sigma_i, \sigma_{\min}$	Jacobian singular value / σ_6 conditioning indicator; its value follows the length unit of the linear rows	[m/rad]
λ_i	Eigenvalue of $JJ^\top$, equal to σ_i^2	[m ² /rad ²]
U_J, Σ_J, V_J	Singular value decomposition factors of J	[-], [m/rad], [-]
u_i, v_i	Left / right singular vectors	[-]
v_6, v_7	σ_6 -associated direction / structural null direction	[-]
d_{null}	Null-space damping	[N m s/rad]
k_σ	Sigma-conditioning torque magnitude	[N m]
α_{probe}	Null-direction sampling angle	[rad]
$\delta_\sigma, \Delta_\sigma$	Sigma-comparison deadband / sampled difference	[m/rad]

s	Deadbanded conditioning-direction selector	[-]
$\tau_d, \tau_\sigma, \tau_{\text{null}}$	Projected damping / sigma / total null-space torque	[N m]

Centre-of-Compliance Notation

K_{p_c}, D_{p_c}	Full stiffness / damping matrix defined about the centre of compliance	K : [N/m], [N m/rad]; D : [N s/m], [N m s/rad]
p_c		
$K_{p,c}, K_{R,c}$	Translational / rotational stiffness blocks defined about the centre of compliance	[N/m], [N m/rad]
$D_{p,c}, D_{R,c}$	Translational / rotational damping blocks defined about the centre of compliance	[N s/m], [N m s/rad]

Miscellaneous

T_s	Control sampling time (1 ms)	[s]
I, I_4, I_7	Identity matrices	[-]

Chapter 1

Introduction

PURPLE — revised and confirmed. The chapter was revised in full and every paragraph is settled. No further editing is to be applied to it, and any remaining concern is raised as a comment rather than as a change.

1.1 Motivation

Industrial manipulators commonly apply position control to track commanded trajectories. In surface-finishing applications, the actual surface pose may differ from the programmed pose because of placement tolerances. The surface is positioned manually, the tool is held in a gripper, and the residual angle between them is unknown before contact. Under rigid position control, this angular deviation cannot be absorbed by the commanded motion and generates high contact forces.

Torque-controlled collaborative robots permit contact compliance to be specified directly, and they account for a growing share of industrial deployments [11, 21]. Joint torque sensing and a torque-level command interface let the controller specify how the robot yields under contact, not only where it goes. This capability is particularly useful in surface-finishing tasks, where a small angular mismatch between the tool and the surface can generate undesired contact forces and moments. The objective is therefore not only to reduce these loads, but also to enable passive angular alignment of the grinding face during contact. The controller should allow the tool to rotate toward the surface and reduce the angular mismatch while maintaining the commanded normal contact and the subsequent tangential grinding motion. The resulting alignment is evaluated relative to the physical

surface.

Impedance control specifies this yielding behaviour as a dynamic relation between motion and interaction force [10]. Cartesian impedance control defines this relation in task space, meaning in terms of the Cartesian position and orientation of the end-effector rather than the individual joint coordinates. The compliance can therefore be adjusted separately for different translational and rotational directions. This direction-dependent behaviour is important during contact. A tool moving across a surface should maintain its normal position, while small lateral deviations should not generate large forces. The same principle applies to orientation. When the edge of a tilted tool contacts the surface, the contact wrench contains both a force and a moment. Stiffness values chosen according to the contact geometry allow the tool to rotate slightly toward the surface instead of fully resisting this motion. The resulting angular response is evaluated from the end-effector pose relative to the physical surface. The stiffness and damping matrices therefore determine both the contact response and the free-space tracking behaviour.

1.2 Related Work

The relevant literature comprises three areas: Cartesian impedance control, real-time implementation on torque-controlled robots, and the placement of the centre of compliance. The first area is Cartesian impedance control itself. The controller uses standard serial-manipulator kinematics [19, 3, 15]: homogeneous transformations for representing the end-effector pose and the Jacobian for relating joint motion to Cartesian motion. Khatib established the operational-space formulation, in which a Cartesian wrench is mapped to joint torques through J^T [12]; this mapping is adopted in the controller. Hogan defined the impedance relation between end-effector motion and interaction force [10]. Albu-Schäffer et al. implemented Cartesian impedance control using joint-torque sensing [1], which corresponds to the torque-controlled interface used in this thesis.

The second area concerns the real-time implementation of Cartesian impedance control on torque-controlled robots. Mayr et al. implemented a modular Cartesian impedance controller for the Franka Emika Panda that combines real-time model access, null-space control, signal treatment, and command limiting [14]. The controller developed in this thesis uses the libfranka model interface to obtain the required robot state and model

quantities during the real-time control loop. It also includes a selectable null-space torque for influencing the redundant joint motion without changing the commanded Cartesian task. No separate application-level saturation of the commanded Cartesian wrench, joint torque, or torque rate was applied. The configured robot-side collision thresholds and collision monitoring therefore remained the independent mechanism for interrupting the motion when the corresponding limits were exceeded.

The third area concerns the location of the centre of compliance. Fasse and Broenink formulate spatial impedance about a selected point, so that translation and rotation become coupled through the position of this point [6]. This formulation provides the theoretical basis for the compliance-centre shift used in this thesis. The same principle appears mechanically in remote-centre-compliance devices, in which elastic elements are arranged so that the tool behaves as if it rotates about a virtual point located away from the mechanism itself [16, 23]. More recent mechanically compliant and variable-compliance-centre strategies apply this principle to peg-in-hole insertion [22, 9]. In the mechanically compliant mechanisms, elastic bending produces the alignment motion rather than software control alone. In these approaches, the compliance is arranged so that contact forces reduce misalignment instead of increasing it. Most of these studies focus on insertion tasks and define the compliance centre through either the mechanical design or a strategy adapted to the hole geometry. In this thesis, an equivalent effect is created in software by shifting the virtual centre of compliance away from the TCP. The independent effects of this position and the axis-specific stiffness during planar tool alignment are then investigated on a torque-controlled robot.

1.3 Problem Statement

The angular relation between the tool and the surface plane is uncertain before contact because both the surface placement and the tool mounting carry tolerances. Under rigid position control, this residual angle produces an undesired contact moment rather than a corrective motion. A tool tilted with respect to the surface cannot be brought flat by translational compliance alone, because the wrench that closes the gap must also contain a moment. Translational compliance therefore does not by itself produce the required corrective rotation. Placing the centre of compliance away from the tool centre point (TCP)

couples translation and rotation, so that pressing the tool into the plane also causes it to rotate toward the plane. This placement defines a task-to-impedance mapping rather than a physical pivot. The resulting response was evaluated from the motion measured during contact.

The alignment response cannot be predicted from the gain values alone because it also depends on the contact geometry, the lever direction, the robot configuration, and the compliance of the tool mounting. The experiments therefore compare the effects of a commanded tilt representing the initial angular mismatch between the tool and the surface, the directional stiffness, and the compliance-centre placement under otherwise matched conditions. Chapters 2 and 3 introduce the frame conventions and controller formulation required for these comparisons.

1.4 Scope and Contributions

Within this scope, the experiments evaluate how a commanded tilt representing the initial angular mismatch between the tool and the surface, the rotational and translational stiffness in the surface directions, and the position of a virtual centre of compliance influence contact-induced alignment. The controller is a phase-scheduled Cartesian impedance controller written in C++ using libfranka and Eigen. It runs in the 1 kHz torque-control loop of the Franka Emika Panda. The commanded Cartesian wrench is mapped to joint torques through $J^\top$, and the Coriolis torque is obtained from the robot model and added to the command. Each control phase uses its own Cartesian stiffness values, while the damping is calculated from an estimate of the Cartesian inertia.

A surface-fixed coordinate frame is constructed from the surface geometry. Its axes consist of the unit surface normal n_s and two mutually orthogonal unit vectors t_1 and t_2 lying in the surface plane. The directions t_1 and t_2 therefore represent the two independent tangential directions along the surface. This frame is used to define the translational and rotational stiffness values according to the contact geometry.

Each run follows a fixed sequence of control phases. First, the tool is brought to the commanded orientation and approaches the surface up to a specified geometric clearance. Based on the rectangular grinding-face geometry and the current tool orientation, the

controller predicts the point on the tool expected to reach the surface first. For this purpose, the four tool corners are compared along the approach direction. A single leading corner is selected for a general tilt. If two neighbouring corners are equally close to the surface, their midpoint is used as the predicted contact point at the centre of the leading edge. If all four corners are equally close to the surface, the centre of the grinding face is used. This predicted contact point is fixed when the clearance is reached and remains unchanged during set-up and grinding.

During contact set-up, the desired position of the predicted contact point is moved gradually from the clearance position toward the surface and slightly beyond the surface plane. The physical surface prevents the tool from reaching this virtual target, so the resulting position error generates the normal pressing force. Because the robot command is defined at the TCP, the controller calculates the corresponding TCP target using the fixed geometric offset between the TCP and the predicted contact point.

The desired tool orientation remains equal to the orientation reached at the clearance point, but it is not enforced rigidly. The finite rotational stiffness allows the actual tool orientation to change when the contact forces produce a moment, so the tool can rotate toward the surface during set-up.

During this phase, the 6×6 stiffness and damping matrices defined at the virtual centre of compliance are transformed to the TCP through an adjoint congruence transformation. The resulting off-diagonal blocks couple translation and rotation, allowing the commanded pressing motion to generate a corrective moment that helps the tool align with the plane surface during set-up. The effects of this translation-rotation coupling are investigated in this thesis. After contact set-up, the controller proceeds to the tangential grinding motion.

Accordingly, the contributions of this thesis are divided into implemented controller components and experimentally established findings. The implemented work includes a frame-consistent real-time Cartesian impedance controller for the 7-DOF Franka Emika Panda. It also includes a spatial compliance-centre formulation in which the 6×6 stiffness and damping matrices are defined at a selected virtual centre of compliance and transformed to the TCP. In addition, a contact and evaluation methodology was developed in which the physical surface orientation and the grinding-face axis are defined independently, allowing the alignment of the tool to be evaluated relative to the surface.

The experiments provide an axis-specific evaluation of how rotational and translational stiffness influence contact-induced alignment. Within the tested range, the position of the virtual centre of compliance produced the largest measured change in alignment. This effect depends on the lever direction: a lever that assists alignment about t_1 must be reversed to assist alignment about t_2 .

Projected joint damping and a two-point smallest-singular-value bias were also implemented for the redundant seventh joint. Their effects were evaluated in a separate free-space hold study using an internally generated joint-torque disturbance equivalent to a point force at the end-effector. The results describe the null-space response to this commanded disturbance.

1.5 Thesis Structure

The thesis is structured as follows. Chapter 2 introduces the impedance law, the frame conventions, and the compliance-centre shift. Chapter 3 describes their real-time implementation on the Franka Emika Panda. Chapter 4 presents the experimental methodology, while Chapter 5 reports and discusses the corresponding results. Chapter 6 summarises the main conclusions and outlines future work.

Chapter 2

Theoretical Background

PURPLE — revised and confirmed. The chapter was revised in full and every paragraph is settled. No further editing is to be applied to it, and any remaining concern is raised as a comment rather than as a change.

This chapter presents the mathematical basis of the Cartesian impedance controller. The required frame conventions and pose errors are introduced first. The subsequent sections derive the kinematic mappings, the impedance wrench, the compliance-centre transformation, and the null-space torque.

2.1 Reference Frames

The following right-handed coordinate frames are used in the theoretical formulation and the controller [19, 3].

- **Base frame $\{0\}$:** The base frame is fixed to the robot mounting surface. Its origin is the robot base reference point used by the kinematic model. For the upright Panda configuration used in this thesis, the z_0 -axis is normal to the mounting surface, while the x_0 - and y_0 -axes lie in the mounting plane. Unless stated otherwise, Cartesian positions, directions, velocities, forces, and moments are expressed in this frame.
- **Joint frames $\{1\}, \{2\}, \dots, \{n\}$:** These frames are attached to the successive links of the kinematic chain according to the Denavit–Hartenberg convention [4, 3]. Joint i rotates about z_{i-1} , and p_{i-1} denotes a point on this axis. The joint frames define the transformations between consecutive links and the indexing used in the forward

kinematics and geometric Jacobian.

- **End-effector frame $\{EE\}$:** The end-effector frame is attached to the TCP and moves with the tool. Its position p_{EE} and orientation R_{EE} , both expressed relative to the base frame, define the current Cartesian pose of the robot. The axes x_{EE} , y_{EE} , and z_{EE} describe the current tool orientation. The configured tool approach axis is defined in this frame and is transformed to the base frame using R_{EE} .
- **Desired end-effector frame $\{d\}$:** The desired frame represents the commanded TCP pose. Its origin is p_d , and its orientation is R_d , both expressed relative to the base frame. The difference between the desired and current end-effector frames provides the translational and rotational errors used by the impedance controller.
- **Surface task frame $\{S\}$:** The surface task frame is tied to the configured surface rather than to the moving tool. Its origin is placed at the configured surface point p_s , and its orientation is defined by the two surface tangents t_1 and t_2 and the surface normal n_s :

$$R_{\text{surface}} = \begin{bmatrix} t_1 & t_2 & n_s \end{bmatrix}. \quad (2.1)$$

The directions t_1 and t_2 lie in the surface plane and are mutually orthogonal, while n_s is normal to the surface. This frame is used to describe the commanded tilt and to assign stiffness and damping values according to the two tangential directions and the normal direction. For the contact task,

$$R_{\text{task}} = R_{\text{surface}}. \quad (2.2)$$

2.2 Rigid-Body Transformations

The positions and orientations of the frames introduced in Section 2.1 are related through rotation matrices and homogeneous transformations.

2.2.1 Rotation Matrices

An orientation is represented by a rotation matrix $R \in SO(3)$, satisfying

$$R^\top R = I, \quad \det R = 1, \quad R^{-1} = R^\top. \quad (2.3)$$

The notation bR_a denotes the orientation of frame $\{a\}$ expressed in frame $\{b\}$. Its columns contain the axes of frame $\{a\}$ expressed in frame $\{b\}$. A vector is transformed from frame $\{a\}$ to frame $\{b\}$ according to

$${}^bv = {}^bR_a {}^av. \quad (2.4)$$

The elementary right-handed rotations used later to define the commanded tool tilt are

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}, \quad R_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix}, \quad R_z(\gamma) = \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (2.5)$$

2.2.2 Homogeneous Transformations

A position is represented by a vector $p \in \mathbb{R}^3$. Rotation and translation are combined in a homogeneous transformation [3, 19]:

$${}^bT_a = \begin{bmatrix} {}^bR_a & {}^bp_a \\ 0^\top & 1 \end{bmatrix}, \quad (2.6)$$

where bp_a is the position of the origin of frame $\{a\}$ expressed in frame $\{b\}$. A point expressed in frame $\{a\}$ is transformed to frame $\{b\}$ according to

$${}^bp = {}^bR_a {}^ap + {}^bp_a. \quad (2.7)$$

Successive transformations are composed by matrix multiplication:

$${}^cT_a = {}^cT_b {}^bT_a. \quad (2.8)$$

2.2.3 End-Effector Pose

The controlled pose consists of the end-effector position

$$p_{EE} = \begin{bmatrix} p_{x,EE} & p_{y,EE} & p_{z,EE} \end{bmatrix}^\top \in \mathbb{R}^3 \quad (2.9)$$

and the orientation $R_{EE} \in SO(3)$. Both are expressed in the base frame and combined in the homogeneous transformation

$$T_{EE} = \begin{bmatrix} R_{EE} & p_{EE} \\ 0^\top & 1 \end{bmatrix}. \quad (2.10)$$

In the implementation, T_{EE} is obtained directly from the reported robot state, and the orientation is not converted to an Euler-angle representation.

2.2.4 Cartesian Pose Error

Let the desired end-effector pose be

$$T_d = \begin{bmatrix} R_d & p_d \\ 0^\top & 1 \end{bmatrix}, \quad (2.11)$$

where p_d and R_d are the desired TCP position and orientation, respectively. The current end-effector pose is denoted by T_{EE} .

The relative transformation from the current end-effector frame to the desired frame is

$$T_{err} = T_{EE}^{-1} T_d = \begin{bmatrix} R_{EE}^\top R_d & R_{EE}^\top (p_d - p_{EE}) \\ 0^\top & 1 \end{bmatrix}. \quad (2.12)$$

When the current and desired poses coincide, $T_{err} = I_4$. The rotational and translational blocks of eq. (2.12) therefore both represent zero error in this case.

Position error. The translational block of eq. (2.12) expresses the position error in the current end-effector frame. The Cartesian impedance law uses the equivalent restoring error expressed in the base frame:

$$e_p = p_d - p_{EE} \in \mathbb{R}^3. \quad (2.13)$$

A positive position error therefore points from the current TCP position toward the desired TCP position.

Orientation error. The rotational block of eq. (2.12) is

$$\Delta R = R_{EE}^\top R_d. \quad (2.14)$$

It represents the rotation required to carry the current end-effector orientation to the desired orientation. If both orientations coincide, then $\Delta R = I$.

For use in the impedance controller, ΔR is represented by a rotation angle ϕ and a unit rotation axis $u \in \mathbb{R}^3$ [19, 15]. The angle follows from

$$\text{tr}(\Delta R) = 1 + 2 \cos \phi, \quad (2.15)$$

and is therefore

$$\phi = \arccos\left(\frac{\text{tr}(\Delta R) - 1}{2}\right), \quad \phi \in [0, \pi]. \quad (2.16)$$

Rodrigues' formula expresses the relative rotation as

$$\Delta R = I + \sin \phi [u]_\times + (1 - \cos \phi) [u]_\times^2, \quad (2.17)$$

where

$$[u]_\times = \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix}, \quad [u]_\times^\top = -[u]_\times. \quad (2.18)$$

Subtracting the transpose of ΔR isolates its skew-symmetric part:

$$\frac{1}{2} (\Delta R - \Delta R^\top) = \sin \phi [u]_\times. \quad (2.19)$$

For $\sin \phi \neq 0$, the rotation axis is obtained from

$$u = \frac{1}{2 \sin \phi} \begin{bmatrix} \Delta R_{32} - \Delta R_{23} \\ \Delta R_{13} - \Delta R_{31} \\ \Delta R_{21} - \Delta R_{12} \end{bmatrix}. \quad (2.20)$$

Multiplying the unit axis by the angle gives the orientation-error vector in the current end-effector frame. It is then rotated into the base frame:

$$e_{R,EE} = \phi u, \quad e_R = R_{EE} e_{R,EE} \in \mathbb{R}^3. \quad (2.21)$$

The direction of e_R defines the axis of the required correction, while its magnitude is the angular error in radians. When the current and desired orientations coincide, $\phi = 0$ and $e_R = 0$.

The complete Cartesian error used by the impedance controller is

$$e = \begin{bmatrix} e_p \\ e_R \end{bmatrix} \in \mathbb{R}^6. \quad (2.22)$$

2.3 Kinematics of a 7-DOF Manipulator

Having defined the end-effector pose as the controlled variable, its relation to the joint configuration is introduced next.

2.3.1 Forward Kinematics

Forward kinematics maps the joint configuration

$$q = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix}^\top \in \mathbb{R}^n \quad (2.23)$$

to the end-effector pose. Using the joint frames introduced in Section 2.1, the transformation from the base frame $\{0\}$ to the final link frame $\{n\}$ is

$${}^0T_n(q) = {}^0T_1(q_1) {}^1T_2(q_2) \cdots {}^{n-1}T_n(q_n). \quad (2.24)$$

In the standard Denavit–Hartenberg convention [4, 3, 19], joint i rotates by q_i about the axis z_{i-1} . The transformation between two consecutive joint frames is

$${}^{i-1}T_i(q_i) = R_z(q_i) T_z(d_i) T_x(a_i) R_x(\alpha_i), \quad (2.25)$$

where a_i , d_i , and α_i describe the fixed geometry between the consecutive joint frames, while q_i is the revolute-joint coordinate.

The complete transformation gives the end-effector pose in the base frame:

$${}^0T_n(q) = T_{EE}(q) = \begin{bmatrix} R_{EE}(q) & p_{EE}(q) \\ 0^\top & 1 \end{bmatrix}. \quad (2.26)$$

This convention establishes the joint and frame indexing used in the subsequent Jacobian formulation. In the implemented controller, the forward-kinematic chain is not evaluated separately; the current end-effector pose is obtained from the libfranka model interface.

2.3.2 Differential Kinematics

Differential kinematics relates the joint velocities to the end-effector twist:

$$\dot{x} = \begin{bmatrix} \dot{p}_{EE} \\ \omega_{EE} \end{bmatrix} = J(q)\dot{q}, \quad (2.27)$$

where $\dot{p}_{EE} \in \mathbb{R}^3$ and $\omega_{EE} \in \mathbb{R}^3$ are the linear and angular velocities of the end-effector, respectively. The joint-velocity vector is

$$\dot{q} = \begin{bmatrix} \dot{q}_1 & \dot{q}_2 & \cdots & \dot{q}_n \end{bmatrix}^\top \in \mathbb{R}^n, \quad (2.28)$$

and $J(q) \in \mathbb{R}^{6 \times n}$ is the geometric Jacobian. For the 7-DOF Panda, $n = 7$. The end-effector twist and the Jacobian used in the controller are expressed in the base frame. This relation provides the Cartesian velocity required by the damping term of the impedance controller.

2.3.3 Geometric Jacobian

The geometric Jacobian is written column-wise as

$$J(q) = \begin{bmatrix} J_1 & J_2 & \cdots & J_n \end{bmatrix} \in \mathbb{R}^{6 \times n}, \quad (2.29)$$

where J_i describes the contribution of joint i to the end-effector twist. Since all joints of the Panda are revolute, each column is given by

$$J_i = \begin{bmatrix} z_{i-1} \times (p_{EE} - p_{i-1}) \\ z_{i-1} \end{bmatrix} \in \mathbb{R}^6, \quad (2.30)$$

where z_{i-1} is the axis of joint i , and p_{i-1} is a point on that axis. Both are expressed in the base frame. The upper block describes the linear velocity generated at the end-effector, while the lower block describes the corresponding angular velocity [19, 3].

Because the joint axes and their positions depend on the current configuration, the Jacobian also depends on q . In the implemented controller, $J(q)$ is obtained directly from the libfranka model interface.

The Jacobian describes the local relation between joint velocities and the end-effector twist. For a robot with n joints it is written column-wise,

$$\begin{bmatrix} \dot{p}_{EE} \\ \omega_{EE} \end{bmatrix} = J(q)\dot{q}, \quad J(q) = \begin{bmatrix} J_1 & J_2 & \cdots & J_n \end{bmatrix} \in \mathbb{R}^{6 \times n}, \quad (2.31)$$

where each column J_i is the contribution of joint i . The Panda has only revolute joints, so with z_{i-1} the unit vector along the axis of joint i and p_{i-1} a point on that axis, both expressed in the base frame, the column is

$$J_i = \begin{bmatrix} z_{i-1} \times (p_{EE} - p_{i-1}) \\ z_{i-1} \end{bmatrix} \in \mathbb{R}^6. \quad (2.32)$$

This standard form follows from the instantaneous velocity of a point under a revolute motion [19, 3]. The vector $p_{EE} - p_{i-1}$ is the lever arm from the joint axis to the end-effector, so the cross product $z_{i-1} \times (p_{EE} - p_{i-1})$ is the linear velocity that rotation of joint

i produces at the end-effector, while the lower block z_{i-1} is the corresponding angular velocity. Each column therefore stacks a linear contribution above an angular one.

Because z_{i-1} , p_{i-1} and p_{EE} all depend on the current joint configuration, the Jacobian is configuration dependent, and the influence of each joint on the end-effector motion changes as the robot moves.

2.4 Cartesian Impedance Control

The geometric Jacobian is also used through its transpose to map a Cartesian wrench to joint torques [12]. Before introducing this mapping, the desired Cartesian interaction behaviour is defined.

In this thesis, Cartesian impedance control is formulated as a spring–damper relation between the end-effector motion and the commanded wrench [10, 17, 14]. The Cartesian pose error e , defined in Section 2.2.4, produces a restoring wrench through the stiffness matrix K , while the Cartesian velocity produces an opposing wrench through the damping matrix D . Their entries determine the translational and rotational compliance in the selected task directions [18].

2.4.1 Impedance Wrench

The impedance wrench is evaluated in the robot base frame. The corresponding translational stiffness and damping matrices are $K_{p,0}, D_{p,0} \in \mathbb{R}^{3 \times 3}$, while the rotational matrices are $K_{R,0}, D_{R,0} \in \mathbb{R}^{3 \times 3}$. These matrices are not generally diagonal in the base frame because the direction-dependent gains are defined relative to the task frame. Their base-frame representation therefore changes with the task-frame orientation, even when the local gain values remain unchanged.

The commanded Cartesian force is given by the translational spring–damper law [10, 14]

$$f = K_{p,0}e_p + D_{p,0}(\dot{p}_d - \dot{p}_{EE}) \in \mathbb{R}^3, \quad (2.33)$$

where $\dot{p}_d$ and $\dot{p}_{EE}$ are the desired and measured linear velocities expressed in the base frame. For a fixed position setpoint, $\dot{p}_d = \mathbf{0}$, and the damping term becomes $-D_{p,0}\dot{p}_{EE}$.

The commanded Cartesian moment is

$$m = K_{R,0}e_R - D_{R,0}\omega_{EE} \in \mathbb{R}^3, \quad (2.34)$$

where ω_{EE} is the measured end-effector angular velocity expressed in the base frame. Since no angular reference velocity is commanded, the rotational damping acts directly against ω_{EE} .

The force and moment are combined into the Cartesian wrench

$$F = \begin{bmatrix} f \\ m \end{bmatrix} = \begin{bmatrix} K_{p,0}e_p + D_{p,0}(\dot{p}_d - \dot{p}_{EE}) \\ K_{R,0}e_R - D_{R,0}\omega_{EE} \end{bmatrix} \in \mathbb{R}^6. \quad (2.35)$$

The wrench is ordered as force followed by moment, and all its components are expressed in the base frame. It is mapped to joint torques through the Jacobian transpose in the following subsection.

2.4.2 Joint-Torque Control Law

By the principle of virtual work, the commanded Cartesian wrench is mapped to joint torques through the transpose of the geometric Jacobian [12]:

$$\tau_{\text{cart}} = J^\top(q)F, \quad (2.36)$$

where $\tau_{\text{cart}} \in \mathbb{R}^n$ is the Cartesian impedance torque contribution and $J^\top(q) \in \mathbb{R}^{n \times 6}$. Substituting eq. (2.35) gives

$$\tau_{\text{cart}} = J^\top(q) \begin{bmatrix} K_{p,0}e_p + D_{p,0}(\dot{p}_d - \dot{p}_{EE}) \\ K_{R,0}e_R - D_{R,0}\omega_{EE} \end{bmatrix}. \quad (2.37)$$

The gain matrices in this expression are the base-frame matrices obtained from the selected task-frame gains. A change in the surface-frame orientation therefore changes the Cartesian wrench and the resulting joint-torque contribution, even when the local gain values remain unchanged.

2.4.3 Coriolis and Gravity Compensation in libfranka

In a general model-compensated torque controller, the Cartesian impedance torque is combined with gravity and Coriolis/centrifugal compensation:

$$\tau_{\text{cmd}} = \tau_{\text{cart}} + \tau_g(q) + \tau_c(q, \dot{q}), \quad (2.38)$$

where $\tau_g(q)$ is the gravity torque and $\tau_c(q, \dot{q})$ is the Coriolis and centrifugal torque.

These terms are handled differently by the Franka Control Interface. The Coriolis vector provided by the libfranka model is added explicitly to the command because it is not compensated internally. A separate gravity term is not added. In external joint-torque mode, the robot compensates gravity and motor friction internally, and the commanded torque is interpreted as excluding gravity [7]. Adding $\tau_g(q)$ would therefore compensate gravity twice.

The implemented torque command is consequently

$$\tau_{\text{cmd,impl}} = \tau_{\text{cart}} + \tau_c(q, \dot{q}) = J^\top(q)F + \tau_c(q, \dot{q}), \quad (2.39)$$

which follows the official libfranka Cartesian impedance example [8]. The null-space torque introduced later is added to this expression to obtain the complete commanded torque in eq. (2.92).

2.5 Surface Task Frame

The contact experiment uses a task frame tied to the surface geometry. This frame is defined before the stiffness and damping matrices are introduced, so that the local gain entries can later be interpreted as normal and tangential values instead of as abstract base-frame directions.

Let $n_s \in \mathbb{R}^3$ with $\|n_s\| = 1$ be the unit surface normal, expressed in the robot base frame.

To complete the frame, choose an auxiliary direction $a_s \in \mathbb{R}^3$ that is not parallel to n_s .

This auxiliary direction is only used to define a repeatable tangent direction on the surface. The first tangent vector is obtained by projecting a_s onto the plane orthogonal to n_s ,

and the second tangent vector is then obtained by a cross product. With $\tilde{t}_1$ denoting the unnormalised projected tangent, the construction is a Gram–Schmidt orthonormalization step applied to the surface normal and the auxiliary direction [19, 3]:

$$\begin{aligned}\tilde{t}_1 &= a_s - n_s (n_s^\top a_s), \\ t_1 &= \frac{\tilde{t}_1}{\|\tilde{t}_1\|}, \\ t_2 &= n_s \times t_1.\end{aligned}\tag{2.40}$$

Here, t_1 and t_2 are unit tangent vectors lying in the surface plane. The construction is valid as long as $\|\tilde{t}_1\| \neq 0$, which means that the auxiliary direction a_s must not be parallel to the normal n_s . To match the implemented controller and its parameter order, the surface task frame collects the two tangents first and the normal last,

$$R_{\text{surface}} = \begin{bmatrix} t_1 & t_2 & n_s \end{bmatrix}.\tag{2.41}$$

The column order is fixed: every surface-frame three-vector in the implementation uses $[\text{tangent } 1, \text{tangent } 2, \text{normal}]^\top$. In the gain representation below, the contact task uses this frame by setting $R_{\text{task}} = R_{\text{surface}}$. This makes stiffness values such as K_{t_1} , K_{t_2} , and K_n local surface-frame quantities in that same order.

2.6 Stiffness and Damping Representation

Direction-dependent stiffness and damping gains are most conveniently defined in a task frame whose axes coincide with the desired principal compliance directions. The corresponding matrices are diagonal in that frame but are not generally diagonal when expressed in the robot base frame.

Let $R_{\text{task}} \in SO(3)$ denote the orientation of the task frame relative to the base frame. Its columns contain the task-frame axes expressed in the base frame. The translational and rotational gain matrices used in the impedance wrench are obtained from

$$\begin{aligned}K_{p,0} &= R_{\text{task}} K_{p,\text{task}} R_{\text{task}}^\top, & D_{p,0} &= R_{\text{task}} D_{p,\text{task}} R_{\text{task}}^\top, \\ K_{R,0} &= R_{\text{task}} K_{R,\text{task}} R_{\text{task}}^\top, & D_{R,0} &= R_{\text{task}} D_{R,\text{task}} R_{\text{task}}^\top.\end{aligned}\tag{2.42}$$

For an impedance without translation-rotation coupling in the task frame, the full stiffness and damping matrices are

$$K_{\text{task}} = \begin{bmatrix} K_{p,\text{task}} & 0 \\ 0 & K_{R,\text{task}} \end{bmatrix}, \quad D_{\text{task}} = \begin{bmatrix} D_{p,\text{task}} & 0 \\ 0 & D_{R,\text{task}} \end{bmatrix}. \quad (2.43)$$

Defining

$$T_R = \text{diag}(R_{\text{task}}, R_{\text{task}}), \quad (2.44)$$

their base-frame representations are

$$K_0 = T_R K_{\text{task}} T_R^\top, \quad D_0 = T_R D_{\text{task}} T_R^\top. \quad (2.45)$$

For the contact task, the task frame is the surface frame, whose orientation is

$$R_{\text{task}} = R_{\text{surface}} = \begin{bmatrix} t_1 & t_2 & n_s \end{bmatrix}. \quad (2.46)$$

For example, the local translational stiffness is defined as

$$K_{p,\text{task}} = \text{diag}(K_{t_1}, K_{t_2}, K_n), \quad (2.47)$$

and its base-frame representation is

$$K_{p,0} = R_{\text{surface}} \text{diag}(K_{t_1}, K_{t_2}, K_n) R_{\text{surface}}^\top. \quad (2.48)$$

The rotational stiffness follows the same axis order,

$$K_{R,\text{task}} = \text{diag}(K_{r,t_1}, K_{r,t_2}, K_{r,n}), \quad (2.49)$$

and the damping matrices are constructed analogously. A change in the configured surface orientation therefore changes the corresponding base-frame gain matrices even when the local gain values remain unchanged.

During the approach phase, this surface-frame construction is applied to both the transla-

tional and rotational gain blocks. During contact set-up and grinding, only the rotational stiffness and damping are constructed in the surface frame; the translational matrices remain diagonal in the base-frame directions $[x, y, z]$, as described in Chapter 3. The active gain matrices are based on the configured surface frame and are not rotated with the current end-effector orientation R_{EE} .

2.7 Centre of Compliance

The task-frame transformation of eq. (2.45) rotates the principal stiffness directions while leaving the impedance reference point at the end-effector origin, identified here with the tool centre point (TCP). Independently of this orientation choice, the virtual spring can be centred at a different point. Shifting the impedance reference from the TCP to a centre of compliance p_c transforms a block-diagonal impedance at p_c into a coupled impedance at the TCP [15, 6, 20, 13]. This construction is applied during contact set-up to generate translation-rotation coupling that supports tool alignment.

The lever from the centre of compliance to the TCP, expressed in the base frame, is

$$r_c = p_{TCP} - p_c \in \mathbb{R}^3, \quad p_{TCP} = p_{EE}. \quad (2.50)$$

Let $[r_c]_{\times}$ denote the skew-symmetric matrix satisfying $[r_c]_{\times}a = r_c \times a$. Under the small-angle approximation, the position error at the centre of compliance is

$$e_{p,c} = e_p + [r_c]_{\times}e_R. \quad (2.51)$$

The positive sign follows from the selected lever convention. The vector from the TCP to the centre of compliance is $-r_c$, and therefore

$$e_R \times (-r_c) = [r_c]_{\times}e_R.$$

The translational and rotational errors at the centre of compliance can thus be written as

$$e_c = \begin{bmatrix} e_{p,c} \\ e_R \end{bmatrix} = \text{Ad}(r_c) \begin{bmatrix} e_p \\ e_R \end{bmatrix}, \quad \text{Ad}(r_c) = \begin{bmatrix} \mathbf{I}_3 & [r_c]_\times \\ 0 & \mathbf{I}_3 \end{bmatrix}. \quad (2.52)$$

The point-shift adjoint $\text{Ad}(r_c)$ changes the reference point of the impedance. By contrast, $T_R = \text{diag}(R_{\text{task}}, R_{\text{task}})$ in eq. (2.45) changes its principal directions.

After any required task-frame rotation, let the decoupled stiffness and damping matrices defined at the centre of compliance be

$$K_c = \begin{bmatrix} K_p & 0 \\ 0 & K_R \end{bmatrix}, \quad D_c = \begin{bmatrix} D_p & 0 \\ 0 & D_R \end{bmatrix}. \quad (2.53)$$

All blocks in eq. (2.53) are expressed in the base-frame orientation. The wrench at the centre of compliance is

$$w_c = K_c e_c.$$

Transforming this power-conjugate wrench to the TCP gives

$$w_{\text{TCP}} = \text{Ad}(r_c)^\top w_c.$$

The resulting TCP stiffness is therefore

$$K_{\text{TCP}} = \text{Ad}(r_c)^\top K_c \text{Ad}(r_c) = \begin{bmatrix} K_p & K_p[r_c]_\times \\ [r_c]_\times^\top K_p & K_R + [r_c]_\times^\top K_p[r_c]_\times \end{bmatrix}. \quad (2.54)$$

The same point shift is applied to the damping matrix:

$$D_{\text{TCP}} = \text{Ad}(r_c)^\top D_c \text{Ad}(r_c) = \begin{bmatrix} D_p & D_p[r_c]_\times \\ [r_c]_\times^\top D_p & D_R + [r_c]_\times^\top D_p[r_c]_\times \end{bmatrix}. \quad (2.55)$$

The off-diagonal blocks introduce translation–rotation coupling. A translational error can therefore generate a restoring moment, while a rotational error can generate a restoring force. This coupling is absent from the block-diagonal impedance defined directly at the TCP.

The sign of the translation-to-moment coupling can be checked by setting $e_R = 0$. With

$$f = K_p e_p,$$

the corresponding elastic moment at the TCP is

$$m_c = [r_c]_{\times}^{\top} f = -r_c \times f. \quad (2.56)$$

Reversing the direction of r_c therefore reverses the generated coupling moment. This relation provides a direct sign check for the selected centre of compliance and pressing direction.

The coupling vanishes for $r_c = 0$. Conversely, if K_p is nonsingular, zero coupling implies $r_c = 0$. Since $\text{Ad}(r_c)$ is invertible, the congruence transformation also preserves definiteness:

$$K_c \succeq 0 \implies K_{\text{TCP}} \succeq 0, \quad K_c \succ 0 \implies K_{\text{TCP}} \succ 0. \quad (2.57)$$

The same statements apply to D_c and D_{TCP} . Negative off-diagonal entries therefore represent coupling terms rather than negative stiffness directions.

This formulation is a local small-motion model in which r_c is held fixed while the wrench is evaluated. In the implemented set-up mode, the selected lever remains constant, and the mapped gain matrices are therefore constant during the corresponding motion segment. If p_c or r_c were updated continuously, the gain matrices would become configuration dependent.

The selected point p_c is consequently a local linearised centre of compliance rather than a measured physical pivot. It need not coincide with the physical contact point p_C . The observed motion also depends on the finite rotational stiffness, the lever direction, the environmental constraint, friction, and the additional active controller terms introduced later.

2.8 Null-Space Torque

The Franka Emika Panda has seven joints, while the controlled end-effector task contains six components: three translational and three rotational. The robot is therefore kinematically redundant. The Cartesian impedance behaviour at the end-effector is the primary objective, and the remaining joint-space freedom can be used for a secondary objective. In this thesis, the secondary joint-space action consists of a null-space damping contribution and an active singular-value conditioning contribution. Both are projected using the joint-torque null-space projector N_7 . The required null-space direction, pseudoinverse, and projector are introduced before the two torque contributions are defined.

2.8.1 SVD-Based Null-Space Characterisation

The geometric Jacobian relates the current joint velocity $\dot{q}$ to the end-effector velocity $\dot{x}$:

$$\dot{x} = J(q)\dot{q}, \quad J(q) \in \mathbb{R}^{6 \times 7}, \quad (2.58)$$

where

$$\dot{x} = \begin{bmatrix} \dot{p}_{\text{EE}} \\ \omega_{\text{EE}} \end{bmatrix} \in \mathbb{R}^6 \quad (2.59)$$

contains the three linear and three angular velocity components of the end-effector.

A joint velocity $\dot{q}$ belongs to the kinematic null space when

$$J(q)\dot{q} = 0. \quad (2.60)$$

Such a joint velocity changes the robot configuration while preserving the end-effector position and orientation to first order. The Jacobian is the first-order term in the expansion of the forward kinematics about the current configuration, so a finite displacement along the null space leaves a pose error of second order in its magnitude. When $J(q)$ has full row rank, the null space of the 6×7 Jacobian is one-dimensional.

The singular value decomposition (SVD) is applied to the Jacobian at the current joint

configuration q [2, 5]:

$$J(q) = U_J \Sigma_J V_J^\top, \quad U_J \in \mathbb{R}^{6 \times 6}, \quad \Sigma_J \in \mathbb{R}^{6 \times 7}, \quad V_J \in \mathbb{R}^{7 \times 7}. \quad (2.61)$$

The Jacobian $J(q)$ is the input to the decomposition. The SVD returns the matrices U_J , Σ_J , and V_J , with the singular values contained in Σ_J .

The columns of V_J form an orthonormal basis of the seven-dimensional joint-velocity space. The joint velocity $\dot{q}$ can therefore be expressed as

$$\dot{q} = V_J a = \sum_{i=1}^7 a_i v_i, \quad (2.62)$$

where v_i is the i -th column of V_J , and $a \in \mathbb{R}^7$ contains the components of $\dot{q}$ along these basis directions.

Similarly, the columns of U_J form an orthonormal basis of the six-dimensional end-effector-velocity space. Substituting $\dot{q} = V_J a$ into eq. (2.58) gives

$$\dot{x} = J(q) V_J a = U_J \Sigma_J a. \quad (2.63)$$

Thus, V_J provides the basis directions in which the joint velocity $\dot{q}$ is expressed, while U_J provides the corresponding basis directions in which the end-effector velocity $\dot{x}$ is expressed. The matrix Σ_J scales the mapping between these directions.

For the 6×7 Jacobian, the singular-value matrix has the form

$$\Sigma_J = \begin{bmatrix} \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_6) & 0_{6 \times 1} \end{bmatrix}, \quad (2.64)$$

where

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_6 \geq 0. \quad (2.65)$$

For the first six basis directions,

$$J v_i = \sigma_i u_i, \quad i = 1, \dots, 6, \quad (2.66)$$

where u_i is the i -th column of U_J . A joint velocity in the direction v_i ,

$$\dot{q} = a_i v_i, \quad (2.67)$$

therefore produces the end-effector velocity

$$\dot{x} = a_i \sigma_i u_i. \quad (2.68)$$

The singular value σ_i scales the mapping from the joint-velocity basis direction v_i to the corresponding end-effector-velocity basis direction u_i .

The seventh column of V_J corresponds to the zero column of Σ_J and therefore satisfies

$$Jv_7 = 0. \quad (2.69)$$

A joint velocity of the form

$$\dot{q} = a_7 v_7 \quad (2.70)$$

therefore changes the joint configuration without producing an end-effector velocity. When the Jacobian has full row rank, v_7 spans its one-dimensional kinematic null space.

The smallest of the six task-producing singular values is

$$\sigma_{\min} = \sigma_6. \quad (2.71)$$

The quantities v_7 and σ_6 have different purposes. The vector v_7 represents the redundant joint-motion direction, whereas σ_6 describes the weakest of the six mappings from joint velocity to end-effector velocity. As σ_6 approaches zero, one end-effector velocity direction becomes increasingly difficult to produce, indicating an approach to a loss of Jacobian row rank. In this thesis, $\sigma_{\min}$ is used to compare the conditioning of nearby joint configurations and to select the direction of the conditioning torque.

2.8.2 SVD-Based Pseudoinverse

The Jacobian pseudoinverse maps end-effector-velocity components back to the corresponding joint-velocity directions. Its calculation uses the singular directions obtained from the SVD.

Small singular values lead to large reciprocal values and can amplify numerical errors. A relative singular-value threshold is therefore applied. Let $\rho \geq 0$ denote the configured relative SVD tolerance. The corresponding absolute threshold is calculated from the largest singular value σ_1 :

$$\sigma_{\text{thr}} = \rho \sigma_1. \quad (2.72)$$

A singular direction is retained when

$$\sigma_i > \sigma_{\text{thr}}. \quad (2.73)$$

The pseudoinverse is defined as

$$J^+ = \sum_{\substack{i=1 \\ \sigma_i > \sigma_{\text{thr}}}}^6 \frac{1}{\sigma_i} v_i u_i^\top \in \mathbb{R}^{7 \times 6}. \quad (2.74)$$

For each retained singular direction, J^+ maps an end-effector-velocity component along u_i to the corresponding joint-velocity direction v_i and scales it by $1/\sigma_i$. Directions at or below σ_{thr} receive a reciprocal value of zero.

2.8.3 Null-Space Projectors

The pseudoinverse is used to separate a joint-space vector into a task-related component and a null-space component. For an arbitrary joint velocity $\dot{q}$, this decomposition is

$$\dot{q} = \underbrace{J^+ J \dot{q}}_{\text{task-related component}} + \underbrace{(I_7 - J^+ J) \dot{q}}_{\text{null-space component}}. \quad (2.75)$$

The product $J^+ J$ projects $\dot{q}$ onto the joint-space directions that contribute to the end-effector velocity. Since J includes all three linear and all three angular velocity components,

this task-related component accounts for the complete controlled end-effector pose at the velocity level.

The complementary joint-velocity projector is

$$N_q = I_7 - J^+ J \in \mathbb{R}^{7 \times 7}. \quad (2.76)$$

Applying N_q to an arbitrary joint velocity retains its null-space component:

$$\dot{q}_{\text{null}} = N_q \dot{q}. \quad (2.77)$$

When J has full row rank and all six task-producing singular directions are retained,

$$J \dot{q}_{\text{null}} = J N_q \dot{q} = 0. \quad (2.78)$$

The projected joint velocity can therefore change the robot configuration while preserving the end-effector position and orientation to first order.

A corresponding decomposition is used for a secondary joint torque τ_0 . The joint-torque projector is the transpose of the velocity projector:

$$N_\tau = N_q^\top = I_7 - J^\top J^{+\top} \in \mathbb{R}^{7 \times 7}. \quad (2.79)$$

The secondary torque can then be written as

$$\tau_0 = \underbrace{J^\top J^{+\top} \tau_0}_{\text{task-related component}} + \underbrace{N_\tau \tau_0}_{\text{null-space component}}. \quad (2.80)$$

The first component lies in the joint-torque subspace associated with Cartesian wrenches through $J^\top$. The second component is retained as the secondary null-space torque:

$$\tau_{\text{null}} = N_\tau \tau_0. \quad (2.81)$$

The transpose relation follows from the mechanical power $\tau^\top \dot{q}$ between joint torques and joint velocities. Thus, N_q is applied to joint velocities, while N_τ is applied to secondary joint torques.

For the SVD-based pseudoinverse, J^+J is an orthogonal and symmetric projector. Hence,

$$N_q = N_\tau \quad (2.82)$$

holds analytically. The separate symbols distinguish the quantity on which the projector acts. In the implementation, N_τ is additionally symmetrised to reduce floating-point asymmetry.

When J has full row rank and all six task-producing singular directions are retained,

$$N_q = N_\tau = v_7 v_7^\top. \quad (2.83)$$

In this case, projection retains only the component along the redundant direction v_7 . When a singular value lies at or below the selected threshold, its associated right-singular direction is also assigned to the numerical null space.

2.8.4 Null-Space Damping Torque

The current joint velocity is separated into task-related and null-space components according to eq. (2.75). The damping contribution acts on the null-space component $N_q \dot{q}$:

$$\tau_d = -d_{\text{null}} N_\tau \dot{q} = -d_{\text{null}} N_q \dot{q}, \quad (2.84)$$

where $d_{\text{null}} \geq 0$ is the null-space damping coefficient. The negative sign makes the torque oppose motion along the redundant joint-space direction and dissipate the associated motion.

2.8.5 Singular-Value Conditioning Torque

The second contribution applies an active torque along the redundant direction v_7 . Its sign is selected by comparing $\sigma_{\min}$ at two nearby joint configurations.

Starting from the current joint configuration q , the sampled configurations are

$$q_+ = q + \alpha_{\text{probe}} v_7, \quad q_- = q - \alpha_{\text{probe}} v_7. \quad (2.85)$$

The Jacobian is recalculated at both configurations. The corresponding smallest singular values are

$$\sigma_+ = \sigma_{\min}(J(q_+)) , \quad \sigma_- = \sigma_{\min}(J(q_-)) . \quad (2.86)$$

Their difference is

$$\Delta_\sigma = \sigma_+ - \sigma_- . \quad (2.87)$$

The direction selector is defined as

$$s = \begin{cases} +1, & \Delta_\sigma > \delta_\sigma, \\ -1, & \Delta_\sigma < -\delta_\sigma, \\ 0, & |\Delta_\sigma| \leq \delta_\sigma, \end{cases} \quad (2.88)$$

where $\delta_\sigma \geq 0$ is the comparison deadband. Within this deadband, the selector is set to zero and the conditioning contribution is suppressed.

The selected direction is invariant to the sign convention of v_7 returned by the SVD. Reversing v_7 exchanges q_+ and q_- and reverses s , so the product sv_7 remains unchanged.

The probe angle α_{probe} defines the joint-space distance between the current configuration and the two sampled configurations. The magnitude of the resulting torque is specified independently by k_σ .

The vector v_7 represents the instantaneous null-space direction at the current configuration. For a finite probe angle, q_+ and q_- provide local approximations of the nonlinear constant-pose self-motion. The corresponding preservation of the end-effector pose is therefore a first-order property of the sampled direction.

The selector uses the sign of Δ_σ to choose the sampled direction with the larger value of $\sigma_{\min}$. This produces a two-point directional conditioning strategy based on local comparison rather than on the magnitude of the sampled variation.

The vector sv_7 defines the selected secondary joint-space direction. Projecting it with N_τ retains its component in the numerical joint-torque null space:

$$\tau_\sigma = k_\sigma N_\tau (sv_7), \quad (2.89)$$

where $k_\sigma \geq 0$ specifies the commanded torque magnitude.

When the Jacobian has full row rank and all six task-producing singular directions are retained, v_7 lies in the one-dimensional null space and

$$N_\tau v_7 = v_7. \quad (2.90)$$

The conditioning contribution therefore biases the robot along the selected redundant joint direction, while the damping contribution opposes motion within the projected subspace.

2.8.6 Complete Null-Space Torque

Combining the damping and conditioning contributions gives

$$\tau_{\text{null}} = \tau_d + \tau_\sigma = -d_{\text{null}} N_\tau \dot{q} + k_\sigma N_\tau (sv_7) \in \mathbb{R}^7. \quad (2.91)$$

When the Jacobian has full row rank, its null space is one-dimensional and is spanned by v_7 . If the Jacobian loses row rank, the null space becomes multidimensional. In that case, the implemented conditioning term continues to probe the directions $+v_7$ and $-v_7$. A more general treatment would require consideration of the additional right singular vectors associated with the enlarged null space. This extension is outside the scope of the present work. The damping contribution, by contrast, acts on the joint-motion directions contained in the numerical null space through $N_\tau \dot{q}$.

The activation of the damping and conditioning contributions is a controller configuration choice. The available combinations and the configuration used in the reported experiments are described in section 3.4.

2.9 Complete Joint-Torque Command

The complete joint-torque command combines the Cartesian impedance torque, the null-space torque, and the Coriolis compensation:

$$\tau_{\text{cmd}} = \tau_{\text{cart}} + \tau_{\text{null}} + \tau_c(q, \dot{q}) = J^\top(q)F + \tau_{\text{null}} + \tau_c(q, \dot{q}). \quad (2.92)$$

Here, $\tau_{\text{cart}} = J^\top(q)F$ is the Cartesian impedance torque, τ_{null} is the secondary null-space torque, and $\tau_c(q, \dot{q})$ is the Coriolis and centrifugal compensation obtained from the robot model and added in the control callback. A separate gravity term is not added because gravity compensation is handled internally by the Franka Control Interface, as described in Section 2.4.3.

Chapter 3

Controller Design and Implementation

PURPLE — revised and confirmed. The chapter was revised in full and every paragraph is settled. No further editing is to be applied to it, and any remaining concern is raised as a comment rather than as a change.

This chapter describes the real-time implementation of the Cartesian impedance controller developed in Chapter 2. The controller is written in C++ using libfranka and Eigen and is executed through the Franka Control Interface (FCI) with a nominal control period of 1 ms.

The software combines the Cartesian impedance law, the SVD-based null-space controller, the configured surface and tool geometry, phase-dependent stiffness and damping, and the virtual centre of compliance. These components are coordinated by a phase state machine that orients the tool, approaches the surface, applies the set-up preload, and executes the grinding motion. Additional operating modes provide Cartesian pose holding, set-up-impedance holding, and manual guidance.

The following sections describe the software architecture, the real-time control loop, the implemented Cartesian and null-space torque contributions, the geometric representation of the surface and tool, the phase-dependent impedance parameters, and the execution of the control sequence.

3.1 Software Architecture and Program Execution

The controller is divided into modules for configuration handling, control calculations, real-time execution, operator interaction, and run evaluation. The main program coordinates these modules and manages the execution of successive controller sessions.

Table 3.1: Software modules of the surface-grinding controller.

Module	Responsibility
<code>main.cpp</code>	Connects to the robot, prepares the controller session, starts the selected operating mode, and initiates post-run reporting.
<code>include/</code>	Defines the controller configuration, mathematical types, run-time data structures, and interfaces shared between the software modules.
<code>src/config/</code>	Reads the parameter files and assigns their values to the typed controller configuration.
<code>src/control/</code>	Implements the Cartesian impedance law, null-space torque, phase-dependent gain construction, surface and tool geometry, inertia-scaled damping, and robot support functions.
<code>src/runtime/</code>	Executes the phase state machine and the 1 kHz joint-torque control callback.
<code>src/interface/</code>	Provides the start-up menu, gripper and tool-handling actions, manual guidance, live parameter commands, and terminal output.
<code>src/report/</code>	Stores and writes the recorded controller signals and evaluates the completed run.
<code>tools/</code>	Provides a separate calibration utility for estimating one point and the orientation of the physical surface plane.
<code>measure_plane.cpp</code>	

The parameter directory is resolved relative to the controller executable. The complete configuration is distributed over 16 files in the `params/` directory. These files contain the robot connection and logging settings, collision thresholds, gripper parameters, initial

posture, surface and tool geometry, phase settings, Cartesian gains, inertia-scaled-damping settings, virtual-compliance-centre parameters, null-space parameters, disturbance-test settings, and manual-guidance settings.

At program start, the parameter files are read to obtain the robot address and initial controller configuration. The robot connection is then established, automatic error recovery is requested, the selected collision behaviour is assigned, and the libfranka robot model is loaded. A separate keyboard thread is also started to receive operator requests during the controller session. These requests are transferred through atomic variables shared with the control loop.

Before every new session, all parameter files are read again and the resulting values are stored in a new `ControllerConfig` object. This permits controller and experiment parameters to be changed between runs without recompiling the program. The loader requires all 16 parameter files and rejects duplicate parameter names across them.

The start-up interface provides four controller modes:

- the complete tool-orientation, surface-approach, set-up, and grinding sequence;
- Cartesian pose holding with the dedicated hold impedance;
- Cartesian pose holding with the set-up impedance; and
- manual guidance from the current robot pose.

The same interface provides the robot-preparation and gripper actions required before torque control. The robot can be moved to the configured initial joint posture, the gripper can be opened or closed, and the tool can be collected from or returned to its configured holder posture. When manual guidance is selected, the subsequent sequence or hold mode begins from the pose captured after hand-guided positioning.

Once the operating mode and initial state have been selected, the phase-dependent stiffness and damping matrices are constructed from the active configuration. These matrices, together with the robot and model interfaces, are passed to the real-time control loop.

Experimental signals are stored during control in a bounded ring buffer whose memory is allocated before the torque callback begins. After the controller session has ended, the stored samples are arranged in chronological order, written to the configured CSV file, and used to generate the terminal run summary. A new session can then be started from

the start-up menu with a freshly loaded configuration.

3.2 Real-Time Control Loop

The function `runControlLoop` prepares one controller session and executes the libfranka joint-torque callback. It receives the active controller configuration, the robot and model interfaces, the phase-dependent impedance gains, and the operator-input signals.

Before torque control begins, the current robot state is read once. The reported end-effector transformation provides the initial TCP position p_{start} and orientation R_{start} , while the measured joint configuration provides the initial posture q_{start} . These values initialise the desired pose of the selected sequence or hold mode.

The initial state also provides force and moment bias values for the subsequent contact evaluation. In addition, the controller initialises the configured surface point, the surface frame, the target tool orientation, the descent direction, and the four tool-face corners. The initial phase, phase clocks, operator gates, set-up references, damping cache, and recording buffer are prepared before entering the real-time callback.

The controller is passed to `robot.control` as a joint-torque callback. For every control cycle, the callback receives the current `RobotState` and the elapsed cycle duration and returns the seven commanded joint torques, as shown in listing 3.1.

Listing 3.1: Registration of the joint-torque callback through libfranka.

```

1 | robot.control(
2 |   [&](const franka::RobotState& state,
3 |     franka::Duration period) -> franka::Torques {
4 |
5 |     // Real-time state and torque calculations
6 |
7 |     return franka::Torques(tau_array);
8 |   });

```

During each callback, the elapsed controller time is updated from the period reported by libfranka. The joint state, end-effector Jacobian, Cartesian velocity, end-effector pose, and model-estimated external wrench are then obtained as shown in listing 3.2.

Listing 3.2: Acquisition of the robot-state and model quantities at the beginning of one real-time torque callback.

```

1 | time += period.toSec();
2 |
3 | Map<const Vec7> dq(state.dq.data());
4 | Map<const Vec7> q_current(state.q.data());

```

```

5 |
6 | std::array<double, 42> jacobian_array =
7 |     model.zeroJacobian(Frame::kEndEffector, state);
8 | Map<const Mat6x7> J(jacobian_array.data());
9 |
10 | const Vec6 xdot = J * dq;
11 | const Vec3 pdot = xdot.head<3>();
12 | const Vec3 omega = xdot.tail<3>();
13 |
14 | Map<const Mat4x4> T_EE(state.0_T_EE.data());
15 | const Vec3 p_EE = T_EE.block<3, 1>(0, 3);
16 | const Mat3 R_EE = T_EE.block<3, 3>(0, 0);
17 |
18 | Map<const Vec6> external_wrench(
19 |     state.0_F_ext_hat_K.data());
20 | const Vec3 external_force = external_wrench.head<3>();
21 | const Vec3 external_moment = external_wrench.tail<3>();

```

The vectors `q_current` and `dq` contain the measured joint configuration and velocity. The call to `model.zeroJacobian` returns the 6×7 geometric Jacobian of the configured libfranka end-effector frame. Multiplication by the measured joint velocity provides the TCP linear velocity `pdot` and angular velocity `omega`, both expressed in the robot base frame.

The field `state.0_T_EE` provides the current end-effector transformation in the base frame. Its translational and rotational blocks are assigned to `p_EE` and `R_EE`, respectively. The controller therefore uses the pose reported by libfranka rather than reconstructing it from a separate forward-kinematic model.

The field `state.0_F_ext_hat_K` provides the model-estimated external force and moment. During surface approach, the transition to set-up is determined geometrically from the tool-face clearance. At this transition, the current force and moment estimates are stored as bias values. During set-up, the bias-subtracted external moment is used for the moment-based exit condition. The force and moment estimates are also included in the terminal output, set-up evaluation, and recorded experimental data.

Operator requests are exchanged with the callback through atomic variables. They support transitions between the phase sequence, Cartesian pose hold, set-up-impedance hold, and manual guidance. The same interface receives the gate confirmations and stop request. During the dedicated hold modes, selected null-space and set-up-impedance parameters can also be adjusted. An accepted set-up-impedance change rebuilds the corresponding phase gains and damping cache.

The phase state machine updates the desired TCP position p_d , desired linear velocity $\dot{p}_d$, and desired orientation R_d . Their values depend on the active phase:

- tool orientation maintains the initial TCP position while advancing the orientation command toward the configured target;
- surface approach moves the desired position along the descent direction;
- set-up advances the selected tool feature into the surface;
- grinding retains the achieved preload and can add a tangential trajectory; and
- pose hold regulates the captured Cartesian pose.

After the reference has been updated, the callback calculates the Cartesian position and orientation errors using the conventions introduced in Section 2.2.4. The phase-dependent stiffness and damping matrices are then selected. When a phase gate is active, the dedicated gate holding gains replace the normal phase gains.

The displacement and velocity-error vectors supplied to the impedance calculation are

$$\Delta x = \begin{bmatrix} e_p \\ e_R \end{bmatrix}, \quad \Delta v = \begin{bmatrix} \dot{p}_d - \dot{p}_{EE} \\ -\omega_{EE} \end{bmatrix}. \quad (3.1)$$

The Cartesian impedance calculation returns the commanded wrench

$$F = \begin{bmatrix} f \\ m \end{bmatrix} \in \mathbb{R}^6. \quad (3.2)$$

Depending on the active mode, this wrench is obtained from either the decoupled impedance law of eq. (2.37) or the point-shifted set-up impedance described in Section 3.7.

The Cartesian torque contribution is

$$\tau_{\text{cart}} = J^\top(q)F. \quad (3.3)$$

The SVD-based null-space routine calculates the secondary torque τ_{null} according to the selected null-space mode. During the automatic disturbance experiment in Cartesian pose hold, an additional joint-torque contribution τ_{dist} is generated. Its value is zero during the remaining controller operation.

The Coriolis and centrifugal torque is obtained from the libfranka model. The complete torque command of an impedance-controlled cycle is therefore

$$\tau_{\text{cmd}} = \tau_{\text{cart}} + \tau_{\text{null}} + \tau_{\text{dist}} + \tau_c(q, \dot{q}). \quad (3.4)$$

Manual guidance follows a separate torque command:

$$\tau_{\text{guide}} = \tau_c(q, \dot{q}) - d_{\text{guide}}\dot{q}, \quad (3.5)$$

where d_{guide} is the configured joint-damping coefficient. After the guided pose is captured, its position, orientation, joint configuration, and wrench biases initialise the resumed sequence or hold mode.

For impedance-controlled cycles, the selected state, reference, error, wrench, phase, null-space, disturbance, and joint-torque quantities are sampled at the configured interval and copied into a preallocated ring buffer. When the buffer capacity is reached, the most recent samples replace the oldest entries.

The callback ends when the configured run duration expires, an operator stop is requested, the maximum approach distance is reached, or libfranka terminates the motion. After torque control has ended, the ring-buffer entries are placed in chronological order and returned together with the final controller state. File writing and post-run evaluation are performed outside the real-time callback.

3.3 Implemented Cartesian Impedance Controller

The mathematical formulation of the Cartesian impedance controller is given in Section 2.4. This section describes how the pose errors, phase-dependent gains, and Cartesian wrench are evaluated in the software.

The callback first selects the desired orientation associated with the active phase. During tool orientation, the desired orientation follows the interpolated orientation command. Surface approach uses the final tool-target orientation. At the transition to set-up, the current orientation is captured and retained throughout set-up and grinding. Cartesian pose hold uses the pose stored for the active hold mode.

The position and orientation errors are evaluated as shown in listing 3.3.

Listing 3.3: Calculation of the phase-dependent Cartesian pose error.

```

1 | const Vec3 e_p = desired.p_d - p_EE;
2 |
3 | const Mat3& R_d_used =
4 |     after_contact
5 |     ? R_contact_start
6 |     : (phase == ControlPhase::kPoseHold)
7 |       ? R_d
8 |       : (phase == ControlPhase::kToolOrientation)
9 |         ? R_orient_command
10 |         : R_base_tool_target;
11 |
12 | const Vec3 e_R = applyRotationalAxisMask(
13 |     params,
14 |     orientationError(R_EE, R_d_used),
15 |     R_base_surface);

```

The function `orientationError` forms the current-to-desired relative rotation and converts it to an axis-angle vector. The resulting local rotation vector is transformed to the robot base frame:

Listing 3.4: Base-frame orientation-error calculation.

```

1 | Mat3 R_error = R_current.transpose() * R_desired;
2 | Eigen::AngleAxisd angle_axis(R_error);
3 |
4 | return R_current
5 |     * angle_axis.axis()
6 |     * angle_axis.angle();

```

A rotation angle below the numerical tolerance returns a zero vector. The orientation-error convention therefore corresponds to the corrective base-frame error e_R defined in Section 2.2.4.

The rotational mask is applied after the error has been transformed to the base frame. The function first expresses e_R in the surface frame, selects the enabled components about t_1 , t_2 , and n_s , and transforms the result back to the base frame. The baseline configuration enables all three components. The mask acts on the rotational spring error, while rotational damping continues to use the measured angular velocity and the selected damping matrix.

The callback then combines the translational and rotational quantities into the six-dimensional displacement and velocity-error vectors used by the impedance routine. The desired angular velocity is zero in the implemented control sequence.

Listing 3.5: Assembly and evaluation of the Cartesian impedance command.

```

1 | Vec6 dx;
2 | dx.head<3>() = e_p;
3 | dx.tail<3>() = e_R;
4 |
5 | Vec6 dv;
6 | dv.head<3>() = desired.pdot_d - pdot;
7 | dv.tail<3>() = -omega;

```

```

8
9  const Vec6 wrench =
10      computeCartesianImpedanceWrench(
11          params, phase,
12          Kp_used, Dp_used,
13          KR_used, DR_used,
14          R_base_surface,
15          dx, dv, R_EE);
16
17  const Vec7 tau_task = J.transpose() * wrench;

```

The variables `Kp_used`, `Dp_used`, `KR_used`, and `DR_used` contain the gains selected for the current phase. Their construction and phase assignment are described in Section 3.6. The implementation variable `wrench` follows the force–moment order used in Chapter 2, while `tau_task` corresponds to the Cartesian impedance torque τ_{cart} .

The function `computeCartesianImpedanceWrench` selects between the decoupled Cartesian impedance and the point-shifted impedance associated with the virtual centre of compliance. The decoupled branch is implemented by applying the translational and rotational gain blocks separately:

Listing 3.6: Decoupled Cartesian impedance branch.

```

1  Vec6 wrench;
2  wrench.head<3>() =
3      Kp * dx.head<3>() + Dp * dv.head<3>();
4
5  wrench.tail<3>() =
6      KR * dx.tail<3>() + DR * dv.tail<3>();
7
8  return wrench;

```

This branch is used during tool orientation, surface approach, grinding, and the standard Cartesian pose hold. It is also used during set-up when the virtual centre of compliance is disabled.

The coupled branch is selected when the virtual centre of compliance is enabled during set-up or during the dedicated set-up-impedance hold mode:

Listing 3.7: Selection of compliance-centre coupling.

```

1  const bool coupled =
2      params.use_virtual_compliance_center &&
3      (phase == ControlPhase::kSetup ||
4       phase == ControlPhase::kPoseHold &&
5       params.use_setup_impedance_hold));

```

In this branch, the translational and rotational gain blocks are assembled into spatial 6×6 matrices and shifted from the configured centre of compliance to the TCP. The construction of these matrices and the two implemented definitions of the compliance-centre lever are described in Section 3.7.

The wrench returned by either branch is mapped to joint torque by `J.transpose()` * `wrench`. The resulting `tau_task` contribution is subsequently combined with the null-space, disturbance, and Coriolis torque contributions in the real-time callback.

3.4 SVD-Based Null-Space Control

The mathematical formulation of the null-space projector, projected damping, and singular-value conditioning torque is presented in Section 2.8. This section describes their implementation in the function `computeNullspaceTorque`, which is evaluated during every impedance-controlled callback.

The function receives the current geometric Jacobian, measured joint velocity, robot state, model interface, and active null-space parameters. The current joint configuration is mapped from the robot state, and the full singular value decomposition of the 6×7 Jacobian is calculated. The pseudoinverse and joint-torque null-space projector are then constructed as shown in listing 3.8.

Listing 3.8: Construction of the SVD-based pseudoinverse and joint-torque null-space projector.

```

1 | Map<const Vec7> q_current(state.q.data());
2 |
3 | Eigen::JacobiSVD<Mat6x7> svd_current(
4 |     J, Eigen::ComputeFullU | Eigen::ComputeFullV);
5 |
6 | const Vec6 singular_values =
7 |     svd_current.singularValues();
8 |
9 | const double sigma_max =
10 |     singular_values.maxCoeff();
11 |
12 | const double svd_cutoff =
13 |     std::max(
14 |         0.0,
15 |         params.nullspace_svd_relative_tolerance)
16 |     * sigma_max;
17 |
18 | Mat7x6 J_pinv = Mat7x6::Zero();
19 |
20 | for (int i = 0; i < 6; ++i) {
21 |     if (singular_values(i) > svd_cutoff) {
22 |         J_pinv +=
23 |             (1.0 / singular_values(i))
24 |             * svd_current.matrixV().col(i)
25 |             * svd_current.matrixU().col(i).transpose();
26 |     }
27 | }
28 |
29 | Mat7x7 N_tau =
30 |     Mat7x7::Identity()
31 |     - J.transpose() * J_pinv.transpose();
32 |
33 | N_tau =
34 |     0.5 * (N_tau + N_tau.transpose());

```

The relative tolerance `nullspace_svd_relative_tolerance` is multiplied by the largest singular value of the current Jacobian. This produces the singular-value threshold used during the current callback. Each singular direction above this threshold contributes to the pseudoinverse with the reciprocal $1/\sigma_i$. The remaining reciprocal entries are set to zero.

The implementation forms the joint-torque projector directly according to the expression derived in Section 2.8.3. Averaging the projector with its transpose removes small numerical asymmetries caused by floating-point calculations.

The measured joint velocity is subsequently projected into the numerical null space. The active `NullspaceMode` determines which secondary torque contributions are evaluated:

Listing 3.9: Projection of the joint velocity and selection of the null-space damping contribution.

```

1 | const Vec7 dq_nullspace = N_tau * dq;
2 |
3 | if (params.nullspace_mode ==
4 |     NullspaceMode::kOff) {
5 |     return Vec7::Zero();
6 | }
7 |
8 | const bool damping_enabled =
9 |     params.nullspace_mode ==
10 |         NullspaceMode::kDampingOnly
11 |     ||
12 |     params.nullspace_mode ==
13 |         NullspaceMode::kDampingAndSigma;
14 |
15 | Vec7 tau_damping = Vec7::Zero();
16 |
17 | if (damping_enabled) {
18 |     tau_damping.noalias() =
19 |         -params.nullspace_damping
20 |         * dq_nullspace;
21 | }
22 |
23 | if (params.nullspace_mode ==
24 |     NullspaceMode::kDampingOnly) {
25 |     return tau_damping;
26 | }
27 |
28 | const bool sigma_only =
29 |     params.nullspace_mode ==
30 |         NullspaceMode::kSigmaOnly;
31 |
32 | const Vec7 sigma_fallback =
33 |     sigma_only
34 |     ? Vec7::Zero()
35 |     : tau_damping;

```

The four selectable modes are defined in the controller as

Listing 3.10: Selectable null-space operating modes.

```

1 | enum class NullspaceMode {
2 |     kOff = 0,
3 |     kDampingOnly = 1,
4 |     kSigmaOnly = 2,
5 |     kDampingAndSigma = 3
6 | };

```

The disabled mode returns zero secondary torque. The damping-only mode returns the negative projected joint-velocity damping. The sigma-only mode evaluates the singular-value conditioning contribution, while the combined mode adds both contributions.

For singular-value conditioning, the seventh column of the current right-singular-vector matrix is selected and normalised. This vector corresponds to the redundant direction v_7 introduced in Section 2.8.1. Two nearby joint configurations are then formed along its positive and negative directions.

Listing 3.11: Construction and evaluation of the two sampled joint configurations.

```

1 | Vec7 n =
2 |     svd_current.matrixV().col(6);
3 |
4 | if (n.norm() <= 1e-9) {
5 |     return sigma_fallback;
6 | }
7 |
8 | n.normalize();
9 |
10 | const double alpha =
11 |     std::abs(
12 |         params.nullspace_probe_step_rad);
13 |
14 | if (alpha <= 1e-12) {
15 |     return sigma_fallback;
16 | }
17 |
18 | const Array7 q_plus_array =
19 |     vec7ToArray(
20 |         Vec7(q_current + alpha * n));
21 |
22 | const Array7 q_minus_array =
23 |     vec7ToArray(
24 |         Vec7(q_current - alpha * n));
25 |
26 | const std::array<double, 42> J_plus_array =
27 |     model.zeroJacobian(
28 |         Frame::kEndEffector,
29 |         q_plus_array,
30 |         state.F_T_EE,
31 |         state.EE_T_K);
32 |
33 | const std::array<double, 42> J_minus_array =
34 |     model.zeroJacobian(
35 |         Frame::kEndEffector,
36 |         q_minus_array,
37 |         state.F_T_EE,
38 |         state.EE_T_K);
39 |
40 | Map<const Mat6x7> J_plus(
41 |     J_plus_array.data());
42 |
43 | Map<const Mat6x7> J_minus(
44 |     J_minus_array.data());
45 |
46 | const double sigma_plus =
47 |     smallestSingularValue(J_plus);
48 |
49 | const double sigma_minus =
50 |     smallestSingularValue(J_minus);

```

The parameter `nullspace_probe_step_rad` defines the joint-space distance α_{probe} between the current configuration and the two sampled configurations. The Jacobians are evaluated using the current flange-to-end-effector and end-effector-to-stiffness-frame

transformations provided in the robot state.

The smallest singular values of the sampled Jacobians are compared through the configured deadband. The sign of their difference selects the sampled direction with the larger value of $\sigma_{\min}$.

Listing 3.12: Selection and projection of the singular-value conditioning torque.

```

1 | if (!std::isfinite(sigma_plus) ||
2 |    !std::isfinite(sigma_minus)) {
3 |     return sigma_fallback;
4 | }
5 |
6 | const double sigma_difference =
7 |     sigma_plus - sigma_minus;
8 |
9 | const double deadband =
10 |     std::max(
11 |         0.0,
12 |         params.nullspace_sigma_deadband);
13 |
14 | if (std::abs(sigma_difference)
15 |     <= deadband) {
16 |     return sigma_fallback;
17 | }
18 |
19 | const double sigma_direction =
20 |     (sigma_difference > 0.0)
21 |     ? 1.0
22 |     : -1.0;
23 |
24 | const Vec7 best_direction =
25 |     sigma_direction * n;
26 |
27 | const Vec7 tau_sigma =
28 |     params.nullspace_sigma_gain
29 |     * N_tau
30 |     * best_direction;
31 |
32 | return sigma_only
33 |     ? tau_sigma
34 |     : tau_damping + tau_sigma;

```

The parameter `nullspace_sigma_deadband` specifies the minimum difference required for selecting one sampled direction. Within the deadband, the sigma-only mode returns zero torque and the combined mode retains the damping contribution.

The vector `best_direction` corresponds to sv_7 . Multiplication by the projector retains its numerical null-space component, while `nullspace_sigma_gain` specifies the conditioning-torque magnitude. The function returns the torque associated with the selected mode.

The baseline configuration is loaded from `nullspace.conf`. It uses the combined damping and singular-value conditioning mode with the parameters

$$d_{\text{null}} = 2.0 \text{ N m s/rad}, \quad k_{\sigma} = 1.0 \text{ N m}, \quad (3.6)$$

$$\alpha_{\text{probe}} = 0.08 \text{ rad}, \quad \delta_{\sigma} = 10^{-6}, \quad \rho = 10^{-4}. \quad (3.7)$$

During Cartesian pose hold, the operator can adjust the active null-space mode, damping coefficient, conditioning-torque magnitude, and probe step. The requested values are transferred through atomic variables and assigned inside the callback. Live null-space tuning is associated with the standard Cartesian pose hold, while the phase sequence and set-up-impedance hold use the configuration loaded before the controller session.

3.5 Surface and Tool Geometry

The mathematical definition of the surface task frame is given in Section 2.5. In the implementation, this frame is constructed from the calibrated plane orientation and a tangent-direction hint. The resulting frame is then used for the tool-orientation target, the approach direction, the phase-dependent gains, and the surface-relative evaluation quantities.

3.5.1 Surface Task Frame

The plane configuration contains one surface point expressed in the robot base frame, two surface-tilt angles, and a tangent-direction hint. The parameter loader converts the tilt angles to a unit surface normal, after which `makeSurfaceFrame` constructs the orthonormal surface frame.

Listing 3.13: Construction of the configured surface point, normal, and task frame.

```

1 | params.surface_point =
2 |     getVec3Xyz("surface_point",
3 |         params.surface_point);
4 |
5 | const double ax =
6 |     params.surface_tilt_x_deg * M_PI / 180.0;
7 | const double ay =
8 |     params.surface_tilt_y_deg * M_PI / 180.0;
9 |
10 | params.surface_normal_base =
11 |     Vec3(std::sin(ay) * std::cos(ax),
12 |         -std::sin(ax),
13 |         std::cos(ay) * std::cos(ax));
14 |
15 | params.surface_normal_base.normalize();
16 |
17 | Vec3 normal =
18 |     normalizedOrFallback(
19 |         params.surface_normal_base,
20 |         Vec3(1.0, 0.0, 0.0));
21 |

```

```

22| Vec3 tangent1 =
23|     params.surface_tangent1_hint_base
24|     - normal
25|     * normal.dot(
26|         params.surface_tangent1_hint_base);
27|
28| if (tangent1.norm() <= 1e-9) {
29|     Vec3 fallback(0.0, 1.0, 0.0);
30|     if (std::abs(normal.dot(fallback)) > 0.95) {
31|         fallback = Vec3(0.0, 0.0, 1.0);
32|     }
33|     tangent1 =
34|         fallback - normal * normal.dot(fallback);
35| }
36|
37| tangent1.normalize();
38|
39| Vec3 tangent2 =
40|     normal.cross(tangent1).normalized();
41|
42| Mat3 R_base_surface;
43| R_base_surface.col(0) = tangent1;
44| R_base_surface.col(1) = tangent2;
45| R_base_surface.col(2) = normal;

```

The code variable `R_base_surface` corresponds to R_{surface} in Chapter 2. Its columns follow the fixed order

$$[t_1, t_2, n_s].$$

Projecting the configured tangent hint onto the plane defines t_1 , while the cross product $n_s \times t_1$ defines t_2 . A fallback direction is selected when the tangent hint is approximately parallel to the surface normal.

The active baseline uses the calibrated surface point

$$p_s = \begin{bmatrix} 0.526 & 0.017 & 0.002 \end{bmatrix}^T \text{ m},$$

together with the calibrated surface tilts

$$\alpha_s = -0.474^\circ, \quad \beta_s = 2.270^\circ.$$

The tangent hint is the positive base-frame x -direction. The configuration selector `use_start_as_surface_point` can alternatively assign the captured controller start position to the runtime surface point. The baseline configuration selects the calibrated point from `surface.conf`:

Listing 3.14: Selection of the surface point used during one controller session.

```

1| Vec3 surface_point_runtime =

```

```

2 |     params.use_start_as_surface_point
3 |     ? p_start
4 |     : params.surface_point;

```

The descent direction is defined once from the surface normal:

Listing 3.15: Surface-normal descent direction.

```

1 | const Vec3 descend_direction =
2 |     -R_base_surface.col(2);

```

It therefore points opposite to n_s , from the free-space starting region toward the configured plane.

3.5.2 Surface-Relative Tool Orientation

The controlled physical tool axis is configured in the end-effector frame. During each callback, its current base-frame direction is obtained from

Listing 3.16: Transformation of the configured tool axis to the robot base frame.

```

1 | Vec3 currentToolAxisInBase(
2 |     const ControllerConfig& params,
3 |     const Mat3& R_EE) {
4 |
5 |     return R_EE
6 |         * normalizedOrFallback(
7 |             params.tool_axis_ee,
8 |             Vec3(0.0, 0.0, 1.0));
9 | }

```

The flat alignment direction is selected as either $+n_s$ or $-n_s$ through `tool_axis_target_sign`. The configured angular offsets about the two surface tangents are combined into one rotation vector and applied to this flat direction:

Listing 3.17: Construction of the surface-relative tool-axis target.

```

1 | const double sign =
2 |     (params.tool_axis_target_sign >= 0.0)
3 |     ? 1.0
4 |     : -1.0;
5 |
6 | const Vec3 flat_axis =
7 |     sign * R_base_surface.col(2);
8 |
9 | const Vec3 rotation_vector =
10 |     (M_PI / 180.0)
11 |     * (params.tool_target_offset_tangent1_deg
12 |         * R_base_surface.col(0)
13 |         + params.tool_target_offset_tangent2_deg
14 |         * R_base_surface.col(1));
15 |
16 | const double angle = rotation_vector.norm();
17 |
18 | const Vec3 tool_axis_target =
19 |     (angle <= 1e-12)
20 |     ? flat_axis
21 |     : Eigen::AngleAxisd(
22 |         angle,

```

```

23 |         rotation_vector / angle)
24 |         * flat_axis;

```

This construction permits the tool axis to be aligned with the plane or commanded with a defined initial tilt about t_1 and t_2 . The shortest rotation from the tool axis at the captured start orientation to the selected target axis is then applied to the complete start orientation:

Listing 3.18: Initial tool-axis alignment while retaining the captured orientation about the tool axis.

```

1 | const Vec3 tool_axis_start =
2 |     currentToolAxisInBase(
3 |         params, R_start).normalized();
4 |
5 | const Mat3 R_axis =
6 |     rotationBetweenUnitVectors(
7 |         tool_axis_start,
8 |         tool_axis_target)
9 |     * R_start;

```

When `command_tool_twist` is enabled, the rectangular-face long axis provides an additional in-plane orientation reference. Its current direction and the selected surface tangent are projected onto the plane perpendicular to the desired tool axis. The configured normal-axis offset then determines the desired twist about that axis.

Listing 3.19: Surface-relative twist command about the desired tool axis.

```

1 | const Vec3 face_long_axis_ee =
2 |     normalizedOrFallback(
3 |         params.tool_contact_half_length_ee,
4 |         Vec3(0.0, 1.0, 0.0));
5 |
6 | const auto perpendicular =
7 |     [&tool_axis_target](const Vec3& v) {
8 |         return v
9 |             - v.dot(tool_axis_target)
10 |                * tool_axis_target;
11 |     };
12 |
13 | const Vec3 current =
14 |     perpendicular(R_axis * face_long_axis_ee);
15 |
16 | const Vec3 zero_reference =
17 |     perpendicular(R_base_surface.col(0));
18 |
19 | const Vec3 target =
20 |     Eigen::AngleAxisd(
21 |         (M_PI / 180.0)
22 |         * params.tool_target_offset_normal_deg,
23 |         tool_axis_target)
24 |     * zero_reference.normalized();
25 |
26 | const Vec3 current_unit = current.normalized();
27 |
28 | double twist =
29 |     std::atan2(
30 |         tool_axis_target.dot(
31 |             current_unit.cross(target)),
32 |         current_unit.dot(target));
33 |
34 | const Mat3 R_base_tool_target =
35 |     Eigen::AngleAxisd(
36 |         twist, tool_axis_target)

```

```
37|      * R_axis;
```

For the centred rectangular face, rotations differing by 180° give an equivalent face orientation. The implementation selects the equivalent half-turn nearest to the captured start orientation, thereby reducing the required orientation motion.

The active baseline defines the controlled axis as $+z_{EE}$ and selects alignment toward the surface through

$$\text{tool_axis_target_sign} = -1.$$

The commanded orientation contains an offset of 10° about t_1 , 0° about t_2 , and an in-plane twist of 90° about the target tool axis. The twist command is enabled in `tool_orientation.conf`.

3.5.3 Rectangular Grinding Face and Clearance

The contact geometry is represented by a rectangular face fixed in the end-effector frame. Its centre, half-width vector, and half-length vector define the four vertices:

Listing 3.20: Construction of the rectangular contact face in the end-effector frame.

```
1| const Vec3& face_center =
2|     params.tool_contact_face_center_ee;
3|
4| const Vec3& half_width =
5|     params.tool_contact_half_width_ee;
6|
7| const Vec3& half_length =
8|     params.tool_contact_half_length_ee;
9|
10| const std::array<Vec3, 4>
11|     tool_face_corners_ee = {{
12|         face_center + half_width + half_length,
13|         face_center + half_width - half_length,
14|         face_center - half_width + half_length,
15|         face_center - half_width - half_length,
16|     }};
```

The baseline face centre is located 20 mm along $+z_{EE}$. Its half-width and half-length are 20 mm along x_{EE} and 60 mm along y_{EE} , respectively. The represented contact face therefore measures $40 \text{ mm} \times 120 \text{ mm}$.

Before set-up, the controller transforms all four vertices to the base frame and projects them onto the descent direction. The maximum projection identifies the part of the face that leads during the approach.

Listing 3.21: Selection of the leading face feature along the descent direction.

```

1 | std::array<double, 4> projection;
2 |
3 | double leading_projection =
4 |     (R_EE * tool_face_corners_ee[0])
5 |     .dot(descend_direction);
6 |
7 | projection[0] = leading_projection;
8 |
9 | for (std::size_t i = 1;
10 |     i < tool_face_corners_ee.size();
11 |     ++i) {
12 |     projection[i] =
13 |         (R_EE * tool_face_corners_ee[i])
14 |         .dot(descend_direction);
15 |
16 |     leading_projection =
17 |         std::max(
18 |             leading_projection,
19 |             projection[i]);
20 | }
21 |
22 | Vec3 selected = Vec3::Zero();
23 | int selected_count = 0;
24 |
25 | for (std::size_t i = 0;
26 |     i < tool_face_corners_ee.size();
27 |     ++i) {
28 |     if (leading_projection - projection[i]
29 |         <= params.tool_contact_feature_tie_tolerance) {
30 |         selected += tool_face_corners_ee[i];
31 |         ++selected_count;
32 |     }
33 | }
34 |
35 | const Vec3 tool_contact_offset_ee =
36 |     selected
37 |     / static_cast<double>(selected_count);

```

Vertices whose projections differ from the leading value by at most the configured tolerance are averaged. Depending on the current orientation, the selected feature is therefore a corner, the centre of a leading edge, or the centre of the complete face. The baseline tie tolerance is

$$0.0001 \text{ m} = 0.1 \text{ mm}.$$

The baseline enables both contact-point control and automatic leading-feature selection. The selected offset is transformed to the base frame to obtain the current controlled feature:

Listing 3.22: Base-frame position of the selected tool feature.

```

1 | const Vec3 tool_contact_point =
2 |     p_EE + R_EE * tool_contact_offset_ee;

```

The leading feature is reevaluated while the tool is being oriented and approached. On entry to set-up, its end-effector-frame offset is stored in `active_tool_contact_offset_ee` and subsequently retained during set-up and grinding.

During surface approach, the clearance calculation distinguishes the exact leading projection from the position of the averaged controlled feature:

Listing 3.23: Geometric clearance evaluation during surface approach.

```

1 | const Vec3 surface_normal =
2 |   (-descend_direction).normalized();
3 |
4 | const double height_above_surface =
5 |   surface_normal.dot(
6 |     p_EE - surface_point_runtime)
7 |   - leading_contact_projection;
8 |
9 | const double controlled_point_height =
10 |   surface_normal.dot(
11 |     tool_contact_point
12 |     - surface_point_runtime);
13 |
14 | const Vec3 projected_surface_point =
15 |   tool_contact_point
16 |   - controlled_point_height
17 |   * surface_normal;
18 |
19 | const bool clearance_reached =
20 |   height_above_surface
21 |   <= params.descend_surface_clearance;

```

The quantity `height_above_surface` represents the normal clearance of the foremost point of the complete rectangular face. It provides the geometric transition condition. The quantity `controlled_point_height` is the signed height of the averaged controlled feature. Its projection onto the configured plane provides the surface reference used by the subsequent set-up motion.

When the clearance condition is satisfied, the feature geometry and current pose are captured:

Listing 3.24: Geometry captured at the transition from approach to set-up.

```

1 | active_tool_contact_offset_ee =
2 |   tool_contact_offset_ee;
3 |
4 | first_contact_tcp = p_EE;
5 |
6 | first_contact_point =
7 |   projected_surface_point;
8 |
9 | setup_push_start =
10 |   -controlled_point_height;
11 |
12 | R_contact_start = R_EE;
13 |
14 | phase = ControlPhase::kSetup;
15 | phase_start_time = time;

```

The internal variable `first_contact_point` stores the projected clearance point on the configured plane. The live signed height determines `setup_push_start`, which produces a continuous desired position when the controller changes from approach to set-up.

During set-up, the desired position of the selected feature is advanced from this projected

point along the descent direction. The feature target is then converted to the corresponding TCP reference using the orientation captured at the clearance transition:

Listing 3.25: Conversion of the set-up feature target to the desired TCP position.

```

1 | const Vec3 edge_target =
2 |     first_contact_point
3 |     + push * descend_direction;
4 |
5 | desired.p_d =
6 |     edge_target
7 |     - R_contact_start
8 |     * tool_contact_offset_ee;
9 |
10 | desired.pdot_d =
11 |     setup_push_velocity
12 |     * descend_direction;

```

This construction allows the translational impedance controller to command the selected tool feature while the Cartesian control law remains formulated at the TCP.

3.6 Phase-Dependent Gains and Inertia-Scaled Damping

The controller assigns separate Cartesian stiffness and damping matrices to the approach, set-up, grinding, pose-hold, and phase-gate modes. The matrices are constructed before the real-time callback begins, while the inertia-scaled damping matrices are updated on entry to the corresponding phase group.

3.6.1 Phase-Dependent Gain Construction

The function `buildPhaseImpedanceGains` constructs all phase-dependent gain matrices from the active controller configuration. Gains defined along the surface directions are transformed to the robot base frame using

Listing 3.26: Transformation of diagonal surface-frame gains to the robot base frame.

```

1 | PhaseImpedanceGains gains;
2 | gains.R_base_surface = makeSurfaceFrame(params);
3 |
4 | auto taskGain = [&](const Vec3& diagonal) -> Mat3 {
5 |     return makeSpatialGainMatrix(
6 |         diagonal,
7 |         gains.R_base_surface);
8 | };
9 |
10 | gains.Kp_approach =
11 |     taskGain(params.approach_Kp_diag);
12 | gains.Dp_approach =
13 |     taskGain(params.approach_Dp_diag);

```

```

14 | gains.KR_approach =
15 |     taskGain(params.approach_KR_diag);
16 | gains.DR_approach =
17 |     taskGain(params.approach_DR_diag);

```

The function `makeSpatialGainMatrix` evaluates

$$G_0 = R_{\text{surface}} G_{\text{surface}} R_{\text{surface}}^\top, \quad (3.8)$$

where G_{surface} is diagonal in the fixed axis order $[t_1, t_2, n_s]$. The resulting matrix G_0 is expressed in the robot base frame and can be applied directly to the base-frame Cartesian error.

The gain-frame policy implemented for each controller mode is summarised in table 3.2.

Table 3.2: Coordinate frames used for the phase-dependent Cartesian gains.

Controller mode	Translational gains	Rotational gains
Tool orientation and surface approach	Surface frame $[t_1, t_2, n_s]$	Surface frame $[t_1, t_2, n_s]$
Set-up and grinding	Selectable between base frame $[x, y, z]$ and surface frame $[t_1, t_2, n_s]$	Surface frame $[t_1, t_2, n_s]$
Cartesian pose hold	Base frame $[x, y, z]$	Base frame $[x, y, z]$
Set-up-impedance hold	Same construction as set-up	Same construction as set-up
Phase-gate hold	Base frame $[x, y, z]$	Surface frame $[t_1, t_2, n_s]$

For set-up translation, the parameter `setup_translation_surface_frame` selects the coordinate frame:

Listing 3.27: Selection of the translational set-up gain frame.

```

1 | gains.Kp_setup =
2 |     params.setup_translation_surface_frame
3 |     ? taskGain(params.setup_Kp_surface_diag)
4 |     : params.setup_Kp_diag.asDiagonal();
5 |
6 | gains.Dp_setup =
7 |     params.setup_translation_surface_frame
8 |     ? taskGain(params.setup_Dp_surface_diag)
9 |     : params.setup_Dp_diag.asDiagonal();
10 |
11 | gains.KR_setup =
12 |     taskGain(params.setup_KR_diag);
13 |
14 | gains.DR_setup =

```

```
15| taskGain(params.setup_DR_diag);
```

The baseline configuration selects base-frame translational gains for set-up and grinding.

The corresponding rotational gains remain aligned with the surface tangents and normal.

Cartesian pose-hold gains are assembled as diagonal base-frame matrices:

Listing 3.28: Construction of the pose-hold and phase-gate gains.

```
1| gains.Kp_hold =
2|     params.hold_Kp_diag.asDiagonal();
3| gains.Dp_hold =
4|     params.hold_Dp_diag.asDiagonal();
5| gains.KR_hold =
6|     params.hold_KR_diag.asDiagonal();
7| gains.DR_hold =
8|     params.hold_DR_diag.asDiagonal();
9|
10| gains.Kp_pause =
11|     params.pause_hold_Kp_diag.asDiagonal();
12| gains.Dp_pause =
13|     params.pause_hold_Dp_diag.asDiagonal();
14| gains.KR_pause =
15|     taskGain(params.pause_hold_KR_diag);
16| gains.DR_pause =
17|     taskGain(params.pause_hold_DR_diag);
```

The real-time callback selects the active matrices from the current control phase:

Listing 3.29: Selection of the active phase-dependent gain matrices.

```
1| const Mat3* Kp_phase =
2|     params.use_setup_impedance_hold
3|     ? &Kp_setup
4|     : &Kp_hold;
5|
6| const Mat3* Dp_phase =
7|     params.use_setup_impedance_hold
8|     ? &Dp_setup_eff
9|     : &Dp_hold_eff;
10|
11| const Mat3* KR_phase =
12|     params.use_setup_impedance_hold
13|     ? &KR_setup
14|     : &KR_hold;
15|
16| const Mat3* DR_phase =
17|     params.use_setup_impedance_hold
18|     ? &DR_setup_eff
19|     : &DR_hold_eff;
20|
21| switch (phase) {
22|     case ControlPhase::kToolOrientation:
23|     case ControlPhase::kSurfaceApproach:
24|         Kp_phase = &Kp_approach;
25|         Dp_phase = &Dp_approach_eff;
26|         KR_phase = &KR_approach;
27|         DR_phase = &DR_approach_eff;
28|         break;
29|
30|     case ControlPhase::kSetup:
31|     case ControlPhase::kGrinding:
32|         Kp_phase = &Kp_setup;
33|         Dp_phase = &Dp_setup_eff;
34|         KR_phase = &KR_setup;
35|         DR_phase = &DR_setup_eff;
36|         break;
37|
38|     case ControlPhase::kPoseHold:
39|     case ControlPhase::kManualGuidance:
40|         break;
41| }
```

Set-up and grinding therefore share the same stiffness and damping matrices. The difference between these phases lies in the reference trajectory and in the selection of the coupled or decoupled impedance law.

An active phase gate overrides the normal phase matrices with the dedicated gate-hold gains:

Listing 3.30: Gain override during an operator confirmation gate.

```

1 | const Mat3& Kp_used =
2 |   pause_hold_active ? Kp_pause : *Kp_phase;
3 |
4 | const Mat3& Dp_used =
5 |   pause_hold_active ? Dp_pause_eff : *Dp_phase;
6 |
7 | const Mat3& KR_used =
8 |   pause_hold_active ? KR_pause : *KR_phase;
9 |
10 | const Mat3& DR_used =
11 |   pause_hold_active ? DR_pause_eff : *DR_phase;
```

These four matrices are passed to the Cartesian impedance routine during the current callback.

3.6.2 Calculation of Inertia-Scaled Damping

Each phase first receives the damping matrices configured in the parameter files. When inertia-scaled damping is enabled for a phase group, the controller replaces these matrices with values calculated from the current operational-space inertia.

The joint-space inertia matrix is obtained from the libfranka model. The implementation uses an LDLT factorisation to solve for $M^{-1}J^T$ and constructs the inverse operational-space inertia:

Listing 3.31: Calculation of the operational-space inertia estimate.

```

1 | Eigen::LDLT<Mat7x7> mass_ldlt(joint_mass);
2 |
3 | const Mat7x6 Minv_Jt =
4 |   mass_ldlt.solve(J.transpose());
5 |
6 | Mat6x6 lambda_inv =
7 |   J * Minv_Jt;
8 |
9 | lambda_inv =
10 |   0.5 * (lambda_inv
11 |     + lambda_inv.transpose());
12 |
13 | Mat6x6 lambda_inv_regularized =
14 |   lambda_inv;
15 |
16 | lambda_inv_regularized
17 |   .diagonal().array() += 1e-9;
18 |
19 | Eigen::LDLT<Mat6x6> lambda_ldlt(
20 |   lambda_inv_regularized);
```

```

21 |
22 | Mat6x6 Lambda_base =
23 |     lambda_ldlt.solve(
24 |         Mat6x6::Identity());

```

This calculation corresponds to the regularised estimate

$$\Lambda_0 = \left(JM^{-1}J^\top + 10^{-9}I_6 \right)^{-1}. \quad (3.9)$$

The matrix is symmetrised before and after its factorisation. It is then transformed from the robot base axes to the coordinate frame of the active gain block:

Listing 3.32: Transformation and extraction of directional Cartesian inertia.

```

1 | const Mat6x6 T_task =
2 |     blockDiagonalRotation(R_task);
3 |
4 | Mat6x6 Lambda_task =
5 |     T_task.transpose()
6 |     * Lambda_base
7 |     * T_task;
8 |
9 | for (int i = 0; i < 3; ++i) {
10 |     result.translational(i) =
11 |         Lambda_task(i, i);
12 |
13 |     result.rotational(i) =
14 |         Lambda_task(i + 3, i + 3);
15 | }

```

The first three diagonal entries provide the directional translational inertias, while the last three provide the directional rotational inertias. For each direction i , the inertia-scaled damping coefficient is calculated from

$$d_i = 2\zeta\sqrt{\lambda_i k_i}, \quad (3.10)$$

where λ_i is the corresponding directional Cartesian inertia, k_i is the directional stiffness, and ζ is the configured damping ratio. For $\zeta = 1$, the expression gives the critical-damping coefficient of the corresponding decoupled direction.

The calculation is implemented independently for the three translational and three rotational directions:

Listing 3.33: Calculation and limiting of the inertia-scaled damping coefficients.

```

1 | for (int i = 0; i < 3; ++i) {
2 |     const double inertia_i =
3 |         std::max(0.0, inertia(i));
4 |
5 |     const double stiffness_i =
6 |         std::max(0.0, stiffness(i));
7 |
8 |     const double calculated =

```

```
9      2.0 * zeta
10      * std::sqrt(
11          inertia_i * stiffness_i);
12
13      damping(i) =
14          std::min(
15              max_damping,
16              std::max(
17                  min_damping(i),
18                  calculated));
19  }
```

The common upper bound is defined by `auto_damping_max`. The parameter `auto_damping_min_from_manual` selects either zero or the configured damping value as the lower bound.

The coordinate frame used for the inertia estimate follows the frame of the corresponding stiffness matrix:

- approach translation and rotation use the surface frame;
- set-up translation uses the selected base or surface frame, while set-up rotation uses the surface frame;
- Cartesian pose hold uses the robot base frame; and
- the phase gate uses base-frame translation and surface-frame rotation.

The damping matrices are cached by phase group. Entering tool orientation or surface approach calculates the approach damping. Entering set-up, grinding, or set-up-impedance hold calculates the set-up damping. Cartesian pose hold uses its hold damping, while an active operator gate uses the gate-hold damping. Leaving a phase group clears its cached state so that a later entry uses the current robot configuration.

For the phase-gate hold, separate damping ratios are derived from the configured translational and rotational gate-hold damping values. Cartesian pose hold can similarly derive one damping ratio from its configured translational damping and apply it to the translational and rotational inertia-scaled damping calculations.

The baseline configuration enables inertia-scaled damping for approach, set-up, and the phase gates. Standard Cartesian pose hold uses its configured damping matrices. The exact stiffness, damping-factor, and limiting values used during the experiments are listed in Chapter 4.

When the inertia calculation satisfies the numerical validity checks, the calculated damping matrices replace the configured matrices for the active phase group. The configured

matrices remain available as the initial and fallback values.

3.7 Virtual Centre of Compliance

The theoretical transformation from a centre-defined Cartesian impedance to an equivalent TCP impedance is derived in Section 2.7. The implementation applies this transformation inside `computeCartesianImpedanceWrench`. The phase-dependent translational and rotational gain blocks are first expressed in the robot base frame and are then shifted from the selected virtual centre of compliance to the TCP.

The coupled branch is activated during set-up and during the dedicated set-up-impedance hold mode:

Listing 3.34: Selection of the virtual-compliance-centre branch.

```

1 | const bool coupled =
2 |     params.use_virtual_compliance_center &&
3 |     (phase == ControlPhase::kSetup ||
4 |      (phase == ControlPhase::kPoseHold &&
5 |       params.use_setup_impedance_hold));
6 |
7 | if (!coupled) {
8 |     Vec6 wrench;
9 |     wrench.head<3>() =
10 |         Kp * dx.head<3>()
11 |         + Dp * dv.head<3>();
12 |
13 |     wrench.tail<3>() =
14 |         KR * dx.tail<3>()
15 |         + DR * dv.tail<3>();
16 |
17 |     return wrench;
18 | }
```

Tool orientation, surface approach, grinding, and standard Cartesian pose hold therefore use the decoupled impedance branch. A pre-grinding confirmation gate retains `ControlPhase::kSetup`; the coupled set-up impedance consequently remains active while the controller waits at the achieved preload.

The controller supports two definitions of the virtual centre. Both follow the lever convention

$$r_c = p_{\text{TCP}} - p_c$$

introduced in Equation (2.50). The first definition specifies the centre position relative to the TCP in end-effector coordinates. The second specifies the lever r_c directly in surface

coordinates.

Listing 3.35: Resolution of the compliance-centre lever in the robot base frame.

```

1 | Vec3 r_c = Vec3::Zero();
2 |
3 | if (params.compliance_center_in_tool_frame) {
4 |     r_c =
5 |         -(R_EE
6 |          * params.compliance_center_offset_ee);
7 | } else {
8 |     r_c =
9 |         R_base_surface
10 |        * params.r_tcp_from_compliance_center_surface;
11 | }

```

For the tool-frame definition, `compliance_center_offset_ee` represents $p_c - p_{TCP}$. The multiplication by R_{EE} expresses this offset in the robot base frame, while the negative sign converts it to the implemented lever convention. Since the measured orientation R_{EE} is used during every callback, this lever follows the current end-effector orientation.

For the surface-frame definition, `r_tcp_from_compliance_center_surface` contains the components of r_c along $[t_1, t_2, n_s]$. Multiplication by $R_{base_surface}$ expresses the lever in the robot base frame. The surface frame and configured lever remain fixed during one controller session, so this definition produces a constant base-frame lever throughout set-up.

When virtual-compliance-centre control is enabled, the configuration loader requires exactly one of these definitions:

Listing 3.36: Validation of the compliance-centre configuration.

```

1 | if (params.use_virtual_compliance_center &&
2 |     params.compliance_center_in_tool_frame ==
3 |     params.compliance_lever_in_surface_frame) {
4 |     throw std::runtime_error(
5 |         "Virtual compliance-center control requires "
6 |         "exactly one definition.");
7 | }

```

The function `complianceShiftMatrix` constructs the point-shift matrix derived in Equation (2.52). The skew-symmetric lever matrix occupies the upper-right block because the Cartesian vectors follow the ordering [translation, rotation].

Listing 3.37: Implementation of the spatial point shift and gain congruence.

```

1 | Mat6x6 complianceShiftMatrix(
2 |     const Vec3& r_c) {
3 |
4 |     Mat6x6 adjoint = Mat6x6::Zero();
5 |
6 |     adjoint.block<3, 3>(0, 0) =
7 |         Mat3::Identity();
8 |
9 |     adjoint.block<3, 3>(0, 3) =

```

```

10     skewMatrix(r_c);
11
12     adjoint.block<3, 3>(3, 3) =
13         Mat3::Identity();
14
15     return adjoint;
16 }
17
18 Mat6x6 shiftGainToTcp(
19     const Mat6x6& gain_at_center,
20     const Vec3& r_c) {
21
22     const Mat6x6 adjoint =
23         complianceShiftMatrix(r_c);
24
25     return adjoint.transpose()
26         * gain_at_center
27         * adjoint;
28 }

```

The phase-dependent 3×3 translational and rotational matrices are combined into centre-defined spatial gain matrices. The same congruence is then applied independently to stiffness and damping:

Listing 3.38: Construction and evaluation of the coupled TCP impedance.

```

1  const Mat6x6 K_tcp =
2      shiftGainToTcp(
3          blockDiagonal(Kp, KR),
4          r_c);
5
6  const Mat6x6 D_tcp =
7      shiftGainToTcp(
8          blockDiagonal(Dp, DR),
9          r_c);
10
11 return K_tcp * dx + D_tcp * dv;

```

The function `blockDiagonal` places K_p and D_p in the translational blocks and K_R and D_R in the rotational blocks. These matrices have already been transformed into the frame selected for the active phase. The subsequent congruence changes the impedance reference point while preserving the selected principal-axis orientation.

The off-diagonal blocks generated by the point shift couple translation and rotation. A translational displacement or velocity error can therefore contribute to the commanded TCP moment, while a rotational error can contribute to the commanded TCP force. The coupling disappears when the configured lever is zero, in which case the shifted matrices reduce to their block-diagonal form.

During set-up, the desired orientation remains the orientation captured at the clearance transition. This orientation acts as a soft rotational reference while the selected tool feature is advanced into the surface. The resulting rotation is determined by the coupled impedance, the contact wrench, the directional stiffness, and the physical tool–surface interaction rather than by a prescribed rotational trajectory.

The set-up and grinding phases share the same 3×3 stiffness and damping blocks. At the transition to grinding, the state machine assigns `ControlPhase::kGrinding` before the Cartesian wrench is evaluated later in the same callback. The first grinding sample therefore uses the decoupled branch. The implementation applies this change as a direct switch from the shifted 6×6 matrices to the decoupled gain blocks, while the achieved penetration and desired orientation remain continuous across the transition.

The set-up-impedance hold mode permits the translational stiffness, rotational stiffness, commanded tilt, and compliance-centre coordinates to be adjusted through the operator interface. Centre-position commands are entered in end-effector coordinates, while lever commands are entered in surface coordinates. An accepted update is converted to the selected internal representation, after which the phase gains and inertia-scaled-damping cache are rebuilt.

After set-up, the reporting module resolves the selected centre in both end-effector and surface coordinates. It also evaluates the effective TCP stiffness and damping matrices, the largest translation-rotation coupling entry, and the eigenvalue ranges of the shifted matrices. These quantities support verification of the configured lever, its sign, and the resulting coupled impedance.

3.8 Control Modes and Phase Sequence

The controller state is represented by the enumeration `ControlPhase`. It contains the four phases of the automatic sequence together with Cartesian pose hold and manual guidance:

Listing 3.39: Control phases and standalone operating modes.

```
1 | enum class ControlPhase {  
2 |     kToolOrientation,  
3 |     kSurfaceApproach,  
4 |     kSetup,  
5 |     kGrinding,  
6 |     kPoseHold,  
7 |     kManualGuidance  
8 | };
```

The automatic sequence follows

`kToolOrientation` $\longrightarrow$ `kSurfaceApproach` $\longrightarrow$ `kSetup` $\longrightarrow$ `kGrinding`.

The orientation phase can be omitted through the controller configuration. The initial phase is selected before the real-time callback begins:

Listing 3.40: Selection of the initial phase.

```

1 | const ControlPhase sequence_first_phase =
2 |     params.enable_orientation_phase
3 |     ? ControlPhase::kToolOrientation
4 |     : ControlPhase::kSurfaceApproach;
5 |
6 | const ControlPhase initial_phase =
7 |     params.run_phase_sequence
8 |     ? sequence_first_phase
9 |     : ControlPhase::kPoseHold;
10 |
11 | ControlPhase phase = initial_phase;
12 | ControlPhase restart_phase = initial_phase;

```

The variable `restart_phase` stores the phase resumed after manual guidance. The complete sequence, standard pose hold, and set-up-impedance hold can also be selected during an active controller session. The requested mode is received through an atomic variable and applied inside the callback.

3.8.1 Tool Orientation

The tool-orientation phase maintains the captured start position while the desired orientation is advanced toward the surface-relative target described in Section 3.5.2. The orientation command follows a rate-limited interpolation from the captured start orientation.

The transition condition is evaluated from two errors. The tool-axis error measures the angular separation between the current and desired tool axes. When the in-plane twist command is enabled, the remaining rotation about the target axis is evaluated separately as the spin error.

Listing 3.41: Tool-orientation convergence and phase transition.

```

1 | const bool orientation_reached =
2 |     tool_axis_error
3 |     <= params.approach_orient_error_threshold
4 |     &&
5 |     (!params.command_tool_twist ||
6 |     spin_error
7 |     <= params.approach_orient_spin_error_threshold);
8 |
9 | const bool orient_timed_out =
10 |     params.approach_orient_timeout > 0.0
11 |     &&
12 |     phase_time >= params.approach_orient_timeout;
13 |
14 | if (phase_time >= params.approach_orient_min_time &&
15 |     (orientation_reached || orient_timed_out)) {
16 |     phase = ControlPhase::kSurfaceApproach;
17 |     phase_start_time = time;

```

```

18 |
19 |     contact_force_bias = external_force;
20 |     contact_moment_bias = external_moment;
21 | }

```

The configured minimum duration allows the orientation command to begin settling before convergence is accepted. The timeout provides a second transition path and allows the sequence to continue with the orientation reached within the available interval. The force and moment estimates are reassigned as diagnostic bias values when surface approach begins.

3.8.2 Surface Approach

During surface approach, the desired TCP position advances from the captured start position along the descent direction:

Listing 3.42: Surface-approach translation reference.

```

1 | const double phase_time =
2 |     time - phase_start_time;
3 |
4 | const double distance =
5 |     std::min(
6 |         params.descend_speed
7 |         * (phase_time - gate_paused_time),
8 |         params.descend_max_distance);
9 |
10 | desired.p_d =
11 |     p_start + distance * descend_direction;
12 |
13 | desired.pdot_d =
14 |     params.descend_speed * descend_direction;

```

The variable `gate_paused_time` removes operator-confirmation time from the descent trajectory. The commanded distance is limited by `descend_max_distance`.

The transition to set-up is determined from the geometric clearance of the leading part of the rectangular tool face. The estimated external force is displayed as a diagnostic quantity during approach, while the phase transition uses

Listing 3.43: Geometric transition from surface approach to set-up.

```

1 | const double height_above_surface =
2 |     surface_normal.dot(
3 |         p_EE - surface_point_runtime)
4 |     - leading_contact_projection;
5 |
6 | const bool clearance_reached =
7 |     height_above_surface
8 |     <= params.descend_surface_clearance;
9 |
10 | if (clearance_reached && !gate_blocking) {
11 |     active_tool_contact_offset_ee =
12 |         tool_contact_offset_ee;
13 |
14 |     first_contact_tcp = p_EE;

```

```

15 | first_contact_point =
16 |     projected_surface_point;
17 |
18 | setup_push_start =
19 |     -controlled_point_height;
20 |
21 | R_contact_start = R_EE;
22 |
23 | contact_force_bias = external_force;
24 | contact_moment_bias = external_moment;
25 |
26 | phase = ControlPhase::kSetup;
27 | phase_start_time = time;
28 | }

```

The selected tool-feature offset is locked at this transition. The controller also stores the current TCP pose, the projected point on the surface, the signed starting value of the set-up penetration, and the current force and moment estimates.

When the optional pre-set-up gate is enabled, reaching the clearance first captures the measured TCP position. The controller then holds this position with the dedicated phase-gate gains:

Listing 3.44: Operator gate between approach and set-up.

```

1 | if (!gate_setup_armed) {
2 |     gate_setup_armed = true;
3 |     gate_setup_hold_pd = p_EE;
4 |     signals.gate_continue.store(false);
5 | }
6 |
7 | if (signals.gate_continue.load()) {
8 |     gate_setup_passed = true;
9 |     signals.gate_continue.store(false);
10 | } else {
11 |     gate_blocking = true;
12 |     pause_hold_active = true;
13 |
14 |     desired.p_d = gate_setup_hold_pd;
15 |     desired.pdot_d.setZero();
16 |
17 |     gate_paused_time += period.toSec();
18 | }

```

The geometric clearance condition remains satisfied while the gate is active. After confirmation, the feature and contact references are captured from the current measured state and the controller enters set-up.

Reaching the maximum permitted descent distance before the clearance condition sets the stop request and records the approach as unsuccessful.

3.8.3 Set-Up

The set-up phase advances the selected tool feature from the projected surface point into the plane. The scalar penetration begins at the signed feature coordinate captured during

the approach transition and progresses toward `setup_push_end` at the configured speed.

Listing 3.45: Set-up penetration and TCP reference.

```

1 | double setup_push_velocity = 0.0;
2 |
3 | const double push =
4 |     setupPush(
5 |         params,
6 |         phase_time - gate_grind_paused_time,
7 |         setup_push_start,
8 |         setup_push_velocity);
9 |
10 | const Vec3 edge_target =
11 |     first_contact_point
12 |     + push * descend_direction;
13 |
14 | desired.p_d =
15 |     edge_target
16 |     - R_contact_start
17 |     * tool_contact_offset_ee;
18 |
19 | desired.pdot_d =
20 |     setup_push_velocity
21 |     * descend_direction;

```

The function `setupPush` integrates a constant signed speed and clamps the result at the configured final penetration. Subtracting `gate_grind_paused_time` freezes this trajectory while the controller waits at the pre-grinding gate.

The desired orientation remains `R_contact_start`, which is the orientation captured at the clearance transition. The tool can consequently rotate relative to this soft reference under the coupled impedance and the physical contact wrench.

During set-up, the changes in external force and moment are calculated relative to the values captured at the approach transition:

Listing 3.46: Set-up termination condition.

```

1 | const double force_delta_norm =
2 |     (external_force
3 |     - contact_force_bias).norm();
4 |
5 | const double moment_delta_norm =
6 |     (external_moment
7 |     - contact_moment_bias).norm();
8 |
9 | const bool stopped_on_moment =
10 |     phase_time >= params.setup_min_time
11 |     &&
12 |     moment_delta_norm
13 |     >= params.setup_moment_threshold;
14 |
15 | if (!gate_grind_armed &&
16 |     !stopped_on_moment &&
17 |     phase_time < params.setup_timeout) {
18 |     break;
19 | }

```

The force change is recorded and included in the set-up evaluation. The phase exit is governed by the bias-subtracted moment norm after the configured minimum duration.

The set-up timeout provides the alternative exit condition.

Before continuing, the controller evaluates and prints the set-up result once. This report includes the achieved pose, feature position, alignment change, force and moment changes, moment transferred to the selected feature, active impedance matrices, and compliance-centre diagnostics.

When the pre-grinding gate is enabled, the controller remains in the set-up phase after the exit condition has been reached:

Listing 3.47: Operator gate between set-up and grinding.

```

1 | if (params.pause_before_grind &&
2 |     !gate_grind_passed) {
3 |     if (!gate_grind_armed) {
4 |         gate_grind_armed = true;
5 |         signals.gate_continue.store(false);
6 |     }
7 |
8 |     if (!signals.gate_continue.load()) {
9 |         gate_grind_paused_time
10 |            += period.toSec();
11 |
12 |         break;
13 |     }
14 |
15 |     gate_grind_passed = true;
16 |     signals.gate_continue.store(false);
17 | }
18 |
19 | grind_push = push;
20 | phase = ControlPhase::kGrinding;
21 | phase_start_time = time;

```

The coupled set-up impedance and the achieved pressed-feature reference remain active while the controller waits. After confirmation, the current penetration is stored in `grind_push` and becomes the normal preload of the grinding phase.

3.8.4 Grinding

Grinding retains the final set-up penetration. The controller can either add a tangential sweep or maintain only the normal penetration.

For an active sweep, the selected tangent is obtained from the surface frame. The scalar sweep trajectory is then added to the pressed-feature reference:

Listing 3.48: Grinding reference with an optional tangential sweep.

```

1 | const Vec3 n = descend_direction;
2 | Vec3 edge_target;
3 |
4 | if (params.grind_sweep_enabled) {
5 |     const Vec3 grind_tangent =
6 |         (params.grind_tangent_axis == 2)
7 |         ? Vec3(R_base_surface.col(1))
8 |         : Vec3(R_base_surface.col(0));
9 |
10 |     double sweep_s = 0.0;

```

```

11 | double sweep_s_dot = 0.0;
12 |
13 | grindSweep(
14 |     time - phase_start_time,
15 |     params.grind_amplitude_m,
16 |     grindStrokeDuration(params),
17 |     sweep_s,
18 |     sweep_s_dot);
19 |
20 | edge_target =
21 |     first_contact_point
22 |     + grind_push * n
23 |     + sweep_s * grind_tangent;
24 |
25 | desired.pdot_d =
26 |     sweep_s_dot * grind_tangent;
27 | } else {
28 |     const double edge_penetration =
29 |         n.dot(
30 |             tool_contact_point
31 |             - first_contact_point);
32 |
33 |     edge_target =
34 |         tool_contact_point
35 |         + (grind_push - edge_penetration) * n;
36 | }
37 |
38 | desired.p_d =
39 |     edge_target
40 |     - R_contact_start
41 |     * tool_contact_offset_ee;

```

The function `grindSweep` uses a fifth-order smooth-step trajectory. The first stroke moves from the centre position to the positive amplitude. Subsequent strokes alternate smoothly between the positive and negative amplitudes with zero velocity at each reversal.

When the sweep is disabled, the reference is corrected only along the surface normal. Its tangential coordinates follow the current tool-feature position, which preserves the set-up penetration without imposing a tangential trajectory.

Grinding retains the set-up stiffness and damping values. The Cartesian impedance routine selects the decoupled branch after the phase has changed to `kGrinding`, as described in Section 3.7.

3.8.5 Pose Hold and Manual Guidance

The pose-hold state supports two impedance configurations. Standard pose hold uses the dedicated hold gains, while set-up-impedance hold applies the set-up gains and can retain the virtual centre of compliance. Both modes regulate a captured Cartesian pose.

When the set-up-impedance hold is selected during a run, the measured pose is first captured as the current controller state. The reference is then moved smoothly toward the pose at which the hold session began. Translation uses the configured approach speed, while orientation uses the configured maximum angular rate. After both references have

reached their stored values, the controller continues as a stationary hold.

Manual guidance can be entered from any impedance-controlled phase. The controller then returns Coriolis compensation together with joint-space damping:

Listing 3.49: Manual-guidance torque and pose recapture.

```
1 | const Vec7 tau_guide =  
2 |     coriolis  
3 |     - params.manual_guidance_damping * dq;  
4 |  
5 | if (signals.proceed_requested.load()) {  
6 |     signals.proceed_requested.store(false);  
7 |  
8 |     restartFromPoseReached(true);  
9 |     phase = restart_phase;  
10 |  
11 |     hold_return_p = p_start;  
12 |     hold_return_R = R_d;  
13 |     hold_returning = false;  
14 | }
```

After operator confirmation, the measured position, orientation, joint configuration, surface reference when selected, and external-wrench biases are recaptured. The controller then resumes the phase stored in `restart_phase`.

3.8.6 Run Termination

The callback finishes normally when a positive experiment duration has elapsed or the operator issues the stop command:

Listing 3.50: Normal termination of the real-time callback.

```
1 | if ((params.experiment_duration > 0.0 &&  
2 |     time >= params.experiment_duration)  
3 |     ||  
4 |     signals.stop_requested.load()) {  
5 |     return MotionFinished(  
6 |         Torques(tau_array));  
7 | }
```

The same stop signal is assigned when the maximum approach distance is reached. Robot-side safety monitoring can also interrupt the torque-control session through a libfranka exception. The subsequent error handling and collision configuration are described in Section 3.9.

3.9 Robot Preparation and Safety Handling

Robot preparation is performed before the real-time impedance callback begins. It includes

the initial FCI connection, automatic error recovery, collision configuration, motion to the selected initial posture, and the optional gripper and tool-transfer actions.

3.9.1 Robot Connection and Collision Configuration

At program start, the robot address is read from the controller configuration and used to establish the FCI connection. Automatic error recovery is requested before the collision behaviour and robot model are loaded:

Listing 3.51: Robot connection, error recovery, and model initialisation.

```
1 | const ControllerConfig startup_config =
2 |     readControllerConfig(parameter_files);
3 |
4 | Robot robot(startup_config.robot_ip);
5 |
6 | try {
7 |     robot.automaticErrorRecovery();
8 | } catch (const franka::Exception& e) {
9 |     fprintf(stderr,
10 |            "Automatic error recovery failed: %s\n",
11 |            e.what());
12 |     return -1;
13 | }
14 |
15 | configureCollisionBehavior(
16 |     robot, startup_config);
17 |
18 | Model model = robot.loadModel();
```

Failure of the recovery request terminates the program before a motion command is issued. Further recovery or unlocking is then performed through Franka Desk before restarting the controller.

The function `configureCollisionBehavior` first applies the standard robot behaviour provided by the `libfranka` example utilities. The custom thresholds defined in `safety.conf` are subsequently assigned when their selector is enabled:

Listing 3.52: Assignment of the configured collision thresholds.

```
1 | void configureCollisionBehavior(
2 |     Robot& robot,
3 |     const ControllerConfig& params) {
4 |
5 |     setDefaultBehavior(robot);
6 |
7 |     if (!params.use_custom_collision_behavior) {
8 |         return;
9 |     }
10 |
11 |     const Array7 torque_acc =
12 |         filledArray7(
13 |             params.collision_torque_acc);
14 |
15 |     const Array7 torque_nom =
16 |         filledArray7(
17 |             params.collision_torque_nom);
18 |
19 |     const Array6 force_acc =
```

```

20         filledArray6(
21             params.collision_force_acc);
22
23     const Array6 force_nom =
24         filledArray6(
25             params.collision_force_nom);
26
27     robot.setCollisionBehavior(
28         torque_acc, torque_acc,
29         torque_nom, torque_nom,
30         force_acc, force_acc,
31         force_nom, force_nom);
32 }

```

The two joint-torque parameters define the acceleration-phase and nominal thresholds in Nm. Each scalar is assigned uniformly to all seven joints. The two Cartesian parameters are similarly assigned to all six force and moment components in the acceleration and nominal phases. The same array is passed for the lower and upper threshold of each group.

The controller returns the calculated joint torque directly to libfranka:

Listing 3.53: Transfer of the calculated torque command to libfranka.

```

1 | const Array7 tau_array =
2 |     vec7ToArray(tau_cmd);
3 |
4 | return Torques(tau_array);

```

Torque boundedness and collision response are therefore handled by the robot-side monitoring and the configured collision behaviour. The application adds no separate saturation of the Cartesian wrench, null-space torque, complete joint-torque command, or joint-torque rate.

3.9.2 Initial Joint Posture

The parameter `q_init_case` selects one of the joint configurations stored in `initial_pose.conf`. The corresponding seven joint angles are assigned to `q_init` when the configuration is loaded.

The phase sequence and both Cartesian hold modes begin from this posture. The start-up interface moves the robot to it using the libfranka `MotionGenerator`:

Listing 3.54: Motion to the configured initial joint posture.

```

1 | const auto move_to_q_init = [&]() {
2 |     const Vec7 q_init =
3 |         Map<const Vec7>(params.q_init.data());
4 |
5 |     MotionGenerator motion_generator(
6 |         0.4, params.q_init);
7 |

```

```
8 | robot.control(motion_generator);  
9 |  
10 | q_init_reached = true;  
11 | };
```

The speed factor is fixed at 0.4 for this preparation motion. A local flag records whether the posture has already been reached and prevents repeated motion when the operator performs several preparation actions before selecting a run mode.

Selection of the phase sequence, standard pose hold, or set-up-impedance hold calls `ensure_q_init`. Manual guidance forms the alternative start path and begins from the current robot pose. The guided pose is captured before the subsequent sequence or hold mode starts.

3.9.3 Gripper Verification

The start-up interface provides separate commands for opening, grasping, and recalibrating the Franka Hand. Each action reads the gripper state before and after the command and verifies the resulting finger position.

For opening, successful verification requires the libfranka motion command to finish, the calibrated maximum stroke to contain the requested width, and the measured final width to reach the target within a 2 mm tolerance. Grasp verification requires the grasp command to finish, the `is_grasped` flag to be active, and the measured width to lie within the configured inner and outer tolerances.

Before either action, the reported maximum finger width is compared with the largest width used by the controller. A smaller calibrated stroke indicates that gripper homing was performed while an object restricted the finger motion. The start-up interface then requests recalibration with empty fingers.

A configured gripper action can also be executed automatically immediately before torque control. A previously verified manual action is retained and is therefore not repeated. When the requested action is grasping, an already verified tool grasp is also retained:

Listing 3.55: Selection of the automatic gripper action before torque control.

```
1 | if (params.perform_gripper_action_before_run &&  
2 |     !params.start_with_manual_guidance &&  
3 |     !params.startup_gripper_action_completed) {  
4 |  
5 |     Gripper gripper(params.robot_ip);  
6 |  
7 |     if (params.gripper_startup_grasp_tool) {
```

```

8 |     const franka::GripperState state =
9 |         gripper.readOnce();
10 |
11 |     const bool already_holding =
12 |         state.is_grasped &&
13 |         state.width >=
14 |             params.gripper_grasp_width
15 |             - params.gripper_grasp_epsilon_inner &&
16 |         state.width <=
17 |             params.gripper_grasp_width
18 |             + params.gripper_grasp_epsilon_outer;
19 |
20 |     gripper_ok =
21 |         already_holding
22 |         ? true
23 |         : graspTool(params, gripper);
24 | } else {
25 |     gripper_ok =
26 |         openGripper(params, gripper);
27 | }
28 | }

```

The parameter `abort_on_gripper_action_failure` determines whether a failed automatic action terminates the program or allows the selected controller session to continue.

3.9.4 Tool Transfer

The controller also provides an optional sequence for collecting the tool from its holder or returning it. The holder configuration is represented by the joint posture `q_pickup`. A stand-off posture is calculated at the same end-effector orientation and at the configured distance along $-z_{EE}$.

The stand-off configuration is obtained through damped least-squares inverse kinematics. Each iteration calculates the position and orientation error, forms the current end-effector Jacobian, and limits the joint-update norm. The resulting posture is accepted only when all joints remain within their configured limits with a 0.05 rad margin.

After successful calculation, the tool-transfer path is executed as

$$q_{\text{init}} \longrightarrow q_{\text{above}} \longrightarrow q_{\text{pickup}} \longrightarrow q_{\text{above}}.$$

The corresponding implementation is:

Listing 3.56: Joint-space motion sequence for collecting or returning the tool.

```

1 | Array7 q_above{};
2 |
3 | if (!solveStandoffPosture(
4 |     model,
5 |     robot.readOnce(),
6 |     params.q_pickup,
7 |     params.pickup_standoff,
8 |     q_above)) {
9 |     return false;

```

```
10 | }
11 |
12 | int bad_joint = 0;
13 | if (!withinJointLimits(
14 |     q_above, bad_joint)) {
15 |     return false;
16 | }
17 |
18 | ensure_q_init();
19 |
20 | MotionGenerator to_above(
21 |     0.4, q_above);
22 | robot.control(to_above);
23 |
24 | MotionGenerator descend(
25 |     std::max(
26 |         0.05,
27 |         params.pickup_descend_speed_factor),
28 |     params.q_pickup);
29 | robot.control(descend);
30 |
31 | const bool action_ok =
32 |     grasp
33 |         ? graspTool(params, gripper)
34 |         : openGripper(params, gripper);
35 |
36 | MotionGenerator lift(
37 |     std::max(
38 |         0.05,
39 |         params.pickup_descend_speed_factor),
40 |     q_above);
41 | robot.control(lift);
```

When collecting the tool, the gripper is opened and verified before the arm moves toward the holder. At the pickup posture, the selected gripper action is executed and verified before the robot returns to the stand-off posture.

3.9.5 Exception Handling

Robot communication, motion-generation, and torque-control errors are reported through `franka::Exception`. Configuration, file, and numerical errors are reported through `std::exception`. Both exception types leave the active control operation and terminate the program with an error code.

A robot-side safety event can therefore end the torque callback independently of the controller phase or experiment timer. The corresponding exception is reported after control has stopped, and the robot is recovered through Franka Desk before a new program execution.

3.10 Data Recording and Run Evaluation

The controller records the quantities required for the subsequent run evaluation in a bounded in-memory ring buffer. Its memory is allocated before `robot.control` starts, so

the real-time callback performs no dynamic buffer growth.

Listing 3.57: Allocation of the bounded logging buffer before torque control.

```
1 | const int log_every_n_cycles =  
2 |     std::max(  
3 |         1,  
4 |         params.log_every_n_cycles);  
5 |  
6 | const std::size_t max_log_rows =  
7 |     static_cast<std::size_t>(  
8 |         std::max(  
9 |             0,  
10 |            params.max_log_rows));  
11 |  
12 | std::vector<LogData> log_data(  
13 |     max_log_rows);  
14 |  
15 | std::size_t control_cycle_count = 0;  
16 | std::size_t log_write_index = 0;  
17 | std::size_t log_rows_written = 0;  
18 | bool log_buffer_wrapped = false;
```

The parameter `log_every_n_cycles` defines the sampling interval in control cycles, while `max_log_rows` defines the maximum number of samples retained in memory. The current baseline records every control cycle and allocates space for 120 000 rows.

The structure `LogData` stores one sample of the measured state, controller reference, calculated command, and active controller mode. The recorded quantities are summarised in table 3.3.

Table 3.3: Signal groups stored during the real-time control loop.

Signal group	Recorded quantities
Time and controller state	Run time, encoded control phase, and encoded null-space mode.
TCP state and reference	Measured and desired TCP positions, translational and rotational errors, measured linear and angular velocities, and desired linear velocity.
Tool and surface geometry	Current controlled tool-feature position, feature offset in the end-effector frame, projected clearance point, TCP position at the clearance transition, desired feature target, and virtual surface penetration.
Alignment	Tool-axis alignment-error components along $[t_1, t_2, n_s]$ and their Euclidean norm.
Cartesian wrench	Commanded force and moment, model-estimated external force and moment, and the force and moment biases captured at the clearance transition.
Null-space controller	Active null-space damping coefficient and the Euclidean norm of the calculated null-space torque.
Disturbance experiment	Disturbance scale, torque-limit scale, force-application point, commanded Cartesian force, and equivalent joint torque.
Joint-torque command	The seven final commanded joint torques returned to libfranka.

The internal variables and CSV columns named `first_contact_tcp` and `first_contact_point` refer to the geometric clearance transition described in Section 3.8.2. They represent the captured TCP position and projected tool-feature point at this transition, rather than a force-detected contact event.

The selected values are copied into the current ring-buffer row at the configured sampling interval:

Listing 3.58: Assignment of one sampled controller state to the ring buffer.

```
1| ++control_cycle_count;
```

```

2
3 if (max_log_rows > 0 &&
4     control_cycle_count
5         % static_cast<std::size_t>(
6             log_every_n_cycles)
7         == 0) {
8
9     LogData& row =
10         log_data[log_write_index];
11
12     row.time = time;
13     row.phase = static_cast<int>(phase);
14     row.nullspace_mode =
15         static_cast<int>(
16             params.nullspace_mode);
17
18     row.p_EE = p_EE;
19     row.p_d = desired.p_d;
20     row.tool_contact_point =
21         tool_contact_point;
22     row.first_contact_tcp =
23         first_contact_tcp;
24     row.first_contact_point =
25         first_contact_point;
26     row.edge_target = edge_target_log;
27     row.tool_contact_offset_ee =
28         tool_contact_offset_ee;
29
30     row.e_p = e_p;
31     row.e_R = e_R;
32     row.alignment_error_surface =
33         R_base_surface.transpose()
34         * toolSurfaceAlignmentErrorBase(
35             params,
36             R_EE,
37             R_base_surface);
38     row.alignment_angle =
39         row.alignment_error_surface.norm();
40
41     row.pdot = pdot;
42     row.pdot_d = desired.pdot_d;
43     row.omega = omega;
44
45     row.f = f;
46     row.m = m;
47     row.external_force = external_force;
48     row.external_moment = external_moment;
49     row.contact_force_bias =
50         contact_force_bias;
51     row.contact_moment_bias =
52         contact_moment_bias;
53
54     row.push = push_log;
55     row.disturbance = disturbance;
56     row.tau_nullspace_norm =
57         tau_nullspace.norm();
58     row.nullspace_damping =
59         params.nullspace_damping;
60     row.tau_cmd = tau_cmd;
61 }

```

The current implementation represents the null-space action through the active mode, damping coefficient, and resulting torque norm. These quantities permit the secondary torque to be distinguished from the Cartesian and disturbance contributions during post-processing.

After assigning one row, the write index is advanced cyclically:

Listing 3.59: Advancement of the bounded ring-buffer index.

```

1 log_write_index =
2     (log_write_index + 1)

```

```

3 |     % max_log_rows;
4 |
5 | if (log_rows_written < max_log_rows) {
6 |     ++log_rows_written;
7 | } else {
8 |     log_buffer_wrapped = true;
9 | }

```

Once the allocated capacity is reached, new samples replace the oldest rows. The buffer therefore retains the most recent `max_log_rows` samples instead of increasing its memory requirement during control.

3.10.1 Chronological Reconstruction and CSV Output

After the torque callback has finished, the ring-buffer contents are converted to chronological order. When the buffer has wrapped, the oldest retained sample begins at `log_write_index`:

Listing 3.60: Reconstruction of the chronological sample order after control.

```

1 | std::vector<LogData> ordered_log_data;
2 | ordered_log_data.reserve(
3 |     log_rows_written);
4 |
5 | if (log_buffer_wrapped) {
6 |     ordered_log_data.insert(
7 |         ordered_log_data.end(),
8 |         log_data.begin()
9 |         + static_cast<std::ptrdiff_t>(
10 |             log_write_index),
11 |         log_data.end());
12 |
13 |     ordered_log_data.insert(
14 |         ordered_log_data.end(),
15 |         log_data.begin(),
16 |         log_data.begin()
17 |         + static_cast<std::ptrdiff_t>(
18 |             log_write_index));
19 | } else {
20 |     ordered_log_data.insert(
21 |         ordered_log_data.end(),
22 |         log_data.begin(),
23 |         log_data.begin()
24 |         + static_cast<std::ptrdiff_t>(
25 |             log_rows_written));
26 | }
27 |
28 | result.log =
29 |     std::move(ordered_log_data);

```

The ordered samples and final controller state are returned in a `RunResult`. File output is subsequently performed by `writeRunLogs`, outside the real-time callback:

Listing 3.61: Post-run CSV output and terminal summary.

```

1 | void writeRunLogs(
2 |     const ControllerConfig& params,
3 |     const RunResult& result) {
4 |
5 |     if (result.descend_failed) {
6 |         printSection("descend stopped");

```

```
7 |     printf(  
8 |         "    maximum distance reached "  
9 |         "before the clearance height.\n");  
10 | }  
11 |  
12 | printJointStartEndTableDeg(  
13 |     result.q_start,  
14 |     result.final_q);  
15 |  
16 | writeLogToCsv(  
17 |     result.log,  
18 |     params.csv_file_name);  
19 |  
20 | printFinalSummary(  
21 |     result.final_p_d,  
22 |     result.final_p_EE,  
23 |     result.final_e_p,  
24 |     result.final_e_R,  
25 |     params.csv_file_name);  
26 | }
```

The CSV writer creates the parent output directory when required and uses a fixed column order. Cartesian positions, velocities, forces, moments, disturbances, and joint torques are written in their stored SI units. Alignment errors are converted from radians to degrees for the output file.

The bias-subtracted external-force and external-moment columns are calculated during CSV writing from the stored estimates:

Listing 3.62: Writing the bias-subtracted external wrench to the CSV file.

```
1 | writeVec3(  
2 |     log_file,  
3 |     row.external_force  
4 |     - row.contact_force_bias);  
5 |  
6 | writeVec3(  
7 |     log_file,  
8 |     row.external_moment  
9 |     - row.contact_moment_bias);
```

The commanded Cartesian wrench and the model-estimated external wrench are therefore retained as separate quantities. The former is generated by the impedance controller, while the latter is supplied by the robot state and used for set-up evaluation.

When several controller sessions are executed without restarting the program, the session number is inserted before the CSV filename extension:

Listing 3.63: Generation of separate CSV filenames for repeated sessions.

```
1 | if (session > 1) {  
2 |     config.csv_file_name =  
3 |         sessionFileName(  
4 |             config.csv_file_name,  
5 |             session);  
6 | }
```

For example, a second session based on `controller_log.csv` is written as `controller_log_s2.csv`.

3.10.2 Set-Up Evaluation

A dedicated set-up report is generated once when the moment threshold or set-up time-out is reached. The report captures the final TCP pose, selected tool-feature position, external wrench change, clearance-transition references, active gain matrices, and set-up termination condition.

The external moment is also transferred from the TCP to the selected tool feature. The implementation uses the bias-subtracted external force and moment:

Listing 3.64: Calculation of the external moment at the selected tool feature.

```

1 | const Vec3 edge_from_tcp =
2 |   tool_contact_point - p_EE;
3 |
4 | const Vec3 contact_moment_at_edge =
5 |   (external_moment
6 |    - contact_moment_bias)
7 |   -
8 |   edge_from_tcp.cross(
9 |     external_force
10 |    - contact_force_bias);

```

The terminal set-up report contains:

- the set-up termination condition and phase duration;
- the external-force and external-moment changes;
- the TCP and tool-feature displacements from the clearance transition;
- the tool-alignment error before and after set-up;
- the alignment-error components along t_1 , t_2 , and n_s ;
- the force, moment, displacement, and rotation resolved in the surface frame; and
- the active translational and rotational stiffness and damping matrices.

The displayed force–displacement and moment–angle ratios provide a pointwise diagnostic of the completed set-up state. They are calculated only when the corresponding displacement or rotation exceeds its numerical minimum.

When compliance-centre diagnostics are enabled, the report additionally resolves r_c in surface coordinates and p_c in end-effector coordinates. It reconstructs the shifted TCP stiffness and damping matrices and reports their diagonal entries, largest translation–rotation coupling terms, and eigenvalue ranges. The complete 6×6 matrices can also be printed for verification of the configured lever and gain transformation.

The final post-run summary displays the joint displacement between the start and end

configurations, the desired and measured final TCP positions, the final translational and rotational errors, and the path of the generated CSV file.

The controller implementation described in this chapter forms the basis of all reported experiments. Chapter 4 presents the calibration procedure, the controller parameters used during the measurements, the varied experimental conditions, and the quantities derived from the recorded signals.

Chapter 4

Experimental Methodology

4.1 Experimental Investigation

GREEN — sufficient. The completed campaign, fixed baseline, five cases, run count, repetitions, and separation from the internally commanded hold study are all introduced before the detailed procedure.

Four parameters were investigated: the commanded tilt, the rotational stiffness about each surface tangent, the translational stiffness perpendicular to the tilted tangent, and the position of the virtual centre of compliance. Each can change the rotation observed during contact. The commanded tilt sets the initial angular error. The two stiffness entries determine how strongly the controller resists the resulting translational and rotational motion. The compliance-centre position determines how much of the translational contact response is converted into a moment, through the point-shifted impedance developed in the preceding chapters.

Two methodological changes distinguish these measurements from the earlier campaign preserved in appendix D. The surface plane and the grinding-face axis were calibrated independently of one another, which defines the commanded tilt against a measured physical surface frame instead of against a configured plane that differed from it. The rotational stiffness was also varied on one tangent at a time. This separated the t_1 and t_2 contributions that a paired sweep cannot identify.

Three probed points defined the plane. A fourth point provided an independent check and lay 0.82 mm from it. A separate set of seatings calibrated the grinding-face normal,

with an independent angular residual of 0.50° . Projecting the direction from the first probed point to the second gives $+t_1$, with $t_2 = n_s \times t_1$.

Every condition shares one baseline impedance. Its translational block is diagonal in base coordinates and its rotational block in the surface frame,

$$K_p = \text{diag}(2000, 2000, 350) \text{ N/m (base } xyz), \quad K_R = \text{diag}(5, 5, 50) \text{ N m/rad (surface),}$$

the values derived in eq. (4.15) and eq. (4.17), together with a 60 mm virtual press at 50 mm/s and a 4 s set-up timeout. Projected null-space damping and the singular-value-based torque were both active during all reported runs, in the combined mode of table C.4. Fixing the commanded spin of the grinding face at 115.05° from t_1 makes the leading contact feature follow from the calibrated surface frame rather than from the initial joint configuration.

The runs were divided into five cases. In Case A the response to the commanded tilt alone was measured and used as the baseline for the remaining cases. In Cases B and C one stiffness entry was varied. In Case D only the compliance-centre lever was varied, and in Case E the Case-A tilt was commanded about the tool-face axes rather than the surface tangents.

Table 4.1: Completed experimental cases.

Case	Test	Tilt	Varied parameter and values	n	Purpose
A	Commanded tilt	varied	Commanded offset 0° ; $+5^\circ$ and $+10^\circ$ about t_1 ; $+5^\circ$ and $+10^\circ$ about t_2	15	Establish the response to the commanded tilt on each surface tangent.
B	Axis-specific rotational stiffness	$+10^\circ$ about t_1 or t_2	K_R of the tilted tangent = 15 and 50 N m/rad; orthogonal entry held at 5 N m/rad	12	Separate the t_1 and t_2 rotational-stiffness contributions.
C	Cross-axis translational stiffness	$+10^\circ$ about t_1 or t_2	K_p perpendicular to the tilted tangent = 300 and 800 N/m; one corner per axis at $K_p = 300$ N/m with $K_R = 50$ N m/rad	18	Measure the translational contribution and its interaction with rotational stiffness.
D	Virtual centre of compliance	$+10^\circ$ about t_1 or t_2	Perpendicular lever $r_c = -60, 0$ and $+60$ mm; and the lever at the grinding-face centre, $r_c = (0, -3.5, 19.7)$ mm for t_1 and $(3.5, 0, 19.7)$ mm for t_2	24	Measure the response to lever placement and to its sign.
E	Tilt about the tool axes	10° about y_{EE} or x_{EE}	Resolved onto the tangents as $t_1 = -4.23^\circ$, $t_2 = +9.06^\circ$ for y_{EE} ; and $t_1 = +9.06^\circ$, $t_2 = +4.23^\circ$ for x_{EE}	6	Attribute the tangent-axis asymmetry to the surface frame or the tool face.

Three runs per setting. Cases A, B, C and E command the decoupled impedance; Case D commands the point-shifted impedance with the levers listed, expressed in the surface frame as $r_c = p_{TCP} - p_c$. Every case holds the baseline $K_p = \text{diag}(2000, 2000, 350)$ N/m in base xyz and $K_R = \text{diag}(5, 5, 50)$ N m/rad in the surface frame, except where the table states otherwise, together with a 60 mm virtual press at 50 mm/s and a 4 s set-up timeout.

Cases A–E comprise 75 contact runs. All completed set-up and were included in the analysis. The null-space terms were held constant in this campaign and therefore did not define an additional contact case. A separate internally commanded hold study is

described in section 4.4.6.

Each run followed the same sequence. The robot oriented the grinding-face axis, then descended to a geometrically defined clearance. Set-up pressed the selected leading tool feature into the plane, holding the clearance orientation as a soft reference and commanding no time-varying task-space tipping trajectory, and the grinding motion followed. The evaluation used the EE-inferred tool-axis angle, the rotation during contact, and the displacements of the feature and the TCP.

4.2 Robot and Software Configuration

GREEN — sufficient. The robot interface, control period, state/model inputs, command structure, and active phase set are reproducible and consistent with the implementation chapter.

The experiments ran on the Franka Emika Panda through the libfranka torque-control interface, with the controller executing inside the Franka Control Interface at the nominal control period

$$T_s = 1 \text{ ms.} \quad (4.1)$$

One cycle retrieves the robot state, evaluates the Cartesian impedance control law, and returns a joint torque command. The evaluation draws on the end-effector transformation T_{EE} and the joint velocity $\dot{q}$ from the state, and on the geometric Jacobian $J(q)$ and the Coriolis term from the libfranka model interface. The commanded torque has the form

$$\tau_{\text{cmd}} = J^\top(q)F_{\text{cmd}} + \tau_{\text{null}} + \tau_c(q, \dot{q}), \quad (4.2)$$

where F_{cmd} is the commanded Cartesian impedance wrench and τ_{null} is the projected null-space torque. No gravity term appears in the user callback. The Franka Control Interface already applies robot-side gravity and motor-friction compensation to any external torque command sent through libfranka, so adding one would double it.

Four states make up the active sequence: `approach_orient`, `approach_descend`, `set_up`, and `grind`. Hold and manual guidance are available at start-up, but neither is a contact

phase. Null-space mode is selected on its own and, once enabled, stays active for the whole commanded sequence.

4.2.1 Experimental System and Reproducibility

GREY — logical or evidential gap. The software stack is well recorded, but payload parameters, robot transforms, and real-time-computer hardware were not recorded. These omissions prevent exact replication of dynamics and timing.

The controller computer ran Ubuntu 20.04.3 on the 5.9.1-rt20 PREEMPT_RT kernel, with g++ 9.3 at C++14 and -O2, Eigen 3.3.7, and libfranka 0.7.1. Table 4.2 collects the rest of the documented system; parameter values for the individual studies appear in the tables further down.

Table 4.2: Experimental system.

Item	Configuration used in the reported experiments
Robot and interface	Franka Emika Panda operated through the libfranka torque-control interface at a nominal 1 kHz control rate.
Operating system and kernel	Ubuntu 20.04.3; Linux 5.9.1-rt20 with PREEMPT_RT.
Compiler and libraries	g++ 9.3, C++14, -O2; Eigen 3.3.7; libfranka 0.7.1.
Tool and surface	Rectangular tool held in the Franka Hand by custom fingers; plane surface mounted on a raised base board. The configured tool-face dimensions and tool offset are reported in table C.1.
Contact condition	Dry surface contact. No environment friction model or friction coefficient is used in the control command.
Recorded quantities	Robot pose and velocity, selected tool feature, phase state, Cartesian error, commanded joint torque, and robot-model estimate of the external wrench.

Several things went unrecorded: the payload parameters, the robot transforms, and the hardware of the real-time computer. Chapter 6 treats their influence as a limitation.

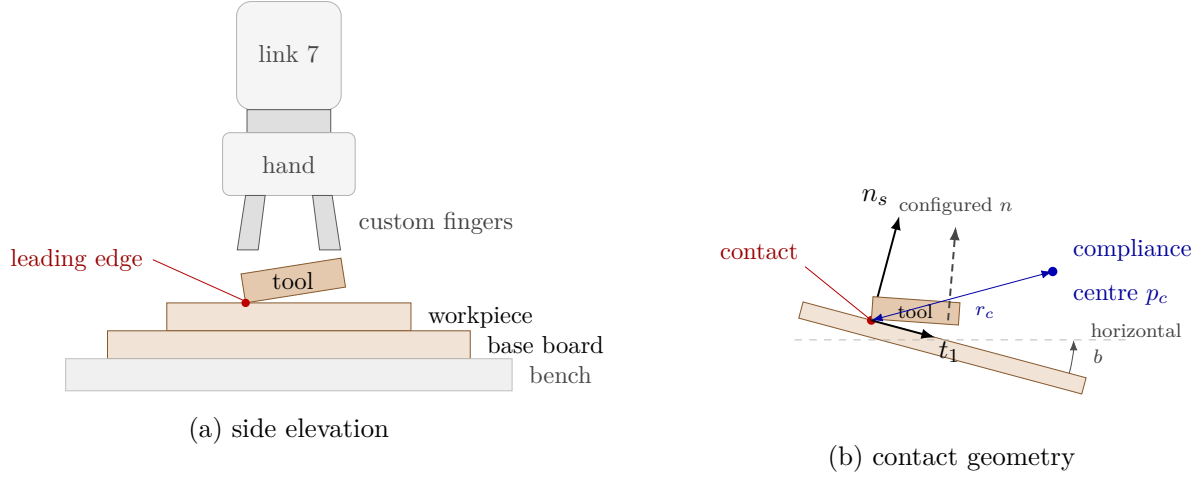


Figure 4.1: Experimental set-up and surface-frame contact geometry.

4.2.2 Configured and Physical Plane

GREY — logical or evidential gap. The two plane definitions are separated correctly, but the physical-plane estimate has substantial angular spread and only one held-out point. Absolute residual-angle claims therefore remain weak.

Two plane definitions were used, and they are not interchangeable. The controller operated with the configured plane alone: the plane point $p_s = [0.55, -0.06, 0.025]^\top$ m and the normal n_s , obtained from $a = 0^\circ$ and $b = 5^\circ$ in eq. (4.4). Approach, clearance, feature projection, surface frame and commanded tool orientation all follow from it.

The physical orientation of the surface was estimated for the alignment metric. Three robot-probed points defined the plane, and a fourth independent point lay 0.82 mm from it. The procedure was repeated three times, giving $a_{\text{phys}} = -0.72 \pm 0.31^\circ$ and $b_{\text{phys}} = 14.99 \pm 1.91^\circ$ in the tilt parametrisation of eq. (4.4), from which n_{phys} follows. The controller itself plays no part in this estimate: it is configured with a and b directly and has no calibration routine.

Between n_s and n_{phys} lies 10.01° , and that separation is the initial geometric misalignment the alignment study works on. The repeatability of the probing limits how far an absolute residual angle can be trusted, the spread on b_{phys} being the larger of the two. A change measured within one run is less exposed to that offset, which is why the reported outcome is a change rather than a final absolute angle.

4.3 Contact Geometry

YELLOW — weak or unclear. The geometry is necessary, but much of the surface frame, tool-axis target, rectangular face, and lever notation repeats Chapter 3. A shorter experiment-specific statement with cross-references would be clearer.

The task geometry is defined by a surface plane, a tool edge, and a selected local linearised centre of compliance. Let

$$n_s \in \mathbb{R}^3, \quad \|n_s\| = 1 \quad (4.3)$$

be the surface normal expressed in the robot base frame. In the implemented configuration it is generated from rotations a about base x and b about base y ,

$$n_s = R_y(b)R_x(a)e_z = \begin{bmatrix} \sin(b) \cos(a) & -\sin(a) & \cos(b) \cos(a) \end{bmatrix}^\top. \quad (4.4)$$

Here $a = 0^\circ$ and $b = 5^\circ$, with the plane through $p_s = [0.55, -0.06, 0.025]^\top$ m. Two orthonormal tangents t_1 and t_2 span it: projecting the configured auxiliary direction into the plane gives the first, and $t_2 = n_s \times t_1$ the second. The implemented column order is

$$R_{\text{surface}} = \begin{bmatrix} t_1 & t_2 & n_s \end{bmatrix}, \quad (4.5)$$

matching the surface task frame of eq. (2.41). Every surface-frame vector and every diagonal gain in this chapter follows the order $[t_1, t_2, n_s]$ unless a table states another semantic order explicitly. With the active auxiliary direction $a_s = e_x$ and the angles above, the axes come out as

$$t_1 = [\cos 5^\circ, 0, -\sin 5^\circ]^\top, \quad t_2 = [0, 1, 0]^\top.$$

Rotating about t_2 is therefore rotation about base y , while rotating about t_1 tips the tool about the orthogonal tangent, which lies in the tilted base x - z plane.

One fixed point would not describe the grinding face, so the face is carried explicitly. Its centre sits 20 mm along $+Z_{\text{EE}}$ from the end-effector origin, and it measures 40 mm along X_{EE} by 120 mm along Y_{EE} . The controller transforms all four corners into the base frame

and picks whichever corner, edge, or complete face leads along the descent direction, averaging any features whose projected heights agree to within 0.1 mm. Without that tolerance an exact single-axis tilt would pick one endpoint of a physical edge at random. The configured physical face-normal axis is a_{EE} , as defined in section 3.5.2. For the current tool $a_{EE} = e_z$, and the orientation target satisfies

$$R_d a_{EE} = -n_s. \quad (4.6)$$

The minus sign turns the $+Z_{EE}$ grinding face toward the surface. What the controller applies is the minimum rotation carrying the starting nominal tool axis onto $-n_s$, which preserves as much of the remaining twist as it can. Setting $R_d = R_{\text{surface}}$ would not: it would command the twist as well.

The parallel-jaw grasp allowed approximately $\pm 2^\circ$ of relative tool rotation about y_{EE} . This direction lies mainly in the surface-tangent plane for the tested mounting pose, but it is not identical to t_2 . The calibrated tangent is fixed in the base frame, whereas $y_{EE} = R_{EE} e_{y,EE}$ moves with the end-effector.

A point p_c carries the selected centre of compliance, with the lever $r_c = p_{\text{TCP}} - p_c$ of eq. (2.50). Its p_{TCP} is the origin of the end-effector pose the robot reports, so relating that point to the physical hand and tool means carrying any configured flange-to-end-effector transform along with it.

Case matters in the subscript. Lower-case c belongs to the chosen virtual centre p_c ; upper-case C appears later for the physical contact point p_C , whose lever $r_C = p_C - p_{\text{TCP}}$ points the other way. The two are never interchangeable.

A force f at the contact produces a moment about the TCP through r_C , which is why the selected stiffness matrix will not read as three independent translational springs beside three independent rotational ones. Virtual-centre behaviour requires translation and rotation to be selected together.

4.4 Gain Selection Methodology

YELLOW — weak or unclear. The section is technically complete but very long. It mixes frame bookkeeping, sizing assumptions, damping policy, compliance-centre construction, and experimental variation before the reader reaches the actual tested levels.

Notation follows Chapter 2. A subscript task marks a gain written in a local task frame, and a surface-frame quantity takes the axis order $[t_1, t_2, n_s]$. No single frame serves every phase and every gain block, however:

- approach translation and rotation are diagonal in the surface frame and transformed to the base frame;
- set-up and grind translation are diagonal directly in base $[x, y, z]$, while their rotational gains are diagonal in $[t_1, t_2, n_s]$;
- hold gains are isotropic in the base frame; the enabled pre-set-up gate overrides only translation with an isotropic base-frame gain and retains the isotropic approach rotational gain; and
- during set-up only, the selected base-frame translational and rotational blocks can be shifted from a commanded centre of compliance to the TCP, producing a coupled 6×6 gain.

Table 4.3 maps the stiffness across every phase — hold, both approach states, the pre-set-up gate, set-up, grind and manual guidance — and table 4.4 gives the matching inertia-scaled-damping policy. The coordinate frame is part of the gain definition and must be retained whenever the gain is calculated or transformed.

Base-frame errors and base-frame gains determine the control law of eq. (2.37). For experimental interpretation, the same quantities are expressed in the surface task frame as

$$e_{\text{task}} = \begin{bmatrix} e_{p,\text{task}} \\ e_{R,\text{task}} \end{bmatrix} = T_R^\top \begin{bmatrix} e_p \\ e_R \end{bmatrix}, \quad \dot{x}_{\text{task}} = T_R^\top \begin{bmatrix} \dot{p}_{\text{EE}} \\ \omega_{\text{EE}} \end{bmatrix}, \quad (4.7)$$

where $T_R = \text{diag}(R_{\text{surface}}, R_{\text{surface}})$. The representation serves interpretation, where both blocks are being read in the surface frame. This representation does not alter the stored set-up translation block, which remains diagonal in base XYZ.

The complete local Cartesian stiffness matrix keeps the task-frame notation of eq. (2.43). In the standard spatial-stiffness block form [6, 17],

$$K_{\text{task}} = \begin{bmatrix} K_{p,\text{task}} & K_{pR,\text{task}} \\ K_{Rp,\text{task}} & K_{R,\text{task}} \end{bmatrix}. \quad (4.8)$$

Translational displacement reaches force through $K_{p,\text{task}}$ and rotational displacement reaches moment through $K_{R,\text{task}}$, while the off-diagonal blocks $K_{pR,\text{task}}$ and $K_{Rp,\text{task}}$ represent the translation–rotation coupling. This block form separates the coupling terms from the diagonal translational and rotational terms. During set-up, the full matrix is constructed analytically from a selected centre of compliance and the source stiffness blocks. The resulting 36 entries are given below.

The construction assumes a symmetric positive-definite stiffness and a symmetric positive-semidefinite damping matrix,

$$K_{\text{task}} = K_{\text{task}}^\top, \quad K_{\text{task}} \succ 0. \quad (4.9)$$

Both properties are preserved by the congruence and were verified for the assembled 6×6 matrices. This verification detects sign and transpose errors without introducing an additional tuning parameter.

4.4.1 Quasi-Static Stiffness Sizing and Inertia-Scaled Damping

GREY — logical or evidential gap. The selected force, moment, displacement, and angle scales are engineering choices rather than measured contact requirements. The blocked spring calculations are useful bounds, but they do not validate the chosen gains or the coupled damping.

The stiffness values were sized offline and the calculation was not evaluated on the robot. It defines a virtual-spring command scale rather than an estimated contact force. At low velocity the elastic parts of the implemented command separate from damping and inertia,

$$f = K_p e_p, \quad m = K_R e_R. \quad (4.10)$$

For a principal coordinate i , the stiffness is the ratio between the chosen elastic-load scale and the corresponding error scale:

$$k_{p,i} = \frac{|f_i|}{|e_{p,i}|}, \quad (4.11)$$

$$k_{R,i} = \frac{|m_i|}{|e_{R,i}|}. \quad (4.12)$$

The values inserted into these ratios were sizing values rather than measured contact loads. Where a moment scale came from an offset force, its component about the selected unit axis a_i was

$$|a_i^\top (r_C \times f_C)| \leq \|r_C\| \|f_C\|,$$

rather than the scalar lever-arm bound being used without its axis direction [15, 19].

Frame-consistent directional stiffness. Outside a principal-gain frame the scalar rule of eq. (4.11) no longer applies directly. For a gain diagonal in frame R_F , the base-frame matrix and the effective slope along a base-frame unit direction u are

$$K_{p,0} = R_F \text{diag}(k_{p,1}, k_{p,2}, k_{p,3}) R_F^\top, \quad k_{p,\text{eff}}(u) = u^\top K_{p,0} u = \sum_{i=1}^3 k_{p,i} (r_i^\top u)^2. \quad (4.13)$$

An error $e_p = \delta u$ gives the elastic component $\delta k_{p,\text{eff}}(u)$ along u , and along a non-principal direction it also raises the transverse component $\delta(I - uu^\top)K_{p,0}u$. The same construction covers $K_{R,0}$ and a rotational direction. Set-up translation needs it: that block is diagonal in base XYZ while the push follows the tilted surface normal.

Set-up/grind calculation. Base y and the active t_2 coincide, which makes the lateral entry easy to size. Treating the completely blocked 30 mm grind stroke as a 60 N elastic design scale gives

$$k_y = \frac{60}{0.030} = 2000 \text{ N/m}, \quad k_x = k_y. \quad (4.14)$$

The press direction takes a nominal 65 N elastic scale at the configured 0.180 m endpoint, so $k_{p,\text{eff}}(n_s) = 65/0.180 = 361.11 \text{ N/m}$. That coordinate starts near -0.020 m at the

clearance hand-off and is scheduled to +0.180 m, roughly 0.200 m of reference travel, which at 0.050 m/s takes 4 s — deliberately the same as the set-up timeout. Of that travel the first 20 mm closes the geometric clearance, and the remaining 180 mm is the endpoint against which the blocked-load design scale is defined. With $a = 0^\circ$, $b = 5^\circ$, and $K_{p,0} = \text{diag}(k_x, k_y, k_z)$, solving

$$k_{p,\text{eff}}(n_s) = k_x \sin^2 b + k_z \cos^2 b$$

gives

$$k_z = \frac{k_{p,\text{eff}}(n_s) - k_x \sin^2(5^\circ)}{\cos^2(5^\circ)} = 348.6 \text{ N/m} \longrightarrow 350 \text{ N/m}. \quad (4.15)$$

Rounded, the implemented block is $\text{diag}(2000, 2000, 350) \text{ N/m}$ in base XYZ. The same matrix in the active surface frame reads

$$R_{\text{surface}}^\top K_{p,0} R_{\text{surface}} \approx \begin{bmatrix} 1987.47 & 0 & 143.26 \\ 0 & 2000 & 0 \\ 143.26 & 0 & 362.53 \end{bmatrix} \text{ N/m}. \quad (4.16)$$

A completely blocked 0.180 m normal reference change then works out at roughly 65.3 N along the normal, 25.8 N along t_1 , and 70.2 N of total elastic-force magnitude. All three assume the tracking error never closes, whereas the robot follows part of the reference, so they are upper bounds on the virtual spring rather than predicted loads.

The rotational block starts from a 15 N design force acting through the 60 mm tool half-length, a moment scale of about 0.90 N m. Allowing roughly 10° about the two surface tangents but only 1° about the surface normal, and converting to radians,

$$k_{R,t_1} = k_{R,t_2} \approx \frac{0.90}{0.1745} = 5.16 \longrightarrow 5, \quad k_{R,n} \approx \frac{0.90}{0.01745} = 51.6 \longrightarrow 50 \text{ N m/rad}. \quad (4.17)$$

The selected ratio provides lower rotational stiffness about the surface tangents than about the surface normal. During set-up these are source gains at the selected centre of compliance. Once the congruence shift has been applied they are, in general, no longer

the diagonal rotational slopes a pure rotation at the TCP would meet. Grind applies the same source blocks without a point shift.

Table 4.3: Phase-specific stiffness and design points.

Phase / mode	Source stiffness and frame	Design-point interpretation and command structure
Hold	$K_p = 300I_3$ N/m and $K_R = 40I_3$ N m/rad, base	3 N at 10 mm; 0.70 N m at 1°; decoupled.
Approach orient and descend	$K_p = 150I_3$ N/m and $K_R = 90I_3$ N m/rad, surface definition	3 N at 20 mm; 3.15 N m at the 0.035 rad orientation threshold; decoupled.
Pre-set-up gate	$K_p = 5000I_3$ N/m, base; retains $K_R = 90I_3$ N m/rad	25 N at 5 mm; only translation is overridden; decoupled.
Set-up	$k_p = [2000, 2000, 350]$ N/m, base XYZ; $k_R = [5, 5, 50]$ N m/rad, surface $[t_1, t_2, n_s]$	60 N per blocked 30 mm t_2 stroke and approximately 65 N per blocked 180 mm normal reference; source blocks are shifted through the centre of compliance.
Grind	same K_p and K_R source blocks as set-up	Same design scales; the command is decoupled and the active sweep is along t_2 .
Manual guidance	no Cartesian stiffness	Coriolis compensation plus 0.5 N m s/rad joint damping.

The optional pre-grind gate stayed disabled for every reported run. Enabling it holds the sequence in set-up and continues the coupled pressed command; the 5000 N/m pre-set-up lock is a different gate and is not engaged. Null-space mode 3 adds a projected joint torque on top of all of this, independently of the Cartesian stiffness map.

Inertia-scaled damping. Cartesian damping was calculated at entry to each phase group using eqs. (3.9) and (3.10). The Cartesian inertia was first expressed in the coordinate frame of the corresponding gain block. Some phases used the damping ratio ζ directly. For the hold and pre-set-up gate, a common damping ratio was derived from the fixed damping matrix listed in table 4.4.

Table 4.4: Phase-specific inertia-scaled-damping policy.

Phase / mode	Inertia/gain frame	Ratio or derived target	Manual fallback
Approach orient and descend	surface for translation and rotation	$\zeta = 1.9$, computed once and shared	$D_p = [20, 20, 20]$; $D_R = [12, 12, 12]$
Pre-set-up gate	base translation; surface rotation	separate damping ratios derived from $D_p = 200I_3$ and the cached approach D_R	$D_p = 200I_3$; inherited cached or manual approach D_R
Set-up and grind	base translation; surface rotation	$\zeta = 1.0$, computed after clearance and shared	$D_p = [10, 10, 175]$; $D_R = [10.01, 10.01, 10]$
Hold	base for translation and rotation	one damping ratio derived from $D_p = 50I_3$, then also applied to rotation	$D_p = 50I_3$; $D_R = 8I_3$
Manual guidance	not applicable	no Cartesian inertia-scaled damping	0.5 N m s/rad joint damping

Only finite, positive directional inertia values were accepted. The fixed phase-specific matrix in the final column of table 4.4 supplied the damping whenever the inertia-based value was unavailable.

Two things limit eq. (3.10). It reads only the diagonal of the TCP task-inertia estimate and drops the off-diagonal coupling, and its ζ is a dimensionless tuning factor rather than an identified modal damping ratio. During set-up the source damping blocks are calculated at the TCP and then shifted through the centre of compliance by the same congruence as the stiffness. Symmetry and positive semidefiniteness survive that shift. Critical damping of the compliance-centre/contact system does not follow from $\zeta = 1$.

4.4.2 Compliance-Centre Gain Design

GREEN — sufficient. The active mixed-frame source gains, constant reference, stored lever convention, surface-frame resolution, and moment sign check are all explicit.

Only `set_up` applies the compliance-centre construction. A rotation into the surface frame changes the principal directions of the translational and rotational blocks but leaves the

translation–rotation blocks equal to zero. The centre of compliance consequently remains at the TCP unless the point shift of eqs. (2.52), (2.54) and (2.55) is applied. The signs and the complete block-matrix derivation are given in section 2.7.

The active source stiffness, written wholly in base-frame coordinates, is

$$K_{c,0} = \text{blkdiag}\left(\text{diag}(2000, 2000, 350), R_{\text{surface}} \text{diag}(5, 5, 50) R_{\text{surface}}^\top\right). \quad (4.18)$$

Its translational entries carry N/m and its rotational entries N m/rad. Written out this way the mixed-frame construction shows plainly: translation diagonal in base XYZ, rotation alone diagonal in surface coordinates $[t_1, t_2, n_s]$.

This transformation changes only the gain matrices; it introduces neither an additional wrench nor a time-varying task-space rotation. Its experimental effect was evaluated from the EE-inferred tool-axis alignment, the selected feature and TCP displacements, and the commanded joint torques.

Choosing the centre of compliance. The implementation accepts an arbitrary base-frame offset from the projected leading-feature reference captured at clearance:

$$p_c = p_{\Pi, \text{clr}} + \Delta p_c^0, \quad \Delta p_c^0 = \begin{bmatrix} -0.04 & 0.08 & 0 \end{bmatrix}^\top \text{ m}. \quad (4.19)$$

Its superscript 0 is a reminder that the three components are base XYZ and not $[t_1, t_2, n_s]$. The reference is held constant for the run. A scalar choice $p_c = p_s + h n_s$ is a theoretical special case that the active parameterisation does not use. Holding the centre of compliance constant from first contact holds the lever constant too,

$$r_c = p_{\text{TCP}, \text{clr}} - p_c,$$

where $p_{\text{TCP}, \text{clr}}$ is the TCP position captured at the same clearance hand-off. A different Δp_c^0 means a different coupled pair $K_{\text{TCP}}, D_{\text{TCP}}$ out of eqs. (2.54) and (2.55), so the sign and the frame of the vector are stored with every trial. Neither can be recovered from the alignment record afterwards.

Resolved in surface coordinates on the configured $a = 0^\circ$, $b = 5^\circ$ surface, the base-XYZ

offset becomes

$$R_{\text{surface}}^{\top} \Delta p_c^0 \simeq \begin{bmatrix} -39.85 & 80.00 & -3.49 \end{bmatrix}^{\top} \text{ mm}. \quad (4.20)$$

The centre of compliance therefore lies mainly in the $-t_1, +t_2$ quadrant relative to the projected selected feature. The offset is not itself the lever the adjoint uses; the exact lever picks up the captured TCP-to-feature geometry as well. Writing $f = K_{p,0} e_p$ for a purely translational error, the moment the coupling block produces gives a direct sign check,

$$m_c = [r_c]_{\times}^{\top} f = -r_c \times f. \quad (4.21)$$

At the nominal aligned geometry the transverse lever is approximately $0.03985t_1 - 0.08t_2$, and a blocked press with $f \approx -Fn_s$, $F > 0$, gives $m_c \approx -F(0.08t_1 + 0.03985t_2)$. The selected quadrant thus biases negative rotation about both surface tangents, and reversing either transverse offset reverses the corresponding bias. This relation determined the selected direction. Because the translational stiffness is diagonal in the base frame, the commanded force need not be exactly normal to the surface. The bias assists alignment only where its signed components oppose the residual orientation error, which was evaluated from the measured physical-plane alignment.

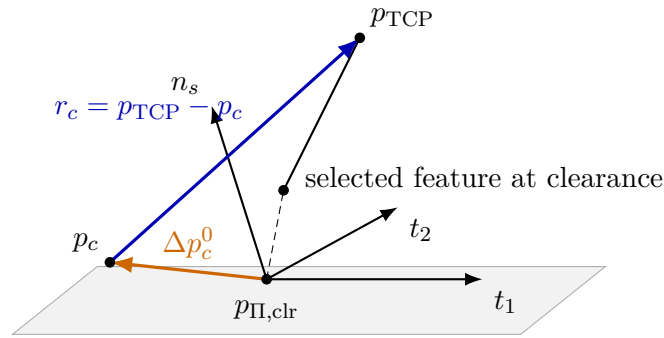


Figure 4.2: Compliance-centre geometry during set-up.

The quantity p_c denotes a local linearised centre of compliance rather than a physical hinge. The tool need not pivot about it during contact, and the relation between the two is affected by four factors: the finite rotational stiffness, the lever itself, friction and constraints at the surface, and the active null-space sigma torque.

4.4.3 Gain Construction and Implementation Sequence

RED — machine-like prose pattern. The numbered workflow and arrow-chain equation restate the preceding gain construction in a generic process template. They add little technical evidence and can be removed without breaking the methodology.

Calculation and implementation follow the controller's own order, so the five steps below are applied in the sequence given:

1. State a phase-specific force–error or moment–angle design pair and calculate the principal source stiffness using eq. (4.11).
2. Attach each diagonal to its implemented frame, form the base-frame block, and check the commanded path with eq. (4.13). For set-up, the base-XYZ translation and surface-frame rotation blocks remain distinct.
3. At phase-group entry, calculate and cache the damping from the current TCP task inertia using eqs. (3.9) and (3.10).
4. During set-up only, shift the selected source stiffness and damping through the centre of compliance using eqs. (2.54) and (2.55). Grind immediately returns to the decoupled source blocks.
5. Check the resulting elastic load and joint-torque scales, then change one declared gain family at a time while retaining the frame labels, computed damping, compliance-centre offset, and remaining parameters with the run record.

The resulting workflow is

$$\begin{aligned}
 &\text{phase design envelope} \rightarrow \text{frame-labelled source } K \rightarrow \text{task-inertia } D \\
 &\quad \rightarrow \text{set-up-only compliance-centre shift} \\
 &\quad \rightarrow \text{load/torque check} \rightarrow \text{controlled trial.}
 \end{aligned} \tag{4.22}$$

4.4.4 Experimental Gain Variation Strategy

GREEN — sufficient. The one-factor comparisons, held quantities, tested levels, interaction corner, repetition count, and interpretation rule are stated clearly.

The effect of each stiffness entry was investigated by varying one parameter while the remaining controller settings were held fixed. This one-factor-at-a-time procedure separates

the response associated with the selected gain from changes caused by other controller parameters, which the paired sweep of the earlier campaign could not do.

In Case B the rotational stiffness of the tilted surface tangent was tested at 5, 15 and 50 N m/rad. When K_{R,t_1} was varied, K_{R,t_2} was held at its baseline value, and the roles were reversed for the t_2 study. This separates the two tangent responses. The normal rotational stiffness $K_{R,n}$ remained at its baseline value because it resists twist about the surface normal rather than tipping about a tangent.

In Case C the translational stiffness perpendicular to the tilted tangent was tested at 300, 800 and 2000 N/m, and one further condition combined the lowest translational stiffness with the highest tested rotational stiffness to examine their interaction. The other source gains, the compliance-centre offset, the phase timing, and the null-space parameters were held constant. A systematic change in the measured response could therefore be associated with the stiffness coordinate that was varied.

Damping needed no separate tuning. At the next entry to the relevant phase group the controller recalculated it from the changed stiffness and the task inertia. Because eq. (3.10) takes the square root of the stiffness, scaling an entry by α scales its damping coefficient by $\sqrt{\alpha}$,

$$d_{i,\text{test}} = \sqrt{\alpha} d_{i,\text{base}}, \quad (4.23)$$

so doubling a stiffness raises the damping by about 41 % rather than by 100 %. This is an algebraic consequence of the damping law and not a measured result. It holds while the Cartesian inertia $\Lambda_{F,ii}$ and the factor ζ are unchanged and the coefficient stays between the configured floor and ceiling; since the inertia depends on the pose at phase-group entry, the realised coefficients were recorded for every run. The value is cached at phase-group entry and held for the rest of that phase, and the stiffness alone does not reconstruct it without the phase-entry pose, the Jacobian J and the joint-space mass matrix $M(q)$.

Approach, gate and hold gains follow the same one-at-a-time rule against the design pairs of table 4.3. Where a translational entry is assigned from a desired effective normal stiffness, the base- z value follows eq. (4.15), which carries the surface orientation into the assignment.

Evaluation of candidate gains. Candidate stiffnesses were compared through the quantities of section 4.6, with the physical-plane alignment improvement $\Delta\theta_{\text{align,phys}}$ as the primary outcome. The rotational-stiffness study of section 5.4 was evaluated using the alignment improvement, the physical-plane residual angle and the model-estimated normal load. Contact-induced rotation and the displacements of the selected feature and TCP were used to check that the compared runs followed the same contact motion. Each setting was repeated three times and summarised by its mean and standard deviation. A change was interpreted as a parameter effect only when it exceeded the interpretation threshold of section 5.1 and was not accompanied by an incompatible change in contact load or displacement.

4.4.5 Null-Space Settings

GREEN — sufficient. The contact and hold configurations are reconciled in one table and routed to the common theoretical definitions.

The primary contact campaign used the combined null-space mode. The internally commanded hold study used the same projector and probe settings while separating the two torque terms. The corresponding values are listed in table 4.5.

Table 4.5: Null-space settings in the two campaigns.

Quantity	Primary contact campaign	Internally commanded hold study
Control mode	damping and sigma	off, damping only, or sigma only
d_{null}	2 N m s/rad	0 or 2 N m s/rad
k_{σ}	1 N m	0, 1.5, or 2 N m
α_{probe}	0.08 rad	0.08 rad
δ_{σ}	10^{-6}	10^{-6}
ρ	10^{-4}	10^{-4}

Projected damping opposes $N_{\tau}\dot{q}$, while the conditioning torque acts along the selected v_7 direction. Their definitions are given in section 2.8. The contact results describe the combined controller; the internally commanded hold results provide the separated comparison.

4.4.6 Internally Commanded Null-Space Disturbance Study

GREY — logical or evidential gap. The internal point-force equivalent is repeatable and the acceptance rule is explicit, but it does not validate a physical external disturbance, link contact, or the combined null-space mode.

Projected damping and the singular-value bias were isolated in the Cartesian hold mode without surface contact. A repeatable internal disturbance represented the joint-torque equivalent of a point force. It was neither a physical external force nor a force-sensor measurement.

The force point was fixed to link 3 at the link-frame offset $r_p = [0, 0, 0.200]^\top$ m. At the initial pose, the structural null direction $v_7(q_0)$ and the translational point Jacobian $J_p(q_0)$ set the base-frame unit-force direction

$$u_f = \frac{J_p(q_0)v_7(q_0)}{\|J_p(q_0)v_7(q_0)\|_2}. \quad (4.24)$$

This direction maximises the virtual work along the initial structural null direction among unit point forces and was held fixed for the complete trial. The controller evaluated the current point Jacobian and applied

$$\tau_{\text{dist}}(t) = J_p(q(t))^\top f_d(t), \quad f_d(t) = 20s(t)u_f \text{ N}, \quad (4.25)$$

where $s(t)$ rose from zero to one through a half-cosine between 5 s and 7 s, remained at one until 8 s, and returned to zero through a one-second half-cosine. A 3 N m bound was imposed on $\|\tau_{\text{dist}}\|_2$.

Twelve main trials were acquired. Three repetitions were recorded without a null-space torque. A further three used projected damping of 2 N m s/rad. Sigma-only trials used $k_\sigma = 1.5$ and 2 N m, with three repetitions per setting. The predefined acceptance limit was a peak Cartesian position error of 2 mm. Every trial remained within this limit and was included in the comparison.

The disturbance response was measured by the cumulative projected joint excursion

$$E_N = \int_{5\text{s}}^{9\text{s}} \|N_v(q)\dot{q}\|_2 dt. \quad (4.26)$$

For comparison with the other angular results, the cumulative excursion is reported in degrees.

The conditioning response was evaluated from the final change in $\sigma_{\min}$, while Cartesian task retention was represented by the peak position-error norm over the hold.

4.5 Experimental Procedure

GREEN — sufficient. The procedure states what occurred in each run, distinguishes geometric hand-offs from contact detection, and carries the active values and stopping conditions needed for replication.

Every run followed the sequence below, in this order:

1. The active parameter set is recorded, and the Panda moves to the configured initial joint pose. Optionally it is repositioned under damped manual guidance, and the selected start pose is recaptured.
2. The configured gripper action is executed: in the current configuration a grasp at 70 mm width and 60 N rather than an open command. The tool is seated as consistently as its approximately $\pm 2^\circ$ rotational play about y_{EE} allows, and that non-rigid mounting condition is recorded with the run metadata.
3. The known plane and $R_{\text{surface}} = [t_1, t_2, n_s]$ are constructed, together with the 40×120 mm tool face and the active base-XYZ compliance-centre offset. The Franka collision behaviour is configured before torque control starts.
4. In `approach_orient` the TCP position is held while the physical $+Z_{EE}$ axis rotates toward $-n_s$. The phase continues only after at least 0.5 s and a tool-axis error no greater than 0.035 rad.
5. In `approach_descend` the reference moves along $-n_s$ at 0.1 m/s, with a maximum commanded travel of 0.640 m, and the leading face, edge or corner is selected continuously. The transition occurs when the exact most-leading corner height is at most 20 mm above the plane; the averaged tied feature supplies the projected reference and can differ by at most the 0.1 mm tie tolerance. Estimated force does not trigger this transition. The enabled pre-set-up gate holds the reached pose until the operator presses Enter.

6. At this clearance hand-off the selected tool feature, TCP pose, orientation reference, external-wrench baseline and projected leading-feature surface reference are selected and held constant. This transition is defined geometrically and does not indicate force-detected contact.
7. In `set_up` the signed push coordinate ramps continuously from its captured value, approximately -0.020 m, toward $+0.180$ m at 0.050 m/s along $-n_s$, under the coupled compliance-centre 6×6 stiffness and damping. The approximately 0.200 m coordinate span takes 4 s, matching the timeout; the final $+0.180$ m coordinate is a virtual post-plane spring reference and not a claim of 180 mm physical penetration. After 2 s, a 150 N m change in the model-estimated external-moment norm may end the phase. This is a permissive placeholder rather than a calibrated contact criterion, so unless such a large change occurs the independent 4 s timeout governs the transition.
8. In `grind` the final set-up preload is retained and a quintic smooth ping-pong sweep of 0.03 m amplitude at 0.2 Hz is commanded along t_2 . In the callback that satisfies the set-up exit condition, the controller switches immediately, without gain interpolation, to the decoupled set-up gain blocks; the point-shifted branch is no longer active. A wrench and $J^\top w_{\text{cmd}}$ step is therefore possible at this hand-off.
9. The run stops with `e+Enter`, the buffered log is persisted, and the EE-inferred tool-axis error, edge and TCP displacement, commanded joint-torque trace and phase timing are evaluated.

Between repetitions of a controlled comparison, only the declared independent variable was changed. Runs were pooled only when the controller configuration, tool geometry, phase duration, and fixed gains matched; all other runs were treated as separate configurations. The resulting comparison identifies the largest measured alignment among the tested compliance-centre settings for this tool, plane, and contact sequence. It does not establish a universally optimal stiffness matrix.

4.6 Evaluation Metrics

GREY — logical or evidential gap. The equations are clear, but the primary angle depends on a nominal tool-to-EE transform while the tool can rotate in the gripper. The physical grinding-face angle is therefore not independently observed.

The primary angle and the controller rotation error describe different quantities. The nominal grinding-face axis inferred from the end-effector pose is

$$a_T = R_{EE}a_{EE}.$$

Its angle to the independently calibrated physical plane is

$$\theta_{\text{align,phys}} = \cos^{-1}(-a_T^\top n_{\text{phys}}). \quad (4.27)$$

Both a_T and n_{phys} are unit vectors, so the argument lies in $[-1, 1]$ and the inverse cosine is defined for every attainable pose. The minus sign follows from the tool-axis convention in eq. (4.6). The value is inferred from the end-effector pose. Rotation of the tool within the gripper is therefore part of the measurement uncertainty rather than a separate state in the calculation. The corresponding angle to the configured normal n_s is used by the controller, but the physical-plane angle above is used for the reported results.

The physical-plane alignment improvement is

$$\Delta\theta_{\text{align,phys}} = \theta_{\text{align,phys,before}} - \theta_{\text{align,phys,after}}. \quad (4.28)$$

A positive value means that alignment improved. Because the tool mount carries roughly 2° of rotational play, the metric describes the combined robot–gripper–tool response. Contact-induced rotation is reported separately as the finite before–after rotation vector

$$\Delta\theta_C = \text{Log}(R_{\text{after}}R_{\text{before}}^\top)^\vee. \quad (4.29)$$

Displacement of the selected physical feature is

$$\Delta p_C = p_{C,\text{after}} - p_{C,\text{before}}, \quad s_C = \|\Delta p_C\|. \quad (4.30)$$

TCP displacement is reported in the same way. These quantities show whether an angular improvement is accompanied by translation or sliding.

The controller error e_R is measured from the orientation held at the clearance transition. It is therefore not the same as the physical-plane alignment angle and is not used as the primary outcome.

4.6.1 Metric Use by Experimental Case

RED — machine-like prose pattern. The bold metric catalogue and following case table repeat the same information in two uniform forms. This templated duplication reads as machine-organised rather than as necessary experimental reasoning.

Physical-plane alignment improvement is the primary response in all five cases. What separates them is the independent variable stored with each run, and the supporting quantities carried alongside that primary outcome.

Primary—physical-plane alignment improvement. $\Delta\theta_{\text{align,phys}}$, with a positive value denoting improvement.

Physical-plane residual. $\theta_{\text{align,phys,after}}$, reported as a range when the physical-plane calibration uncertainty affects its absolute value.

Contact-induced rotation. The magnitude and direction of the finite before–after rotation $\Delta\theta_C$.

Contact displacement. Selected-feature and TCP displacement during set-up.

Model-estimated normal load. The steady model-estimated normal load, explicitly distinguished from a direct force measurement.

Settling check. The residual changes in tip angle, configured-normal angle, and estimated load after the selected evaluation time.

Table 4.6: Metric use by experimental case.

Case	Quantities used	Role in the evaluation
A	Physical-plane alignment before and after set-up, its signed t_1 and t_2 components, and the steady model-estimated normal load	Establish the response to the commanded tilt and the baseline for the remaining cases.
B	The Case-A quantities against the rotational stiffness of the tilted tangent	Quantify the separate K_{R,t_1} and K_{R,t_2} contributions.
C	The Case-A quantities against the translational stiffness perpendicular to the tilted tangent	Quantify the translational contribution and the stiffness interaction.
D	The Case-A quantities against the surface-frame compliance-centre coordinates, with contact-induced rotation and feature displacement	Quantify the response to lever placement, including its sign.
E	Alignment improvement for tilt about y_{EE} and x_{EE} , compared with the Case-A tangent-axis tilts	Attribute the tangent-axis asymmetry to the surface frame or to the tool face.

Commanded tilt, stiffness entry, and compliance-centre lever were stored with each run as independent variables rather than evaluated as metrics. The primary alignment outcome was interpreted together with displacement and estimated load. Commanded torque, oscillation scores, and stability scores were excluded from the reported outcomes.

4.7 Safety Limits

GREY — logical or evidential gap. The reflex thresholds and procedural guards are documented, but they are not command saturations and do not establish a measured safety margin. The large moment threshold is acknowledged as a permissive placeholder.

Phase-specific damping comes from the task inertia through eqs. (3.9) and (3.10). This directional calculation does not establish contact safety. The implemented controller remains discrete, model dependent, and coupled to a stiff environment [18].

Ahead of the torque callback the reported configuration calls `setCollisionBehavior` with equal joint-side acceleration and nominal thresholds of 140 N m, and equal Cartesian force and moment thresholds of 240 N or N m across all six components. These values specify the thresholds at which the Franka reflex is triggered; they do not saturate the commanded torque or wrench. The user callback applies no additional Cartesian-force, moment, joint-torque, or torque-rate limit. The values document the robot configuration used in the experiments and do not constitute a claim of human safety.

Procedure and geometry supply the remaining guards:

- a 0.640 m maximum approach travel and a geometric 20 mm clearance hand-off;
- an operator gate before set-up, using a 5000 N/m isotropic position hold with calculated inertia-scaled damping coefficients;
- a 4 s set-up timeout independent of the permissive 150 N m external-moment-change stopping threshold; the timeout is expected to govern unless the estimated moment changes exceptionally;
- the configured set-up stiffness and push must be checked against the collision settings before a run; the scalar product $350 \text{ N/m} \times 0.180 \text{ m} \approx 63 \text{ N}$ is only a base- z approximation, while the frame-consistent blocked-reference scales are 65.3 N normal and 70.2 N in total from eq. (4.16); none is a contact-force measurement.

The 150 N m set-up threshold applies to the change in libfranka’s model-estimated external moment. It is not a measured contact threshold. In the reported runs, set-up ended at the configured timeout rather than at this threshold.

Chapter 5

Results and Discussion

All contact runs completed set-up and were retained for analysis. Each reported mean is based on three repetitions of the corresponding setting. One repetition differed substantially from the other two and was retained so that the reported dispersion represents the complete measured response. It is discussed in section 5.10.

The case identifiers introduced in table 4.1 are used throughout this chapter. Case A provides the baseline for the remaining four cases.

5.1 Repeatability and Settling

GREY — logical or evidential gap. The repeatability summary is useful, but the approximately 0.1° interpretation threshold is informal, based on three repetitions per setting, and coexists with one 0.60° late-settling change.

Repeatability comes from the three repetitions of each of the 25 settings. Across settings tilted about t_1 the median within-setting standard deviation of the alignment improvement is 0.02° ; about t_2 it is 0.03° . Changes below approximately 0.1° were interpreted as scatter rather than as parameter effects. This threshold is applied to every comparison in the chapter.

The reported values were calculated from settled intervals. Over the last second of set-up the model-estimated load moved by a median of 0.10 N and the tip angle by a median of 0.00° ; the worst run gave 1.10 N and 0.60° . Preload does not confound the comparisons either. Wherever the lever kept its baseline placement the steady model-estimated normal

load stayed between 44.5 and 49.4 N.

5.2 EE-Inferred Alignment Against the Physical Plane

GREY — logical or evidential gap. The measurement chain is stated honestly, but tool motion in the gripper remains unobserved. The result cannot separate controller rotation from tool-mount rotation.

The primary response is the change, over the set-up interval, in the angle between the EE-inferred tool axis and the calibrated physical plane; a positive value means the angle closed. Approximately 2° of rotational freedom about y_{EE} occurs at the gripper-tool interface, so the quantity is a combined robot-gripper-tool response and leaves controller compliance and gripper compliance unseparated.

Commanded and measured tilts differed, and the discrepancy increased with the command. The transition from orientation to descent occurred once the EE-inferred tool axis was within 2° of the commanded direction. The remaining deviation increased with the commanded angle: a 5° tilt about t_1 produced a first-contact angle of 2.45° , whereas a 10° tilt about t_2 produced 8.45° . Every subsequent comparison uses the measured first-contact angle.

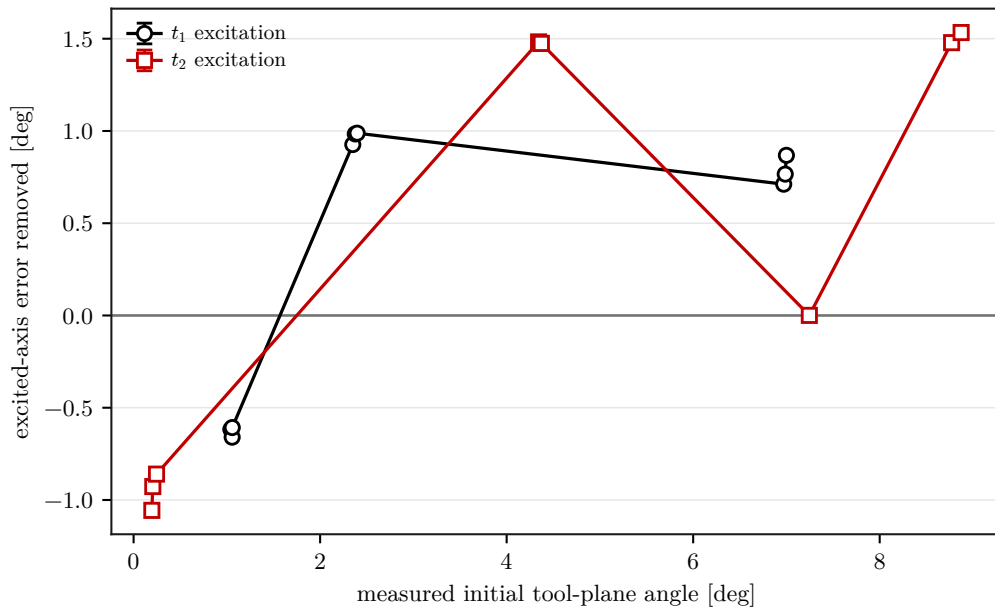
5.3 Case A: Commanded Tilt

GREY — logical or evidential gap. The baseline is reported transparently, but the systematic negative response at zero tilt is not explained or isolated. It remains a confounding mechanism for the tilted comparisons.

The zero-tilt condition was used to quantify the orientation change introduced by the set-up procedure in the absence of a commanded angular offset. The EE-inferred misalignment increased from 1.07° to 2.04° , corresponding to an alignment change of -0.97° . This indicates that set-up itself introduced a systematic orientation change in this condition. The tilted results were therefore interpreted together with, rather than independently of, the zero-tilt response. The zero-tilt value was not subtracted from the values reported in table 5.1.

Table 5.1: Case-A alignment response (mean $\pm$ standard deviation, $n = 3$).

Commanded tilt	Before (deg)	After (deg)	Improvement (deg)
None	1.07 ± 0.00	2.04 ± 0.07	-0.97 ± 0.07
+5° about t_1	2.45 ± 0.02	1.51 ± 0.02	$+0.94 \pm 0.03$
+10° about t_1	6.99 ± 0.01	6.35 ± 0.07	$+0.64 \pm 0.08$
+5° about t_2	4.40 ± 0.02	2.88 ± 0.02	$+1.52 \pm 0.00$
+10° about t_2	8.45 ± 0.96	7.39 ± 0.05	$+1.06 \pm 0.92$

**Figure 5.1:** Case-A alignment response by measured initial angle.

Two observations hold across the case. Tilting the tool about t_2 recovered more angle than tilting it about t_1 at both commanded magnitudes, 1.52° against 0.94° at 5° and 1.06° against 0.64° at 10° . The second observation is visible in fig. 5.1: the larger the commanded angle, the smaller the fraction of it recovered. That fraction fell from 38 % at 5° about t_1 to 9 % at 10° about the same axis, which indicates that over this range the correction is not proportional to the error that provokes it.

The 10° t_2 condition showed substantially greater variability than the other settings. Its standard deviation of 0.92° is an order of magnitude above every other entry in table 5.1, and one repetition accounts for it: that run produced 0.00° , whereas the other two repetitions under the same configuration produced 1.56° and 1.62° . The run is retained in the mean. Its possible relation to the tool mount is discussed in section 5.10.

5.4 Case B: Axis-Specific Rotational Stiffness

GREEN — sufficient. The reused baseline is identified, the t_1 result is bounded to the tested range, and the anomalous t_2 repetition is handled without forcing a monotonic conclusion.

In Case B the rotational stiffness of the tilted tangent was raised alone, and the orthogonal entry was held at 5 N m/rad. The 5 N m/rad column of table 5.2 is not a separate experiment: those entries are the 10° Case-A settings, reused as the reference.

Table 5.2: Case-B alignment improvement by tilted-axis stiffness (deg).

Tilted axis	5 N m/rad	15 N m/rad	50 N m/rad
t_1	$+0.64 \pm 0.08$	$+0.57 \pm 0.03$	$+0.49 \pm 0.02$
t_2	$+1.06 \pm 0.92$	$+1.58 \pm 0.04$	$+1.14 \pm 0.03$

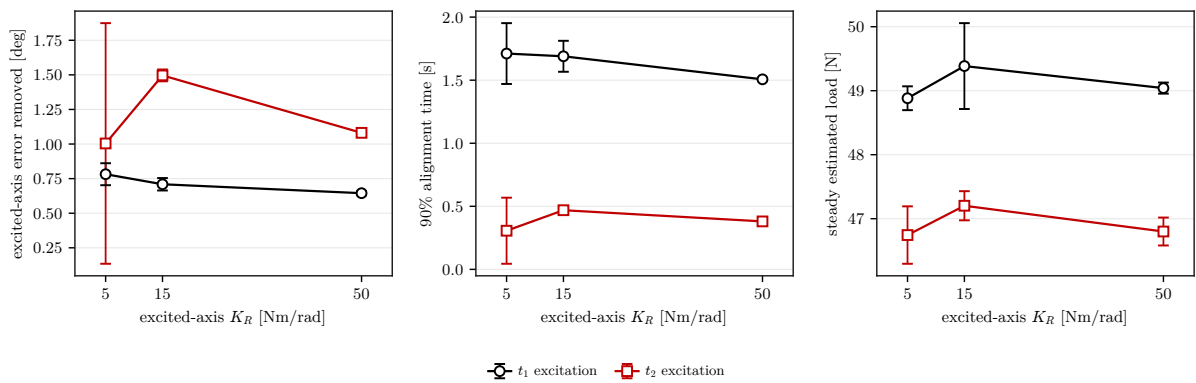


Figure 5.2: Case-B response to tilted-axis rotational stiffness.

For the t_1 tests, the alignment improvement decreased monotonically from 0.64° at 5 N m/rad to 0.57° at 15 N m/rad and 0.49° at 50 N m/rad, as table 5.2 shows. Increased rotational stiffness therefore reduced the measured correction about t_1 within the tested range.

The t_2 result requires a more cautious interpretation. The mean at 5 N m/rad was reduced by the repetition that produced no measured improvement. The other two repetitions averaged approximately 1.59°, which is nearly equal to the 1.58° measured at 15 N m/rad. The 50 N m/rad setting produced 1.14°. The data therefore support a reduction at the highest tested stiffness, but they do not establish a monotonic ordering between the two softer t_2 settings.

Because the two tangent entries were varied one at a time, this finding is axis-specific and does not characterise a common paired value.

5.5 Case C: Cross-Axis Translational Stiffness

GREY — logical or evidential gap. The main stiffness trend is reported clearly, but the claimed interaction is inferred from only one combined low-translation/high-rotation corner per axis. A factorial comparison is missing.

In Case C the translational stiffness perpendicular to the tilted tangent was varied, again against the 10° Case-A settings as the 2000 N/m reference. One further corner per axis was measured, in which the softest translation, $K_p = 300$ N/m, was paired with the stiffest rotation, $K_R = 50$ N m/rad; it is shown as the interaction column of table 5.3.

Table 5.3: Case-C alignment improvement by cross-axis stiffness (deg).

Tilted axis	300 N/m	800 N/m	2000 N/m	Interaction
t_1	$+0.58 \pm 0.02$	$+0.52 \pm 0.01$	$+0.64 \pm 0.08$	$+0.50 \pm 0.02$
t_2	$+1.79 \pm 0.09$	$+1.57 \pm 0.00$	$+1.06 \pm 0.92$	$+1.20 \pm 0.04$

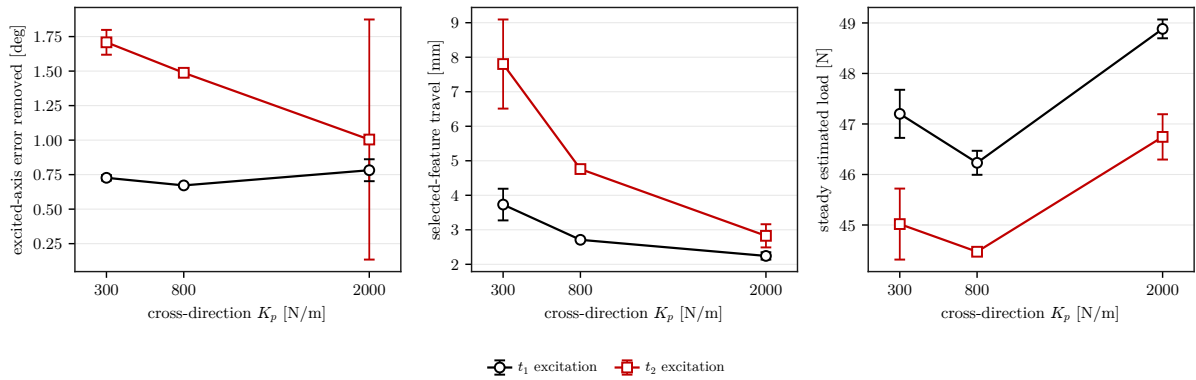


Figure 5.3: Case-C response to cross-axis translational stiffness.

Reducing the perpendicular translational stiffness increased the response about t_2 only. With the tool tilted about t_2 , the improvement rose as K_p fell, reaching 1.79° at 300 N/m, the largest value recorded anywhere with the compliance centre at its baseline placement. About t_1 the entire range moved the response by less than 0.15° , close enough to the interpretation threshold to be treated as no effect.

The combined setting did not reproduce the improvement obtained with low translational stiffness alone. It returned 1.20° about t_2 , well short of the 1.79° reached at the same K_p with $K_R = 5 \text{ N m/rad}$, as the interaction column of table 5.3 shows. This indicates an interaction between the two tested stiffness entries over this range.

5.6 Case D: Virtual Centre of Compliance

GREEN — sufficient. This is the strongest results section. It gives the measured effect, relation to zero lever, sign reversal between axes, load comparison, axial-offset null result, mechanism, and tested-range limitation.

In Case D every gain was held at its baseline value and only the lever $r_c = p_{\text{TCP}} - p_c$ was varied. The lever was directed along the surface tangent perpendicular to the tilted one, which pairs a t_1 tilt with r_{c,t_2} and a t_2 tilt with r_{c,t_1} . In a fourth column the compliance centre was placed on the centre of the grinding face, 20 mm along the tool axis from the end-effector origin and therefore off the tangent plane in which the sweep lies.

Table 5.4: Case-D alignment improvement by perpendicular lever (deg).

Tilted axis	−60 mm	0 mm	+60 mm	Face centre
t_1	$+6.05 \pm 0.14$	$+0.56 \pm 0.01$	$+0.41 \pm 0.01$	$+0.59 \pm 0.03$
t_2	$+0.02 \pm 0.06$	$+1.38 \pm 0.01$	$+6.36 \pm 0.31$	$+1.35 \pm 0.02$

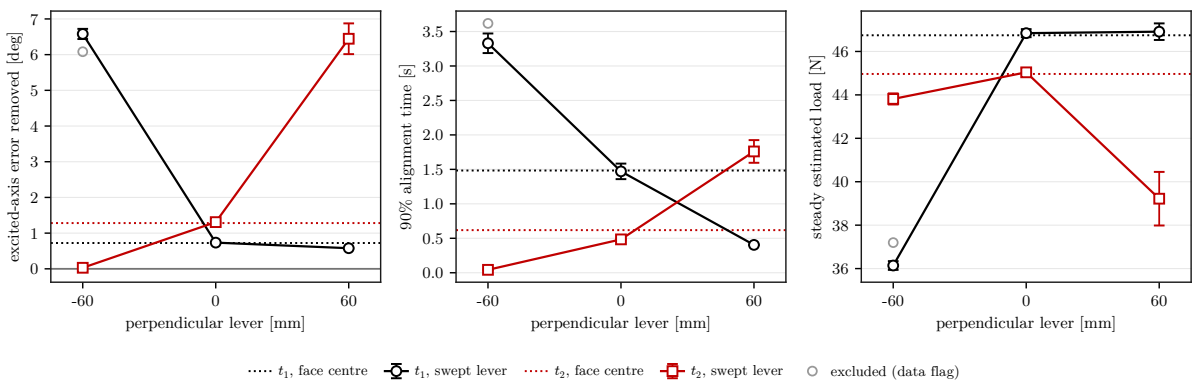


Figure 5.4: Case-D alignment response by compliance-centre lever.

For each tilted axis, one lever direction improved alignment and the opposite direction produced no comparable improvement. The improving direction is termed the *favourable*

direction below. In this direction, the moment generated by the commanded penetration rotates the tool face towards the surface.

With the lever in that direction, the residual misalignment fell from 6.99° to 0.93° with the tool tilted about t_1 , and from 8.50° to 2.14° about t_2 , as table 5.4 shows. These improvements of 6.05° and 6.36° are about ten and five times what the same gains recovered with the lever at zero. Within the tested range they are the largest effect produced by any parameter investigated in this campaign.

The favourable direction is not the same for the two tangents. Correcting a t_1 tilt required a negative lever along t_2 , $r_{c,t_2} = -60$ mm, whereas correcting a t_2 tilt required a positive lever along t_1 , $r_{c,t_1} = +60$ mm, so the favourable lever signs were opposite for the two tilted axes.

Applying $r_{c,t_1} = -60$ mm in the t_2 condition produced only $0.02 \pm 0.06^\circ$, so no improvement above the repeatability scale was measured. The sign of the in-plane lever therefore determined whether the induced moment assisted the corrective rotation.

Contact was also lighter at the two favourable placements, at steady model-estimated normal loads of 36.5 N and 39.2 N against roughly 45 to 47 N at zero lever, with the commanded virtual press unchanged throughout.

The face-centre settings moved the centre 20 mm along the tool axis, from the TCP towards the surface, which places it at the centre of the grinding face. Resolved in the surface frame at the 10° commanded tilt this gives $r_c = (0, -3.5, 19.7)$ mm for the t_1 condition and $(3.5, 0, 19.7)$ mm for t_2 ; the tangential component is not a design choice but $20 \sin 10^\circ$, the projection of an axial offset onto the tangent plane. They produced 0.59° and 1.35° , compared with 0.56° and 1.38° at zero lever, and these differences remained below the interpretation threshold.

This result is consistent with the point-shifted law and requires separate interpretation from the tangential sweep. With the press directed along the surface normal, $f \approx -Fn_s$, the component of r_c along n_s is parallel to f and contributes nothing to $m_c = -r_c \times f$ of eq. (2.56). Only the tangential components of the lever generate the coupling moment. The face-centre setting therefore tests 3.5 mm of effective tangential lever, about a seventeenth of the 60 mm used in the sweep, and its null result is consistent with that magnitude rather than with a property of the normal coordinate itself.

An axial displacement contributes an effective tangential lever of $d \sin \theta$ for a displacement d and a commanded tilt θ . At 10° , an axial displacement of 345 mm would be required to produce the same tangential component as the 60 mm in-plane lever. Axial displacement therefore produces substantially less alignment moment for the investigated geometry.

Within the tested geometry, the measurements therefore support the simpler conclusion that the alignment moment is governed by the tangential component of the lever. A direct 60 mm tangential offset changed the response by several degrees, whereas the 3.5 mm tangential component of the axial offset did not exceed the interpretation threshold.

5.7 Case E: Tilt About the Tool Axes

GREY — logical or evidential gap. The four settings support a consistency check, but the additive superposition model is not independently fitted or validated. The surface-frame attribution remains model-based.

Case E assessed whether the asymmetry between the two tangents was associated with the tool or the surface frame. An effect associated with the grinding-face geometry should remain when the tilt is commanded about the tool axes. The commanded spin places the long axis of the grinding face 25° from t_2 , so a tangent-axis tilt tips the long and the short axis of the face together; in Case E the same 10° was instead commanded about y_{EE} and x_{EE} .

Table 5.5: Case-E improvement by tilt axis.

Tilt axis	Measured initial angle (deg)	Improvement (deg)
t_1	6.99 ± 0.01	$+0.64 \pm 0.08$
t_2	8.45 ± 0.96	$+1.06 \pm 0.92$
y_{EE}	6.81 ± 0.01	$+1.07 \pm 0.03$
x_{EE}	6.78 ± 0.02	$+1.29 \pm 0.02$

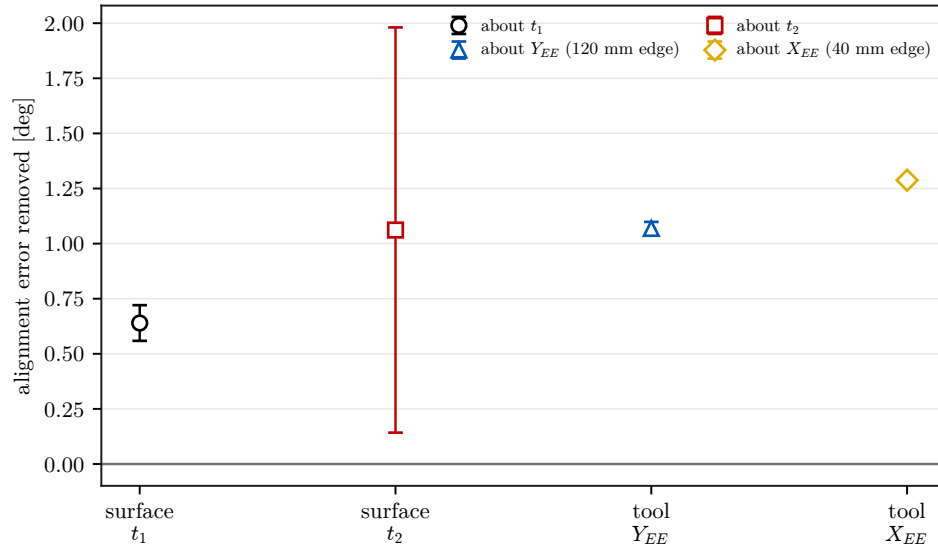


Figure 5.5: Case-E response to surface-tangent and tool-axis tilt.

Both tool-axis results fall between the two tangent-axis ones, as table 5.5 shows, and lie 0.22° apart against the 0.42° that separates the tangent pair. Resolving each tool-axis command onto t_1 and t_2 and adding the matching Case-A improvements predicts 1.25° for x_{EE} and 1.17° for y_{EE} ; the runs gave 1.29° and 1.07° .

The agreement supports, but does not prove, the interpretation that the asymmetry is associated mainly with the surface frame rather than with the geometry of the grinding face. It is a model-based interpretation drawn from four settings, and superposition outside the tested range does not follow from it.

5.8 Internally Commanded Null-Space Response

GREEN — sufficient. The repeated conditions, disturbance magnitude, unclipped waveform, degree-based excursion, conditioning response, task-error criterion, and free-space scope are reported directly.

The commanded disturbance reached 20.00 N in all twelve main trials. Its peak equivalent joint-torque norm lay between 2.17 and 2.45 N m. The torque-limit scale remained one throughout, so every comparison received the requested waveform without clipping. All twelve trials completed and were retained.

Projected damping reduced the cumulative redundant excursion from $(7.596 \pm 0.916)^\circ$ without null-space torque to $(5.687 \pm 0.228)^\circ$ at 2 N m s/rad. This corresponds to a re-

duction of approximately 25 % from the matched zero-torque condition (fig. 5.6a). The final conditioning loss was also smaller, changing from $\Delta\sigma_{\min} = (-2.13 \pm 0.47) \times 10^{-3}$ to $(-1.26 \pm 0.10) \times 10^{-3}$.

Both sigma-only settings produced a small positive final conditioning change. At $k_\sigma = 1.5 \text{ N m}$, the measured value was $(1.95 \pm 0.07) \times 10^{-5}$; at 2 N m , it was $(1.93 \pm 0.03) \times 10^{-5}$. Raising the command therefore increased the cumulative excursion from $(0.283 \pm 0.007)^\circ$ to $(1.619 \pm 0.130)^\circ$ without increasing the retained conditioning change. Mean peak Cartesian position error rose from $0.889 \pm 0.024 \text{ mm}$ to $0.983 \pm 0.053 \text{ mm}$. The largest value across the complete study was 1.304 mm , below the registered 2 mm criterion (fig. 5.6b).

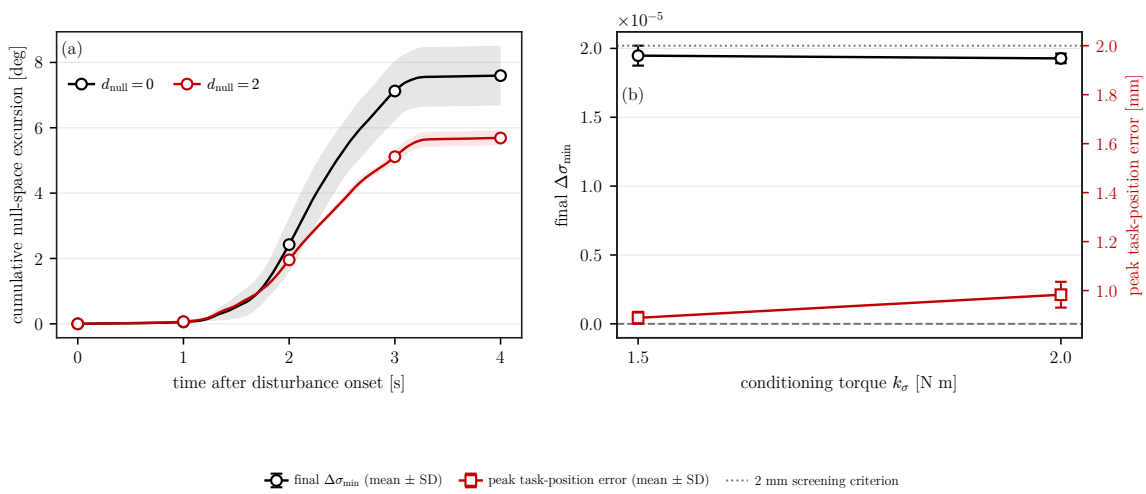


Figure 5.6: Null-space response under the internally commanded point-force equivalent.

The comparison isolates the two null-space terms in free-space hold under an internally commanded point-force equivalent. It characterises this hold condition; physical link contact remains outside its scope.

5.9 Discussion

GREEN — sufficient. The discussion compares effect sizes and then separates the lever mechanism, spatial placement, tuning implications, and limitations.

5.9.1 Relative Influence of the Investigated Parameters

GREEN — sufficient. The cross-parameter ordering is tied to the three result tables and bounded to the investigated configuration and parameter range.

In the investigated configuration, the compliance-centre position had a considerably stronger effect than either the rotational or the translational stiffness. Changing the rotational stiffness of the tilted axis over the tested tenfold range altered the alignment improvement by no more than 0.44° (table 5.2). The largest change caused by the perpendicular translational stiffness, over a roughly sevenfold range, was 0.73° (table 5.3). By comparison, shifting the compliance centre by 60 mm changed the improvement by as much as 5.5° , and with the lever in the unfavourable direction almost no alignment improvement was measured (table 5.4).

These results suggest that the compliance centre, sign included, should be placed before the stiffness entries are refined. Within the investigated configuration and parameter range the normal coordinate of the centre added little once the in-plane placement had been accounted for.

5.9.2 Geometric Interpretation of the Lever

GREY — logical or evidential gap. The cross-product explanation is sound, but the claim about opposite tilt signs and part of the lighter-contact explanation remain model-based because only one commanded tilt sign was measured.

The sign dependence follows from the point-shifted law. Shifting the compliance centre along the tangent perpendicular to the tilted axis converts part of the commanded normal penetration into a moment about that axis, and the sign of the lever sets the direction of that moment: one sign reinforces the rotation that brings the face towards the surface, the other opposes it. Exchanging the roles of t_1 and t_2 reverses the cross product relating the lever to the tilted axis, which is consistent with the favourable direction being opposite for the two tangents.

The lighter contact at the favourable placements is consistent with the same mechanism. A tool that rotates further into conformity takes up part of the commanded penetration itself, so the model-estimated load falls even though the press was never changed.

Two contributions to the commanded wrench therefore respond differently to a change in the sign of the tilt. The rotational spring $K_{R,0}e_R$ is driven by the measured orientation error and reverses with it, so it acts to reduce the misalignment whichever way the tool is tilted. The moment induced by the lever does not. With the press directed into the

surface throughout set-up, the coupling moment $m_c = -r_c \times f$ of eq. (2.56) keeps a direction fixed by r_c alone. The lever is therefore a bias whose sense is chosen in advance, not a feedback term that adapts to the error it is correcting.

The measurements are consistent with that reading. Displacing the compliance centre along the tool axis, a lever nearly parallel to the press and so one that induces little moment, changed the response by less than the interpretation threshold on both tangents. A tangential lever applied against the required rotation instead reduced the measured improvement to 0.02° , compared with 1.38° at zero lever. A placement that induces no moment leaves the response symmetric in the tilt direction; a tangential lever selects one direction of correction and opposes the other.

All commanded tilts had the same sign. The measured asymmetry therefore applies to that sign, while the response to the opposite sign follows only from the model in eq. (2.56).

5.9.3 Placement of the Virtual Centre of Compliance

YELLOW — weak or unclear. The tangential conclusion is well supported. The treatment of the axial coordinate is less clear because only one 20 mm offset was sampled and its effect is interpreted through its tangential projection.

The centre of compliance has three spatial coordinates, of which one was varied systematically in the primary campaign. The measured effect of this coordinate is distinguished below from the remaining untested coordinates.

The coordinate that was swept is the surface tangent perpendicular to the tilted axis, and it determines the size of the correction. Sixty millimetres in the assisting direction raised the improvement from 0.56° to 6.05° about t_1 and from 1.38° to 6.36° about t_2 , while the same distance in the other direction left 0.41° and 0.02° . This coordinate therefore had the largest measured influence on the alignment response.

The coordinate along the tool axis was sampled at a 20 mm displacement towards the surface and produced no change above the interpretation threshold. The point-shifted law accounts for that: while the press stays normal to the surface, the axial part of the lever is parallel to the applied force and generates no moment, so an axial displacement acts only through its projection $d \sin \theta$ onto the tangent plane. Raising the centre above the TCP rather than lowering it towards the face reverses the sign of that projection but

not its magnitude. The axial coordinate therefore had little effect at the tested value.

The conclusions are limited to the investigated coordinate and range. The three tangential levels identify the assisting direction and the size of its effect, but they do not identify an optimum lever length.

5.9.4 Practical Implications for Controller Tuning

GREY — logical or evidential gap. The proposed tuning order is carefully bounded, but lever selection from the sign of the initial tilt has not been validated for negative tilts or for a physical tool-angle measurement.

The measured ordering suggests a tuning sequence. The compliance-centre lever is chosen first, on the tangent perpendicular to the expected tilt and with the sign appropriate to that axis, since it determines the available correction. The rotational stiffness of the tilted axis is then kept as low as the required angular compliance and the allowable displacement permit, because every increase tested returned less angle. The perpendicular translational stiffness is adjusted last, and only where the tool is tilted about t_2 ; about t_1 it did not move the response measurably. This sequence is drawn from one configuration and one parameter range, and is offered as a starting point rather than as a general rule.

That sequence assumes the direction of the initial tilt is known before contact. Where it is not, the bias character of the lever sets a limit on what a single fixed placement can achieve, and two options remain. If the correction must be independent of the tilt direction, the compliance centre is left where it induces no moment, at the TCP or on the tool axis, and the alignment then comes from the rotational spring alone: 0.56° about t_1 and 1.38° about t_2 in this configuration, symmetric in the tilt direction but an order of magnitude below what the favourable lever recovered. If the larger correction is required, one lever direction cannot be retained for every approach. Its direction has to be selected for each run from the sign of the measured initial tilt. The controller can do this because the centre of compliance is defined in software and is held constant only from the clearance hand-off onwards. For the opposite tilt sign, this trade-off remains a model-based design consequence.

5.10 Limitations

GREEN — sufficient. The limitations are grouped by source and state how each one changes the interpretation rather than listing generic caveats.

5.10.1 Experimental Generalisability

GREEN — sufficient. The missing opposite-sign and lever-length experiments are stated explicitly and connected to the claims they restrict.

The experimental scope is defined by the tested matrix. All commanded tilts had the same sign, and the compliance-centre sweep used three tangential levels on one axis at a time. These levels identify the favourable direction and the magnitude of its effect within the investigated range, but they do not identify an optimum lever length or the response to the opposite tilt sign.

5.10.2 Calibration and Measurement Uncertainty

GREEN — sufficient. The plane estimate, nominal tool transform, and model-based load are distinguished from direct measurements.

The primary alignment metric combines the end-effector pose with the independently calibrated physical-plane normal. Its absolute value therefore inherits uncertainty from both the nominal tool transform and the plane calibration. The reported normal load is obtained from the robot’s model-estimated external wrench and is not an independent force measurement.

5.10.3 Tool-Mount Compliance

GREEN — sufficient. The likely direction of the mount uncertainty and the anomalous repetition are discussed without assigning an unsupported sole cause.

The tool could rotate by approximately $\pm 2^\circ$ about y_{EE} relative to the gripper. For the tested mounting orientation, y_{EE} was approximately 25° from t_2 . Contact-induced rotation at the gripper–tool interface may therefore have been more influential in the t_2 conditions than in the t_1 conditions. The end-effector-based metric therefore describes the combined robot–gripper–tool response and may underestimate the physical rotation of the grinding face.

The 10° t_2 repetition that produced 0.00° of EE-inferred improvement is consistent with this explanation, although the tool mount cannot be identified as its sole cause. Retaining the run ensures that the mean and dispersion include the observed variability. The median within-setting standard deviation is 0.02° on t_1 and 0.03° on t_2 , and the two largest dispersions in the measurement set both occur at t_2 . Dispersion also grows with the size of the correction on both axes, however, so the tilt axis does not account for it alone.

5.10.4 Null-Space Validation and Timing Scope

GREEN — sufficient. The free-space, internal-disturbance, combined-mode, contact, and real-time timing boundaries are stated precisely.

The contact campaign used the combined null-space mode, so the individual null-space contributions cannot be separated from its alignment results. The hold comparison isolates them only in free-space hold under an internally commanded point-force equivalent. It does not establish their response to physical link contact. The controller operated through the nominal 1 kHz interface, but the experiments addressed motion response rather than worst-case callback execution time.

Chapter 6

Conclusion and Future Work

GREY — logical or evidential gap. Most conclusions follow Chapter 5, but the claimed separate pilot covering four positive tilt directions is not presented in the current methodology or results. That evidence must be added or the claim removed.

A real-time Cartesian impedance controller was implemented for passive alignment of a tilted grinding tool during contact with a plane. During set-up, the Cartesian stiffness and damping were shifted from a selected centre of compliance to the TCP. The resulting translation–rotation coupling allowed the normal contact force to generate a corrective moment.

The zero-tilt condition showed that the set-up procedure introduced a small systematic rotation even without a commanded angular offset. The tilted conditions were therefore interpreted relative to this baseline, without subtracting it from the reported measurements.

Within the investigated range, the in-plane compliance-centre position produced the largest measured variation in alignment. A lever directed to assist the required rotation increased the response by approximately one order of magnitude about t_1 and by a factor of about five about t_2 . The required lever direction reversed between the two tangent axes. When the lever opposed the required rotation, almost no alignment improvement was measured. The centre of compliance consequently acts as a directional bias whose sign must be selected from the initial tilt.

The stiffness effects were smaller and axis dependent. Raising the rotational stiffness of the tilted axis reduced the measured correction about t_1 , and the stiffest setting also pro-

duced the smallest response about t_2 . Reducing the perpendicular translational stiffness improved the response about t_2 , whereas the corresponding change about t_1 remained below the interpretation threshold. Displacement of the compliance centre along the tool axis produced no measurable change above that threshold in the tested geometry.

Tilts commanded about the tool-face axes produced responses between those of the two surface tangents. This agreement is consistent with an asymmetry associated mainly with the surface frame. In a separate pilot, selecting the lever perpendicular to the complete measured tilt direction improved alignment for the four positive tilt directions investigated. These results support selection of the lever from the measured tilt direction rather than use of one fixed placement for every approach.

The free-space hold study isolated the two null-space contributions under a repeatable internally commanded point-force equivalent. Projected damping reduced cumulative redundant joint excursion by approximately one quarter. Increasing the singular-value torque increased joint motion without increasing the final conditioning improvement. This comparison establishes the relative response of the two terms for the investigated hold condition.

The main contribution is the experimental connection between a spatially shifted Cartesian impedance and the contact geometry of a plane-alignment task. The axis-specific measurements show that compliance-centre placement had a substantially greater influence than the stiffness variations considered here. They also show that the sign of the induced moment is an essential part of that placement decision.

6.1 Limitations

GREEN — sufficient. The conclusion carries only the principal limitations and does not repeat the detailed mechanisms from Chapter 5.

6.1.1 Experimental Generalisability

GREEN — sufficient. The single-sign and single-configuration scope is stated without weakening the measured findings themselves.

The contact conclusions are limited to one robot, tool, plane surface, and parameter range. All commanded tilts had the same sign. The measured lever directions therefore cannot

be transferred to an initial error of the opposite sign without an additional experiment.

6.1.2 Calibration and Measurement Uncertainty

GREEN — sufficient. The EE-based angle and model-estimated load are correctly distinguished from independent physical measurements.

The alignment metric was calculated from the end-effector pose relative to the independently calibrated physical plane. The reported normal load was obtained from the robot’s model-estimated external wrench. It was not measured with an independent force/torque sensor.

6.1.3 Tool-Mount Compliance

GREEN — sufficient. The likely underestimation of grinding-face rotation is carried into the final interpretation with the affected axis identified.

Approximately $\pm 2^\circ$ of rotation was possible between the tool and the gripper about y_{EE} . The reported angle consequently describes the combined robot–gripper–tool response and may underestimate the physical rotation of the grinding face, particularly in the t_2 conditions.

6.1.4 Null-Space Validation

GREEN — sufficient. The distinction between combined contact operation and the separated free-space study remains explicit.

The two null-space terms were active together during the contact campaign and cannot be separated from those runs. The hold study isolated them in free-space hold under an internally commanded point-force equivalent. Its results do not establish their response to a physical link disturbance or during surface contact.

6.2 Future Work

GREEN — sufficient. The three proposed extensions follow directly from the main evidential gaps: tilt sign, independent tool-angle measurement, and physical combined-mode null-space validation.

Three extensions have priority. First, the lever study should include positive and negative initial tilts. This would determine whether selecting the lever from both the axis and sign of the measured error produces comparable improvement in both directions. A subsequent lever-length sweep could establish how the required magnitude depends on the initial angle.

Second, the grinding-face orientation should be measured independently of the end-effector. A rigid mount or an external angular measurement would separate motion within the gripper from the response of the impedance controller, particularly for tilts about t_2 .

Third, the null-space comparison should be repeated with a physical link disturbance and should include the combined damping-and-conditioning mode. This would determine whether the ordering observed with the internal point-force equivalent is retained under physical interaction and during surface contact.

Bibliography

- [1] Alin Albu-Schäffer, Christian Ott, Udo Frese, and Gerd Hirzinger. Cartesian impedance control of redundant robots: Recent results with the DLR-light-weight-arms. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, volume 3, pages 3704–3709, 2003. doi: 10.1109/ROBOT.2003.1242165.
- [2] Stefano Chiaverini, Bruno Siciliano, and Olav Egeland. Review of the damped least-squares inverse kinematics with experiments on an industrial robot manipulator. *IEEE Transactions on Control Systems Technology*, 2(2):123–134, 1994. doi: 10.1109/87.294335.
- [3] John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson Prentice Hall, Upper Saddle River, NJ, 3 edition, 2005.
- [4] Jacques Denavit and Richard S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955.
- [5] Alexander Dietrich, Christian Ott, and Alin Albu-Schäffer. An overview of null space projections for redundant, torque-controlled robots. *The International Journal of Robotics Research*, 34(11):1385–1400, 2015. doi: 10.1177/0278364914566516.
- [6] Ernest D. Fasse and Jan F. Broenink. A spatial impedance controller for robotic manipulation. *IEEE Transactions on Robotics and Automation*, 13(4):546–556, 1997. doi: 10.1109/70.611320.
- [7] Franka Emika GmbH. Franka control interface documentation. <https://frankaemika.github.io/docs/>, 2023. Accessed for robot interface, libfranka model access, and torque control documentation.

- [8] Franka Emika GmbH. libfranka: Cartesian Impedance Control Example. https://github.com/frankaemika/libfranka/blob/master/examples/cartesian_impedance_control.cpp, 2023. Official libfranka example, file `examples/cartesian_impedance_control.cpp`.
- [9] Richard Hartisch and Kevin Haninger. Flexure-based environmental compliance for high-speed robotic contact tasks, 2022.
- [10] Neville Hogan. Impedance control: An approach to manipulation—parts i–iii. *Journal of Dynamic Systems, Measurement, and Control*, 107(1):1–24, 1985.
- [11] International Federation of Robotics. World robotics 2023 report. Technical report, IFR Statistical Department, Frankfurt am Main, 2023.
- [12] Oussama Khatib. A unified approach for motion and force control of robot manipulators. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.
- [13] Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017. URL <https://hades.mech.northwestern.edu/images/7/7f/MR.pdf>. Free preprint.
- [14] Matthias Mayr and Julian M. Salt-Ducaju. A C++ implementation of a cartesian impedance controller for robotic manipulators. *Journal of Open Source Software*, 9(93):5194, 2024. doi: 10.21105/joss.05194.
- [15] Richard M. Murray, Zexiang Li, and S. Shankar Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.
- [16] James L. Nevins and Daniel E. Whitney. Computer-controlled assembly. *Scientific American*, 1989. Background source on robotic assembly and compliance.
- [17] Christian Ott, Alin Albu-Schäffer, Andreas Kugi, and Gerd Hirzinger. On the passivity-based impedance control of flexible joint robots. *IEEE Transactions on Robotics*, 24(2):416–429, 2008.
- [18] Mathew Jose Pollayil, Franco Angelini, Guiyang Xin, Michael Mistry, Sethu Vijayakumar, Antonio Bicchi, and Manolo Garabini. Choosing stiffness and damping

- for optimal impedance planning. *IEEE Transactions on Robotics*, 39(2):1281–1300, 2023. doi: 10.1109/TRO.2022.3216078.
- [19] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Springer, London, 2009.
- [20] Stefano Stramigioli, Claudio Melchiorri, and Stefano Andreotti. Intrinsically passive grasping and manipulation. *IEEE Transactions on Robotics and Automation*, 2000. URL <http://robby.caltech.edu/~jwb/courses/CDS270/papers/passive.grasp.pdf>. Open-access preprint.
- [21] Valeria Villani, Francesco Pini, Francesco Leali, and Cristian Secchi. Survey on human–robot collaboration in industrial settings: Safety, intuitive interfaces and applications. *Mechatronics*, 55:248–266, 2018. doi: 10.1016/j.mechatronics.2018.02.009.
- [22] Sen Wang, Guodong Chen, Hui Xu, and Zheng Wang. A robotic peg-in-hole assembly strategy based on variable compliance center. *IEEE Access*, 2019. doi: 10.1109/ACCESS.2019.2954459.
- [23] Paul C. Watson. Remote center compliance system. U.S. Patent 4,098,001, 1978. Assigned to The Charles Stark Draper Laboratory.

Appendix A

Controller Source Listings

GREEN — sufficient. The listings are abridged to the mathematical paths used by the thesis and connect each important equation to an implementation excerpt.

This appendix contains abridged source excerpts from the surface-grinding controller used in the reported experiments. Only the paths needed to connect the equations to the implementation are shown. The excerpts use fixed-size Eigen aliases such as `Vec3`, `Vec7`, `Mat3`, `Mat6x6`, and `Mat6x7`.

A.1 Surface Task Frame and Spatial Gains

The frame order is $[t_1, t_2, n_s]$. Because $t_2 = n_s \times t_1$, this column order is right-handed.

Listing A.1: Surface frame and gain transformation used by the controller.

```
1 | Mat3 makeSurfaceFrameFromNormalTangent(  
2 |     const Vec3& normal_input, const Vec3& tangent1_input) {  
3 |     const Vec3 normal =  
4 |         normalizedOrFallback(normal_input, Vec3(1.0, 0.0, 0.0));  
5 |     Vec3 tangent1 =  
6 |         tangent1_input - normal * normal.dot(tangent1_input);  
7 |     if (tangent1.norm() <= 1e-9) {  
8 |         Vec3 fallback(0.0, 1.0, 0.0);  
9 |         if (std::abs(normal.dot(fallback)) > 0.95) {  
10 |             fallback = Vec3(0.0, 0.0, 1.0);  
11 |         }  
12 |         tangent1 = fallback - normal * normal.dot(fallback);  
13 |     }  
14 |     tangent1.normalize();  
15 |     const Vec3 tangent2 = normal.cross(tangent1).normalized();  
16 |  
17 |     Mat3 R_surface;  
18 |     R_surface.col(0) = tangent1;  
19 |     R_surface.col(1) = tangent2;  
20 |     R_surface.col(2) = normal;  
21 |     return R_surface;  
22 | }  
23 |  
24 | Mat3 makeSpatialGainMatrix(  
25 |     const Vec3& diagonal_in_task_frame, const Mat3& R_task) {
```

```

26 |     return R_task * diagonal_in_task_frame.asDiagonal()
27 |         * R_task.transpose();
28 | }
    
```

A.2 Tool Orientation and Rotation Error

The physical tool axis is configured in the EE frame. The desired orientation is the minimum rotation of the captured start orientation that maps this axis onto the signed normal; it is not obtained by assigning the complete surface frame to the EE frame.

Listing A.2: Signed tool-axis target and restoring rotation-vector error.

```

1 | Vec3 desiredToolAxisInBase(
2 |     const Parameters& p, const Mat3& R_surface) {
3 |     const double sign = (p.tool_axis_target_sign >= 0.0) ? 1.0 : -1.0;
4 |     return sign * R_surface.col(2); // normal is column 2
5 | }
6 |
7 | Mat3 makeToolOrientationForAlignmentTarget(
8 |     const Parameters& p, const Mat3& R_surface,
9 |     const Mat3& R_start) {
10 |     const Vec3 a_start =
11 |         (R_start * p.tool_axis_ee.normalized()).normalized();
12 |     const Vec3 a_target =
13 |         desiredToolAxisInBase(p, R_surface).normalized();
14 |     return rotationBetweenUnitVectors(a_start, a_target) * R_start;
15 | }
16 |
17 | Vec3 orientationError(
18 |     const Mat3& R_current, const Mat3& R_desired) {
19 |     const Mat3 R_error = R_current.transpose() * R_desired;
20 |     const Eigen::AngleAxisd aa(R_error);
21 |     if (std::abs(aa.angle()) < 1e-9) {
22 |         return Vec3::Zero();
23 |     }
24 |     return R_current * aa.axis() * aa.angle(); // base components
25 | }
    
```

A.3 Geometric Clearance and Phase Targets

The leading point, edge, or face of the rectangular tool is selected along the descent direction. The transition into set-up is based on the exact most-leading corner height; the average of corners tied within 0.1 mm supplies the controlled feature and projected reference. The transition depends on this height; the external-force estimate does not enter the decision. The following excerpt is abridged from the phase switch.

Listing A.3: Phase-dependent tool-feature targets (abridged).

```

1 | const Vec3 descend_direction = -R_surface.col(2);
2 | double leading_projection = 0.0;
3 | const Vec3 tool_contact_offset_ee =
4 |     selectLeadingToolContact(R_EE, leading_projection);
5 | const Vec3 tool_contact =
6 |     p_EE + R_EE * tool_contact_offset_ee;
    
```

```

7 |
8 | case ControlPhase::kApproachDescend: {
9 |     desired.p_d = p_start + distance * descend_direction;
10 |     desired.pdot_d = params.descend_speed * descend_direction;
11 |
12 |     const Vec3 normal = -descend_direction;
13 |     const double leading_corner_height =
14 |         normal.dot(p_EE - surface_point) - leading_projection;
15 |     const double controlled_feature_height =
16 |         normal.dot(tool_contact - surface_point);
17 |     const bool clearance_reached =
18 |         leading_corner_height <= params.descend_surface_clearance;
19 |
20 |     if (clearance_reached && !gate_blocking) {
21 |         active_tool_contact_offset_ee = tool_contact_offset_ee;
22 |         first_contact_tcp = p_EE; // legacy name: captured at clearance
23 |         first_contact_point =
24 |             tool_contact - controlled_feature_height * normal;
25 |         setup_push_start = -controlled_feature_height;
26 |         R_contact_start = R_EE;
27 |         phase = ControlPhase::kSetUp;
28 |     }
29 |     break;
30 | }
31 |
32 | case ControlPhase::kSetUp: {
33 |     const double push =
34 |         setUpPush(params, phase_time, setup_push_start,
35 |             setup_push_velocity);
36 |     const Vec3 edge_target =
37 |         first_contact_point + push * descend_direction;
38 |     desired.p_d =
39 |         edge_target - R_contact_start * tool_contact_offset_ee;
40 |     desired.pdot_d = setup_push_velocity * descend_direction;
41 |     // Exit after minimum time on moment-delta norm, or on timeout.
42 |     break;
43 | }
44 |
45 | case ControlPhase::kGrind: {
46 |     // Keep the final set-up push and optionally add a smooth
47 |     // ping-pong displacement along tangent1 or tangent2.
48 |     edge_target = first_contact_point
49 |         + grind_push * descend_direction
50 |         + sweep_s * grind_tangent;
51 |     desired.p_d =
52 |         edge_target - R_contact_start * tool_contact_offset_ee;
53 |     desired.pdot_d = sweep_s_dot * grind_tangent;
54 |     break;
55 | }

```

The identifiers `first_contact_*` are retained source names. They are captured at the configured clearance, not at a force-detected first contact.

A.4 Centre of Compliance and the Offset Adjoint

The lever passed to the implementation points from the selected pole to the TCP:

$$r_c = p_{\text{TCP}} - p_c.$$

The same congruence is applied to stiffness and damping. In the active configuration, `edge_ref` and `tcp_ref` below are selected at the clearance transition and held constant. The implemented $r_c = p_{\text{TCP,clr}} - p_c$ is constant during set-up. The live option uses the

same expression with current references.

Listing A.4: Pole-to-TCP adjoint and congruence transform.

```

1 | Mat3 skewMatrix(const Vec3& r) {
2 |     Mat3 S;
3 |     S << 0.0, -r(2),  r(1),
4 |         r(2),  0.0, -r(0),
5 |         -r(1),  r(0),  0.0;
6 |     return S;
7 | }
8 |
9 | Mat6x6 offsetAdjoint(const Vec3& r_c) {
10 |     Mat6x6 A = Mat6x6::Zero();
11 |     A.block<3,3>(0,0) = Mat3::Identity();
12 |     A.block<3,3>(0,3) = skewMatrix(r_c);
13 |     A.block<3,3>(3,3) = Mat3::Identity();
14 |     return A;
15 | }
16 |
17 | Mat6x6 adjointTransformedGain(
18 |     const Mat6x6& gain_at_pole, const Vec3& r_c) {
19 |     const Mat6x6 A = offsetAdjoint(r_c);
20 |     return A.transpose() * gain_at_pole * A;
21 | }
22 |
23 | const Vec3 pole = edge_ref + params.coupled_pole_from_edge;
24 | const Vec3 r_c = tcp_ref - pole;
25 | K_tcp = adjointTransformedGain(
26 |     blockDiagonal(Kp_used, KR_used), r_c);
27 | D_tcp = adjointTransformedGain(
28 |     blockDiagonal(Dp_used, DR_used), r_c);
    
```

A.5 Task-Inertia Damping

The joint-space mass matrix is solved rather than explicitly inverted. A 10^{-9} diagonal regularisation is used only for the task-inertia calculation; it is not the null-space pseudoinverse cutoff.

Listing A.5: Task-inertia estimate and per-axis damping.

```

1 | const Mat7x6 Minv_Jt = mass.ldlt().solve(J.transpose());
2 | Mat6x6 Lambda_inv = J * Minv_Jt;
3 | Lambda_inv = 0.5 * (Lambda_inv + Lambda_inv.transpose());
4 | Lambda_inv.diagonal().array() += 1e-9;
5 | Mat6x6 Lambda =
6 |     Lambda_inv.ldlt().solve(Mat6x6::Identity());
7 |
8 | const Mat6x6 T_task = blockDiagonalRotation(R_task);
9 | const Mat6x6 Lambda_task =
10 |     T_task.transpose() * Lambda * T_task;
11 |
12 | for (int i = 0; i < 3; ++i) {
13 |     Dp(i) = 2.0 * zeta
14 |         * std::sqrt(Lambda_task(i, i) * Kp(i));
15 |     DR(i) = 2.0 * zeta
16 |         * std::sqrt(Lambda_task(i + 3, i + 3) * KR(i));
17 | }
    
```

Only finite, positive directional inertia values are used. Otherwise, the phase-specific fixed damping matrix is applied. Each inertia-scaled coefficient is bounded at 8000 in its corresponding damping unit.

A.6 Null-Space Torque

For $J \in \mathbb{R}^{6 \times 7}$, Eigen returns six singular values and a full 7×7 right-singular matrix. Column six in zero-based C++ indexing is therefore v_7 , the structural null vector; it is distinct from v_6 , the right-singular vector paired with $\sigma_{\min} = \sigma_6$.

Listing A.6: Truncated-SVD torque projector and the four-mode null-space law (abridged).

```

1 Eigen::JacobiSVD<Mat6x7> svd(
2     J, Eigen::ComputeFullU | Eigen::ComputeFullV);
3 const auto sigma = svd.singularValues(); // sigma_1 ... sigma_6
4 const double cutoff =
5     params.nullspace_svd_relative_tolerance * sigma.maxCoeff();
6
7 Mat7x6 J_pinv = Mat7x6::Zero();
8 for (int i = 0; i < 6; ++i) {
9     if (sigma(i) > cutoff) {
10         J_pinv += (1.0 / sigma(i))
11             * svd.matrixV().col(i)
12             * svd.matrixU().col(i).transpose();
13     }
14 }
15
16 Mat7x7 N_tau =
17     Mat7x7::Identity() - J.transpose() * J_pinv.transpose();
18 N_tau = 0.5 * (N_tau + N_tau.transpose());
19
20 const Vec7 dq_N = N_tau * dq;
21 const Vec7 tau_d =
22     damping_enabled ? -params.nullspace_damping * dq_N
23     : Vec7::Zero();
24
25 Vec7 n = svd.matrixV().col(6).normalized(); // v_7
26 const double alpha = std::abs(params.nullspace_alpha);
27 const double sigma_plus = sigmaMinAt(q + alpha * n);
28 const double sigma_minus = sigmaMinAt(q - alpha * n);
29 const double difference = sigma_plus - sigma_minus;
30
31 Vec7 tau_sigma = Vec7::Zero();
32 if (std::abs(difference) > params.nullspace_sigma_deadband) {
33     const double sign = (difference > 0.0) ? 1.0 : -1.0;
34     tau_sigma = params.nullspace_k_sigma
35         * N_tau * (sign * n);
36 }
37
38 switch (params.nullspace_mode) {
39     case NullspaceMode::kOff: return Vec7::Zero();
40     case NullspaceMode::kDampingOnly: return tau_d;
41     case NullspaceMode::kSigmaOnly: return tau_sigma;
42     case NullspaceMode::kDampingAndSigma: return tau_d + tau_sigma;
43 }
    
```

Here the code variable `alpha` represents α_{probe} and changes only the two sampled configurations. It does not multiply the active torque. Singular values at or below the cutoff are omitted from the inverse.

A.7 Core Control Callback

Listing A.7: Core impedance, coupled set-up, and torque composition.

```

1 const Vec3 e_p = desired.p_d - p_EE;
2 const Vec3 e_R = applyRotationalAxisMask(
    
```

```

3      params, orientationError(R_EE, R_d_used), R_surface);
4
5      Vec6 dx, dv;
6      dx << e_p, e_R;
7      dv << desired.pdot_d - pdot, -omega;
8
9      Vec6 wrench;
10     if (phase == ControlPhase::kSetUp &&
11         params.use_coupled_stiffness) {
12         // K_tcp and D_tcp come from block-diagonal, manual-pole,
13         // or saved-matrix selection.
14         wrench = K_tcp * dx + D_tcp * dv;
15     } else {
16         wrench.head<3>() =
17             Kp_used * e_p + Dp_used * dv.head<3>();
18         wrench.tail<3>() =
19             KR_used * e_R - DR_used * omega;
20     }
21
22     const Vec7 tau_task = J.transpose() * wrench;
23     const Vec7 tau_null =
24         computeNullspaceTorque(params, model, state, J, dq);
25     const Array7 coriolis_array = model.coriolis(state);
26     const Eigen::Map<const Vec7> coriolis(coriolis_array.data());
27     const Vec7 tau_cmd = tau_task + tau_null + coriolis;
    
```

The null-space function is called in every non-manual phase. Manual guidance uses its separate Coriolis-plus-joint-damping branch.

Appendix B

Recorded Experimental Data

GREEN — sufficient. The appendix records the signals actually used in the evaluation and distinguishes controller fields from recalculated physical-plane quantities.

The controller stores experimental signals in a preallocated ring buffer. This avoids file access and dynamic buffer growth inside the 1 kHz control callback. After the controller stops, the retained rows are written in chronological order to a CSV file.

B.1 Control Phases

GREEN — sufficient. The numeric phase values are mapped directly to their recorded state names.

The integer `phase` identifies the active part of the experiment:

phase	0	1	2	3	4	5
state	<code>approach_orient</code>	<code>approach_descend</code>	<code>set_up</code>	<code>grind</code>	<code>hold</code>	<code>manual_guide.</code>

B.2 Signals Used in the Evaluation

GREEN — sufficient. The table is a useful reproducibility record and keeps units, frames, and model-estimated quantities explicit.

Appendix B.2 lists the groups used to calculate the reported quantities. A suffix `_x, _y, _z` denotes base-frame components, and `_1 . . . _7` denotes joint components.

Table B.1: Recorded signal groups used in the evaluation.

Column group	Meaning
time, phase	Run time and active control state.
p_EE, p_d	Measured and desired TCP positions [m].
tool_contact, edge_target	Selected tool feature and its reference [m].
first_contact_tcp, first_contact	TCP position and projected surface reference captured at the geometric clearance transition; the transition is not force triggered.
e_p, e_R, pdot, pdot_d, omega	Cartesian errors and velocities. During set-up, e_R is measured relative to the clearance orientation selected at the transition and held constant during set-up.
alignment_angle_deg	Angle between the EE-inferred tool axis and the configured surface target [°]. The reported physical-plane angle is recalculated with n_{phys} .
f, m	Commanded Cartesian force [N] and moment [N m].
external_force, external_moment	Model-estimated external wrench from libfranka, expressed in the base orientation and referenced to stiffness frame K .
contact_force_bias, contact_moment_bias	External-wrench values captured at the clearance transition.
push	Set-up penetration reference or grind preload retained after set-up [m].
nullspace_mode, sigma_current	Selected null-space law and current smallest singular value.
nullspace_dq_1..7	Projected joint velocity [rad/s].
tau_sigma_1..7, tau_sigma_norm, tau_nullspace_norm	Singular-value torque and total null-space torque [N m].
nullspace_damping	Active projected damping gain [N m s/rad].

Continued on next page

Table B.1 continued

Column group	Meaning
disturbance_scale, disturbance_force_ base_x..z	Commanded hold-disturbance waveform and point force [N].
tau_disturbance_1..7	Equivalent joint torque $J_p^\top f_d$ in the internally commanded hold study [N m].
tau_cmd_1..7	Final commanded joint torque [N m].

B.3 Evaluation Quantities

GREEN — sufficient. The short summary routes each reported metric to its recorded source without reintroducing unused fields.

Contact alignment is calculated from the measured end-effector orientation and the independently calibrated physical-plane normal. The before value is taken at the start of set-up and the after value from the settled set-up interval. Selected-feature displacement is calculated from `tool_contact`, while the reported normal load is the bias-corrected model estimate.

For the internally commanded hold study, redundant motion is calculated by integrating the norm of the recorded projected joint velocity over the disturbance interval. The conditioning response is the final change in $\sigma_{\min}$, and Cartesian task retention is represented by the peak position-error norm.

Appendix C

Controller Parameters

GREEN — sufficient. The active geometry, phase gains, damping policy, timings, null-space values, and collision settings are collected in one reproducibility record.

This appendix records the active controller configuration used for the reported experiments. It is a reproducibility record for the implementation described in Chapters 3 and 4. The controller loads the disjoint files `common.txt`, `safety.txt`, `sequence.txt`, `hold.txt`, and `guidance.txt` in that order.

C.1 Surface and Tool Geometry

GREEN — sufficient. The configured geometry and initial joint vector are stated with their exact keys and units.

The plane satisfies

$$n_s^\top(p - p_s) = 0, \quad n_s = R_y(b)R_x(a)e_z,$$

and the surface-frame column order is $[t_1, t_2, n_s]$.

Table C.1: Surface and tool parameters.

Quantity	Configuration key	Value
Surface point	<code>surface_point</code>	$[0.55, -0.06, 0.025]$ m
Surface tilts	a, b	$0^\circ, 5^\circ$
Entered first tangent	<code>alignment_target_tangent1</code>	$[1, 0, 0]$
Tool axis in EE	<code>tool_axis_ee</code>	$+e_{z,EE}$
Signed alignment target	<code>tool_axis_target_sign</code>	$-n_s$
Grinding-face centre in EE	<code>tool_contact_face_center_ee</code>	$[0, 0, 0.020]$ m
Grinding-face size	width $\times$ length	0.040×0.120 m
Feature tie tolerance	<code>tool_contact_feature_tie_tolerance</code>	10^{-4} m
Initial-joint case	<code>q_init_case</code>	<code>tilted_tool</code>

The corresponding initial joint vector is

$$q_{\text{init}} = [-0.047771, 0.315777, -0.089546, -2.143518, 0.306679, 2.434032, 0.467459]^\top \text{ rad.}$$

C.2 Phase Sequence and Impedance

GREEN — sufficient. The phase-specific gain frames and transition parameters reconcile the values stated across Chapters 3 and 4.

Table C.2: Phase and mode gains.

Phase / quantity	Active stiffness and frame	Damping fallback
Approach translation	[150, 150, 150] N/m, surface $[t_1, t_2, n_s]$	[20, 20, 20] N s/m
Approach rotation	[90, 90, 90] N m/rad, surface $[t_1, t_2, n_s]$	[12, 12, 12] N m s/rad
Pre-set-up gate translation	5000 N/m, isotropic base	200 N s/m
Pre-set-up gate rotation	90 N m/rad, inherited approach rotation	inherited cached approach D_R
Set-up translation (compliance-centre source)	[2000, 2000, 350] N/m, base $[x, y, z]$	[10, 10, 175] N s/m
Set-up rotation (compliance-centre source)	[5, 5, 50] N m/rad, surface $[t_1, t_2, n_s]$	[10.01, 10.01, 10] N m s/rad
Grind translation (decoupled)	[2000, 2000, 350] N/m, base $[x, y, z]$	[10, 10, 175] N s/m
Grind rotation (decoupled)	[5, 5, 50] N m/rad, surface $[t_1, t_2, n_s]$	[10.01, 10.01, 10] N m s/rad
Hold translation	300 N/m, isotropic base	50 N s/m
Hold rotation	40 N m/rad, isotropic base	8 N m s/rad
Manual guidance	no Cartesian stiffness; joint coordinates	0.5 N m s/rad joint damping

The enabled pre-set-up gate overrides only translational stiffness. The optional pre-grind gate is disabled; if enabled, it remains in set-up and keeps the coupled pressed law rather than using the pre-set-up gate gains.

Table C.3: Phase-transition and trajectory parameters.

Quantity	Configuration key	Value
Orient minimum time	<code>approach_orient_min_time</code>	0.5 s
Tool-axis switch tolerance	<code>approach_orient_error_threshold</code>	0.035 rad
Descent speed / maximum distance	<code>descend_speed</code> , <code>descend_max_distance</code>	0.1 m/s, 0.640 m
Surface clearance	<code>descend_surface_clearance</code>	0.020 m
Set-up minimum time / timeout	<code>setup_min_time</code> , <code>setup_timeout</code>	2.0 s, 4.0 s
Set-up moment-change threshold	<code>setup_moment_threshold</code>	150 N m
Set-up push speed / endpoint	<code>setup_push_speed</code> , <code>setup_push_end</code>	0.050 m/s, 0.180 m
Grind sweep	<code>axis</code> , <code>amplitude</code> , <code>frequency</code>	t_2 , 0.030 m, 0.2 Hz
Pre-set-up / pre-grind gates	<code>pause_before_set_up</code> , <code>pause_before_grind</code>	enabled, disabled

C.3 Inertia-Scaled Damping and Centre of Compliance

YELLOW — weak or unclear. The active policy is complete, but the fitted damping factors are not listed per run and the $\zeta = 1$ heuristic is not a validated modal damping ratio.

Inertia-scaled damping follows eqs. (3.9) and (3.10). It is enabled for hold, approach, set-up, grind, and the pause gate. The approach factor is $\zeta = 1.9$, while set-up and grind use $\zeta = 1.0$. Hold uses one derived damping ratio for translation and rotation. The pause gate uses separate translational and rotational factors because the two gain blocks are expressed in different frames. The configured maximum coefficient is 8000, and the manual values in table C.2 provide the fixed damping when an inertia-scaled value is unavailable.

For set-up, the source damping blocks are calculated from TCP task inertia and then shifted to the selected centre of compliance. This preserves symmetry and positive semidefiniteness, but the factor $\zeta = 1$ remains a directional tuning heuristic; it does not establish a pole-level modal damping ratio for the coupled robot–contact system.

The coupled 6×6 law is enabled only during set-up. The active centre of compliance is selected manually and held constant:

$$p_c = p_{\Pi, \text{clr}} + [-0.04, 0.08, 0]^\top \text{ m}, \quad r_c = p_{\text{TCP}, \text{clr}} - p_c.$$

Here $p_{\Pi, \text{clr}}$ is the selected tool feature projected onto the surface at the 20 mm geometric clearance hand-off. The configuration key `first_contact_point` denotes this geometric reference rather than a force-detected contact edge. The TCP position is captured at the same hand-off, so this active r_c is constant throughout set-up. With `coupled_pole_manual=1`, the matrices constructed from this selected centre of compliance are the active set-up gains.

C.4 Null-Space and Operating Parameters

GREEN — sufficient. The selected mode, gains, probe settings, logging capacity, gripper state, and robot-side thresholds are stated explicitly.

The null-space settings are explained in section 4.4.5. The final two rows of Table C.4 are Franka collision/reflex thresholds configured through the libfranka collision-behaviour interface; they are neither commanded-torque saturation nor a software wrench limiter.

Table C.4: Null-space and operating parameters.

Parameter	Symbol / key	Value
Null-space mode	<code>nullspace_mode</code>	3 (damping + sigma)
Null-space damping	d_{null}	2.0 N m s/rad
Sigma torque magnitude	k_{σ}	1.0 N m
Probe distance	α_{probe}	0.08 rad
Sigma difference deadband	δ_{σ}	10^{-6}
Relative SVD cutoff	ρ	10^{-4}
Recorded-data sampling / capacity	<code>log_every_n_cycles</code> , rows	every callback (1), 120000
Run duration / stop	<code>experiment_duration</code>	0 s: indefinite until e+Enter
Gripper grasp	width, speed, force	0.070 m, 0.050 m/s, 60 N
Tool-mount rotational play	physical observation	approximately $\pm 2^\circ$ about y_{EE}
Manual-guidance damping	<code>manual_guidance_damping</code>	0.5 N m s/rad
Joint-side collision threshold	acceleration/nominal	140 N m on each axis
Cartesian collision threshold	acceleration/nominal	240 N or N m on each channel

Appendix D

Previous Contact-Alignment Campaign

GREY — logical or evidential gap. The comparison data are useful, but the seating-based calibration is described as a physical-plane estimate, whereas the main methodology distinguishes surface probing from grinding-face seating. That measurement identity must be reconciled.

This appendix preserves the completed contact-alignment campaign that preceded the axis-specific experiments. The measurements remain valid for the configurations in which they were obtained and serve as comparison data. They are separated here because the surface plane and commanded tool orientation were not independently parameterised: the controller used a configured plane with $a = 0^\circ$, $b = 5^\circ$, whereas three manual seatings estimated the physical plane as $a = -0.72 \pm 0.31^\circ$, $b = 14.99 \pm 1.91^\circ$. The resulting 10.01° tilt was approximately -0.72° about t_1 and $+9.99^\circ$ about t_2 .

D.1 Test Matrix

GREEN — sufficient. The older campaign is separated from the primary results and its five cases, sample counts, and purposes are recoverable.

The campaign contains 88 contact runs, arranged in the five cases shown in table D.1.

Table D.1: Previous contact-alignment cases.

Case	Test	Design	Purpose
A	Set-up settling	4, 8, and 12 s; one run per duration	Select a settled evaluation time.
B	Formulation equivalence	Decoupled command ($n = 5$) and zero-shift coupled command ($n = 3$)	Check agreement of equivalent impedance forms.
C	Nominal repeatability	Five nominal runs with a physical reset between runs	Establish the run-to-run interpretation scale.
D	Paired tipping stiffness	$K_{R,t_1} = K_{R,t_2} = 5, 15, 50 \text{ N m/rad}$; five runs per setting	Measure the response to a common tangent-axis stiffness.
E	Compliance-centre position	Edge check ($n = 3$); configured-normal sweep ($n = 24$); t_1 sweep ($n = 18$); t_2 sweep ($n = 12$)	Measure the response to the three pole coordinates.

D.2 Settling and Repeatability

YELLOW — weak or unclear. The repeatability sample is useful, but the settling-time choice rests on only one run at each duration and is weaker than the later three-repetition comparisons.

The 4 s set-up ended when the commanded press ramp completed and still contained the load overshoot. The estimated normal load relaxed by approximately 4.6 s. Re-evaluation of the longer traces at 5 s produced the residual changes in table D.2; all are below the campaign’s repeatability-based interpretation threshold.

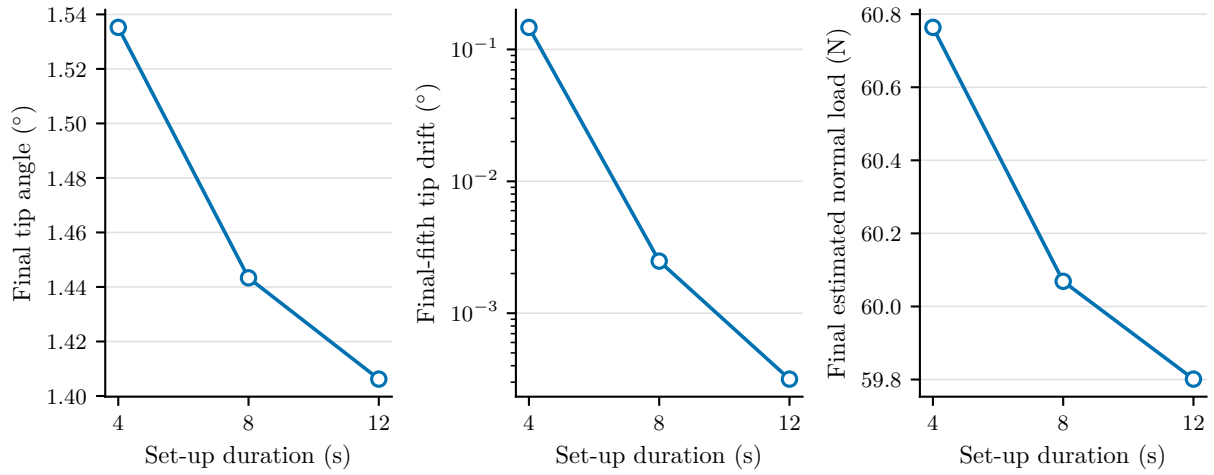

Figure D.1: Previous set-up settling comparison.

Table D.2: Previous residual changes after 5 s.

Trace	Tip difference	Alignment difference	Load difference
8-second run	0.006°	0.017°	0.09 N
12-second run	0.003°	0.002°	0.12 N

The decoupled and zero-shift coupled commands differed by 0.079° in final tip angle, or 0.34 times the larger group standard deviation. No discrepancy exceeding the measured repeatability was detected. Five nominal repetitions gave the dispersions in table D.3.

Table D.3: Previous nominal repeatability ($n = 5$).

Quantity	Mean	σ	2σ
Final tip angle (°)	1.474	0.194	0.388
Configured-normal change (°)	−0.714	0.294	0.588
Estimated normal load (N)	59.752	0.484	0.969
Edge travel (mm)	21.146	0.589	1.178

D.3 Paired Stiffness Results

GREEN — sufficient. The appendix correctly limits the result to a paired tangent-axis setting and does not reinterpret it as an isolated axis effect.

Both tangent-axis gains were changed together in Case D. The data therefore characterise the common setting $K_{R,t_1} = K_{R,t_2}$; they do not identify the separate sensitivities to the two axes. The imposed tilt was predominantly about t_2 , which further limits an axis-specific interpretation.

Table D.4: Previous paired-stiffness results ($n = 5$).

$K_{R,t_1} = K_{R,t_2}$ (N m/rad)	Tip ($^\circ$)	Estimated normal load (N)	Improvement ($^\circ$)
5	2.539 ± 0.132	60.27 ± 0.27	$+2.38 \pm 0.06$
15	1.841 ± 0.066	60.77 ± 0.22	$+1.59 \pm 0.04$
50	1.280 ± 0.032	60.53 ± 0.23	$+0.79 \pm 0.04$

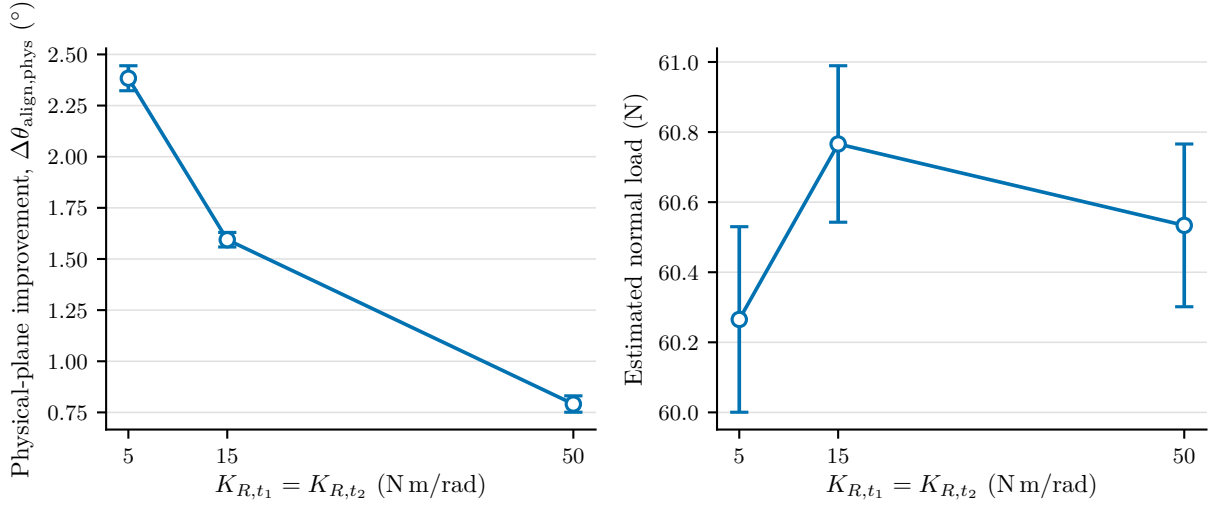


Figure D.2: Previous paired tangent-axis stiffness sweep.

Reducing both stiffnesses from 50 to 5 N m/rad increased the mean physical-plane alignment improvement from 0.79° to 2.38° . The mean model-estimated normal load remained between 60.27 and 60.77 N.

D.4 Compliance-Centre Results

GREY — logical or evidential gap. The additive fit is clearly labelled, but the cross-shaped sampling cannot identify a $t_1 t_2$ interaction and the predicted maximum has not been tested as a setting.

The position campaign contains 57 runs. An additive quadratic model fitted to the 54 coordinate-sweep runs accounts for $R^2 = 0.962$ of the response using t_1 , t_1^2 , t_2 , and t_2^2 .

It predicts a local maximum near $t_1 = +62$ mm, $t_2 = +30$ mm. This is a fitted location rather than a measured setting. Three separate contact-edge runs provide an additional comparison.

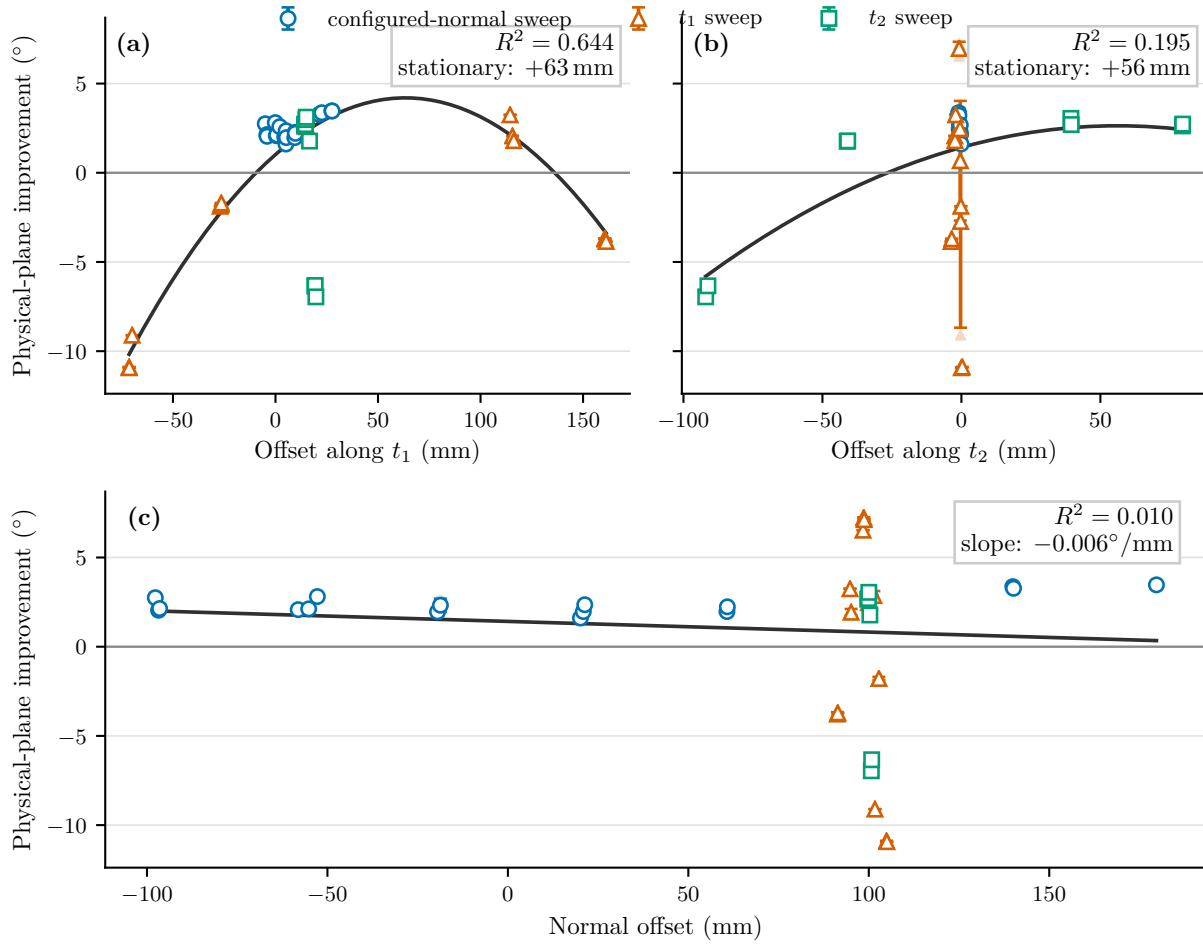


Figure D.3: Previous response by compliance-centre coordinate.

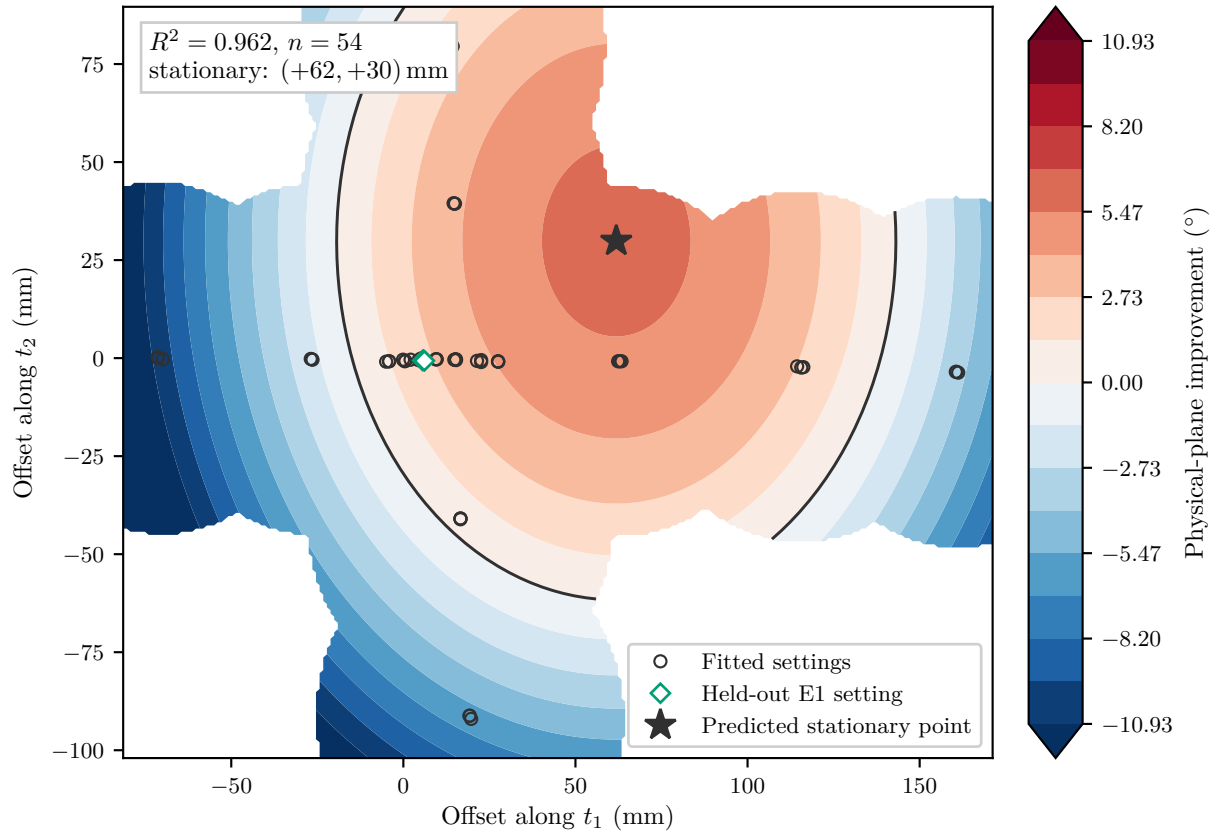


Figure D.4: Previous fitted in-plane response.

The response was non-monotonic. Unsuitable in-plane placements produced alignment improvements as low as -10.31° . Adding the physical normal coordinate to the in-plane model increased R^2 only from 0.962 to 0.966 over the investigated range. This is specific to the tested contact geometry and does not establish that the normal coordinate is generally irrelevant.

D.5 Interpretation Limits

GREY — logical or evidential gap. The limitations are substantial: uncertain calibration, surface motion, tool loosening, model-estimated load, incomplete two-dimensional sampling, and no transfer validation.

The physical plane was estimated by seating the tool face rather than from three calibrated probe points. Absolute residual angles therefore inherit an angular spread of approximately $\pm 1.9^\circ$, although the within-run improvement changes by less than 0.1° across the three estimates. The surface moved under load, the tool loosened during two repeated runs, and the reported contact load comes from the robot's model-estimated

external wrench rather than an independent force/torque sensor.

The compliance-centre samples form a cross rather than a complete two-dimensional grid, so the fitted model contains no $t_1 t_2$ interaction. The predicted local maximum and its transfer to another plane orientation, tool, surface, or robot configuration remain unverified.